

Contents

INTRODUCTION	3
1. PROJECT SYNOPSIS	4
1.1 Key Features	4
1.2 Technology Stack	4
1.3 Benefits	4
2. TECHNOLOGY STACK	5
2.1 Frontend Development	5
2.2 Backend Development	5
2.3 Development Tools	5
3. OBJECTIVE	6
3.1 Enhance Data Accuracy and Efficiency	6
3.2 Improve Decision-Making	6
3.3 Increase Operational Efficiency	6
3.4 Develop a User-Friendly System	6
4. KEY FEATURES	8
5. SCOPE AND FEATURES	10
5.1 Scope	10
5.2 Features	10
6. DATA FLOW DIAGRAM (DFD)	11
6.1 DFD – Level 0 (Context Diagram)	11
6.2 DFD – Level 1 (Detailed Diagram)	12
6.3 DFD – Level 2 (Detailed Internal Flow)	14
6.4 Initialization Process	15
6.5 User Interface	15
7. ENTITY RELATIONSHIP (ER) DIAGRAM	16
7.1 ER Diagram	16
7.2 Entities	17
7.3 Relationships	17
7.4 Data Flow	17
7.5 Database Design	17
8. DESIGN GOALS AND CONSTRAINTS	19
8.1 Design Goals	19
8.2 Design Constraints	19
9. METHODOLOGY	20
9.1 Prototype Model	20
9.2 Adopted Techniques and Tools	20
10. PROJECT REQUIREMENTS	21

10.1	Hardware Requirements	21
10.2	Software Requirements	21
10.3	Functional Requirements	21
10.4	Non-Functional Requirements	21
11.	PROJECT STRUCTURE	22
11.1	At the Root Level	22
11.2	Frontend-Related Components	22
11.3	Backend Logic	22
11.4	Database Layer	23
11.5	Overall Architecture	23
12.	PROJECT DEMONSTRATION	24
12.1	Application Startup	24
12.2	Login Dialog	24
12.3	Admin Dashboard	25
12.3.1	Student Admission	25
12.3.2	Edit Student Details	26
12.3.3	Staff Details	26
12.3.4	Edit Staff Details	27
12.3.5	Student Attendance	27
12.3.6	Staff Attendance	28
12.3.7	Marks	28
12.3.8	Marksheet	29
12.3.9	Student Record	30
12.3.10	Staff Record	30
12.3.11	Fees Module	31
13.	SOURCE CODE	32
14.	CONCLUSION	209

INTRODUCTION

The School Management System is a desktop-based application developed to simplify and automate the administrative and academic activities of educational institutions. Traditionally, schools rely on manual methods for managing student records, attendance, marks, and fee collection, which are time-consuming, error-prone, and difficult to maintain.

This project provides a centralized and digital solution to manage all essential school operations efficiently. It integrates multiple functionalities such as student admission, staff management, attendance tracking, academic record management, and fee collection into a single system. By using a structured database and an interactive graphical user interface, the system ensures accurate data handling and easy access to information.

The application is developed using Python with PySide6 for building the user interface and SQLite for managing the database. The system is designed to be user-friendly so that it can be easily used by school staff with minimal technical knowledge.

Overall, the School Management System aims to improve efficiency, reduce manual workload, and provide a reliable platform for managing school data in an organized manner. It serves as a practical solution for small to medium-sized institutions looking to digitize their operations.

1. PROJECT SYNOPSIS

The School Management System is a desktop-based application designed to automate and manage various administrative and academic activities within a school. The system provides a centralized platform to handle student records, staff information, attendance tracking, academic performance, and fee management efficiently.

Developed using Python and PySide6, the application offers a user-friendly graphical interface and uses SQLite as the backend database for structured data storage. The system reduces manual work, minimizes errors, and improves overall efficiency in managing school operations.

1.1 Key Features

- Student admission with auto-generated IDs
- Staff management with categorization
- Attendance tracking system
- Marks entry and report card generation
- Fee collection and history tracking

1.2 Technology Stack

- Python 3.12
- PySide6 (Python GUI Library – Qt Company)
- SQLite3 (Database)
- docxtpl (MS Word marksheet generation)

1.3 Benefits

- Reduces manual paperwork and errors
- Provides centralized data management
- Improves efficiency and productivity
- Enables quick report generation
- Easy to use with minimal training

2. TECHNOLOGY STACK

2.1 Frontend Development

The frontend of the application is developed using *PySide6*, which provides a powerful framework for building desktop GUI applications. It enables the creation of interactive forms, buttons, tables, and dialogs.



2.2 Backend Development



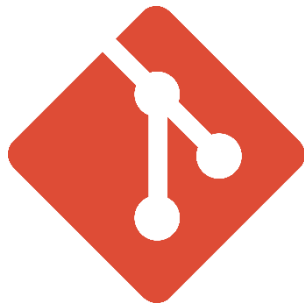
The backend logic is implemented using Python. It handles:

- Data processing
- Database operations
- Business logic
- Report generation

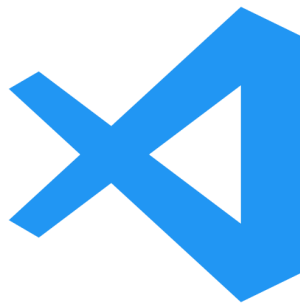
SQLite3 is used as the database for storing all records.

2.3 Development Tools

- VS Code
- Qt Designer (for UI design)
- Python Interpreter
- DB Browser (SQLite3 File Viewer)
- Git (Version Control)



Git (Version Control System)



VS Code



Python

3. OBJECTIVE

3.1 Enhance Data Accuracy and Efficiency

One of the primary objectives of the system is to improve the accuracy of data management by minimizing human errors associated with manual record-keeping. The system automates the process of storing and retrieving data related to students, staff, attendance, marks, and fees.

By using a structured database, it ensures:

- Consistent data storage
- Reduced duplication of records
- Faster data retrieval

3.2 Improve Decision-Making

The system aims to support better decision-making by providing organized and easily accessible information. Administrators can view and analyse records such as student performance, attendance history, and fee status.

This helps in:

- Monitoring academic progress
- Identifying irregular attendance
- Managing financial records effectively

Thus, the system enables informed and data-driven decisions.

3.3 Increase Operational Efficiency

Another key objective is to automate routine administrative tasks that are usually time-consuming when done manually. Tasks such as attendance marking, marks calculation, and fee tracking are handled efficiently by the system.

This results in:

- Reduced workload for staff
- Faster execution of daily operations
- Improved overall productivity

3.4 Develop a User-Friendly System

The system is designed with a focus on usability and simplicity. Using PySide6, the application provides a graphical user interface that is easy to understand and operate.

Key aspects include:

- Simple navigation between modules
- Clear input forms and data displays
- Minimal technical knowledge required

This ensures that the system can be effectively used by school staff without extensive training.

4. KEY FEATURES

The School Management System provides a set of integrated features that automate and simplify various school operations. These features are designed to improve efficiency, accuracy, and ease of use.

◆ 1. Student Management System

The system allows efficient handling of student-related information.

Features include:

- New student admission with auto-generated unique ID
- Storage of complete student details (personal + academic)
- Edit and update existing student records
- Search and filter student data

◆ 2. Staff Management System

This module manages both teaching and non-teaching staff.

Features include:

- Staff registration with unique ID generation
- Categorization (Teaching / Non-Teaching)
- Department-wise organization
- Update and manage staff details

◆ 3. Attendance Management System

The system provides a reliable way to track attendance.

Features include:

- Mark attendance (Present / Absent / Late / Excused)
- Record attendance based on date and class/department
- Store and retrieve attendance history
- Separate tracking for students and staff

◆ 4. Academic Management (Marks System)

This module handles student performance evaluation.

Features include:

- Subject-wise marks entry
- Automatic calculation of totals and percentages
- Storage of academic records
- Report card generation

◆ 5. Fee Management System

The system manages financial transactions efficiently.

Features include:

- Record student fee payments
- Maintain payment history
- Track fee status

- **Store date-wise transaction records**

◆ **6. Data Validation & Integrity**

The system ensures that only valid data is stored.

Features include:

- **Input validation for forms**
- **Prevention of duplicate or incorrect entries**
- **Structured database with relationships**

◆ **7. User-Friendly Interface**

The application provides an intuitive and easy-to-use interface.

Features include:

- **Simple navigation between modules**
- **Clear forms and data display**
- **Minimal technical knowledge required**

5. SCOPE AND FEATURES

5.1 Scope

The School Management System is designed to simplify and automate the daily operations of educational institutions. It provides a centralized platform to manage student records, staff information, attendance, academic performance, and fee collection.

The system is suitable for:

- **Schools**
- **Small to medium educational organizations**

Scope of the system includes:

- **Digital management of student and staff data**
- **Efficient handling of attendance and academic records**
- **Simplified fee management system**
- **Generation of reports and records**

Future Scope:

- **Multi-user login system**
- **Cloud database integration**
- **Mobile application support**
- **Advanced analytics and dashboards**

5.2 Features

The system provides multiple features covering different aspects of school management.

Student Management

- **Add new student with auto-generated ID**
- **Edit and update student details**
- **View and filter student records**

Staff Management

- **Staff registration with unique ID**
- **Categorization (Teaching / Non-Teaching)**
- **Department-wise organization**

Attendance Management

- **Mark attendance (Present/Absent/Late/Excused)**
- **Store daily attendance records**
- **View attendance history**

Academic Management

- **Enter subject-wise marks**
- **Automatic calculation of totals and percentages**
- **Generate report cards**

Fee Management

- **Maintain payment history**
- **Track student fee records**

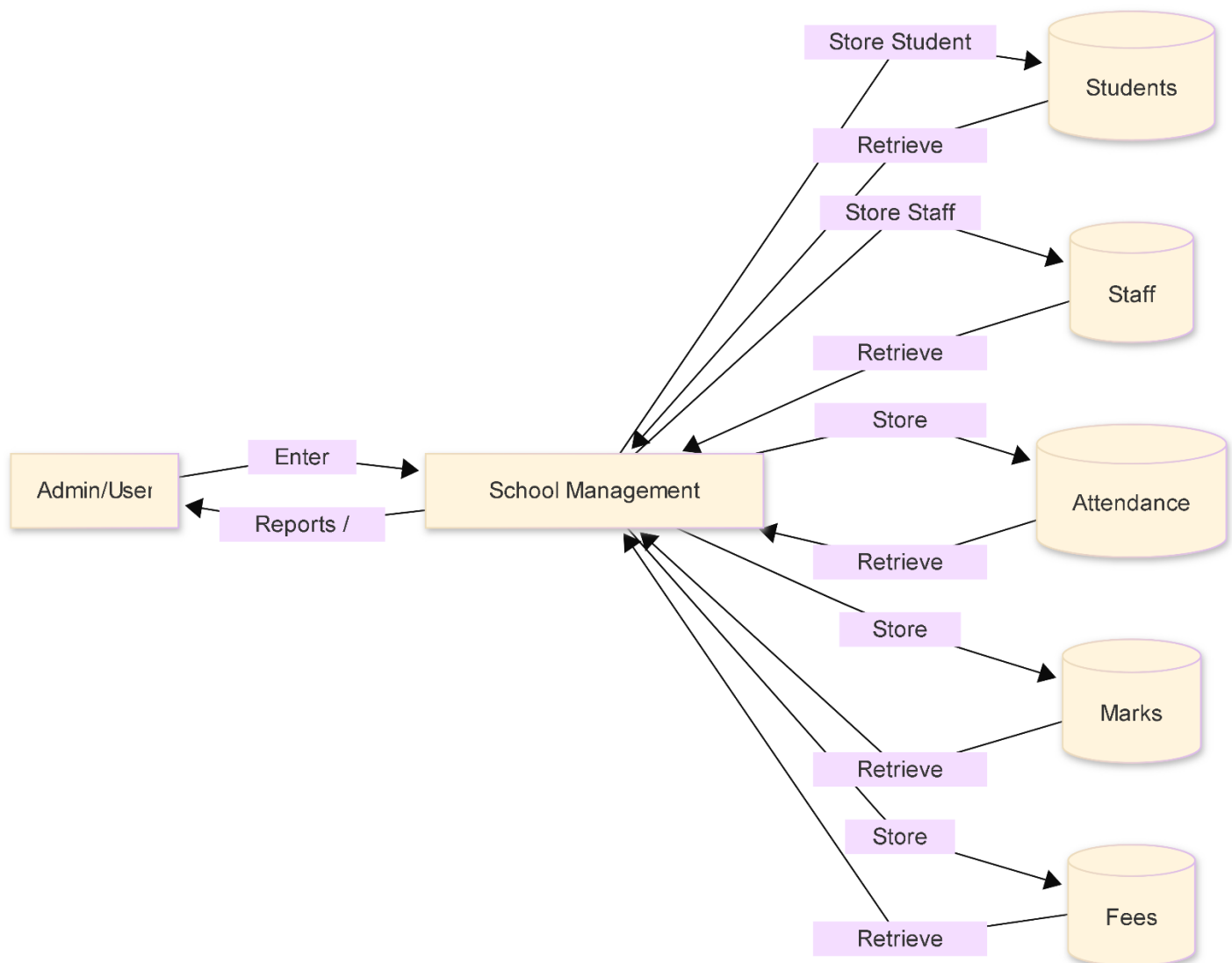
6. DATA FLOW DIAGRAM (DFD)

6.1 DFD – Level 0 (Context Diagram)

The Level 0 Data Flow Diagram, also known as the Context Diagram, represents the entire School Management System as a single process. It illustrates the interaction between the system and the external entity, which is the Admin/User.

In this level, the Admin/User provides inputs such as student details, staff information, attendance records, marks, and fee data to the system. The system processes this information and stores it in the respective databases, including Students, Staff, Attendance, Marks, and Fees databases. The system then retrieves the processed data and provides outputs in the form of reports, records, and summaries.

This diagram provides a high-level overview of the system without showing internal processes.



6.2 DFD – Level 1 (Detailed Diagram)

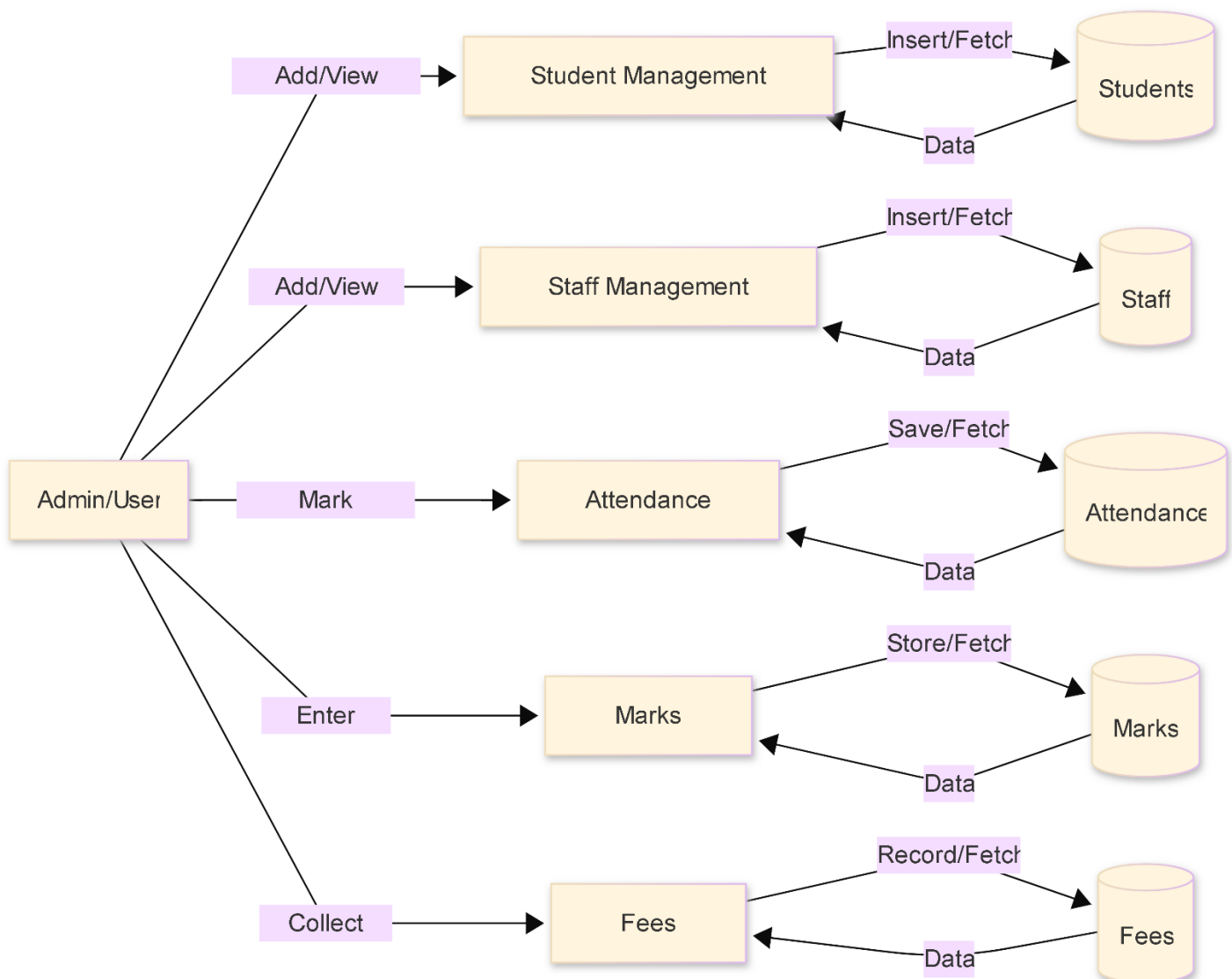
The Level 1 Data Flow Diagram breaks down the entire system into major functional modules. Each module represents a specific operation within the system.

The main modules identified are:

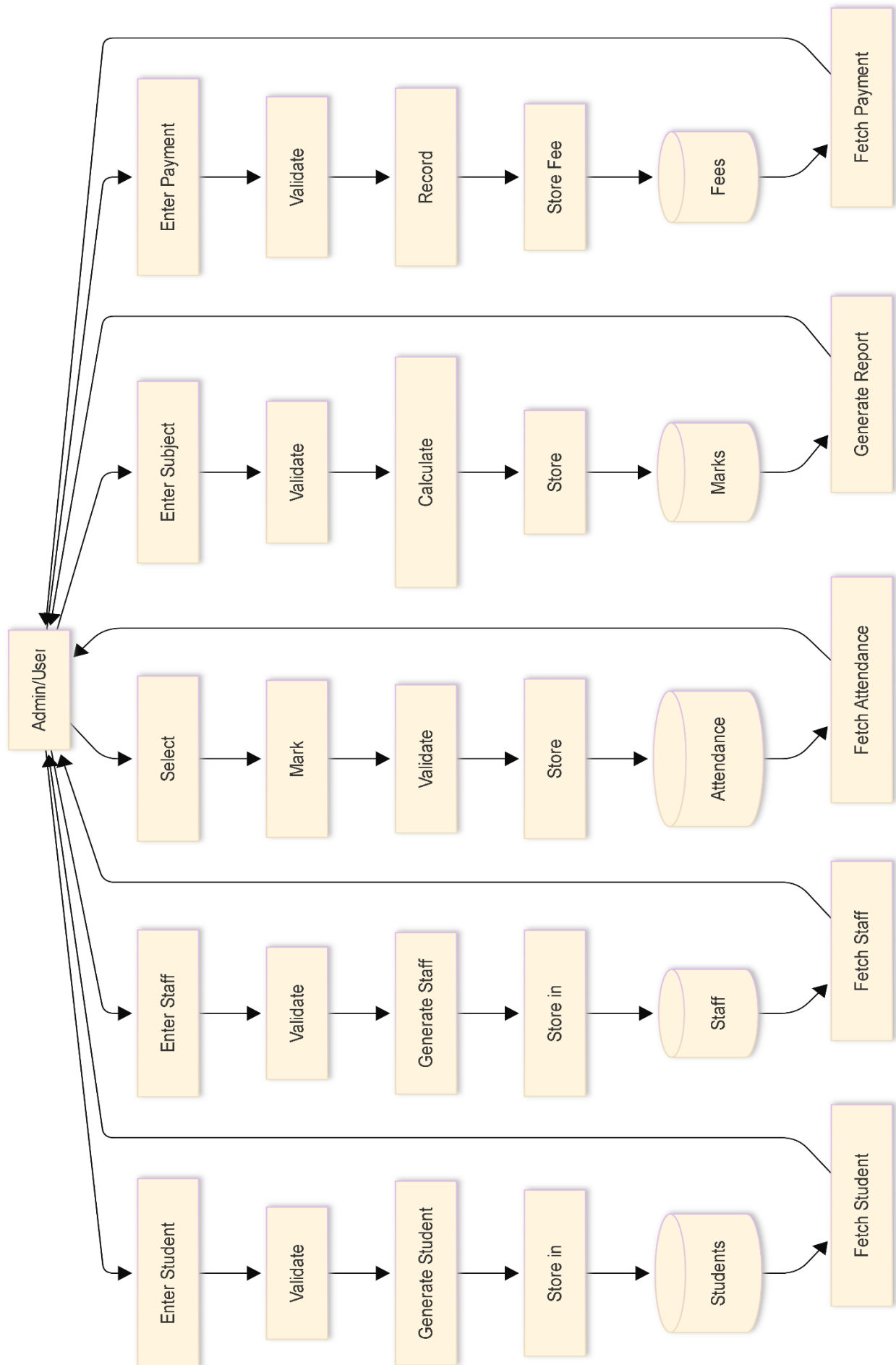
- Student Management Module
- Staff Management Module
- Attendance Module
- Marks Management Module
- Fees Management Module

Each module interacts with its corresponding database table. The Admin/User provides input to these modules, which process the data and store it in the database. The modules can also retrieve stored data when required.

This level provides a clearer understanding of how different parts of the system operate independently while being interconnected.



6.3 DFD – Level 2 (Detailed Internal Flow)



6.4 Initialization Process

When the application starts, the system initializes by establishing a connection with the SQLite database. It checks whether the required tables exist and loads the graphical user interface. Once initialized, the system is ready to accept user input and perform operations.

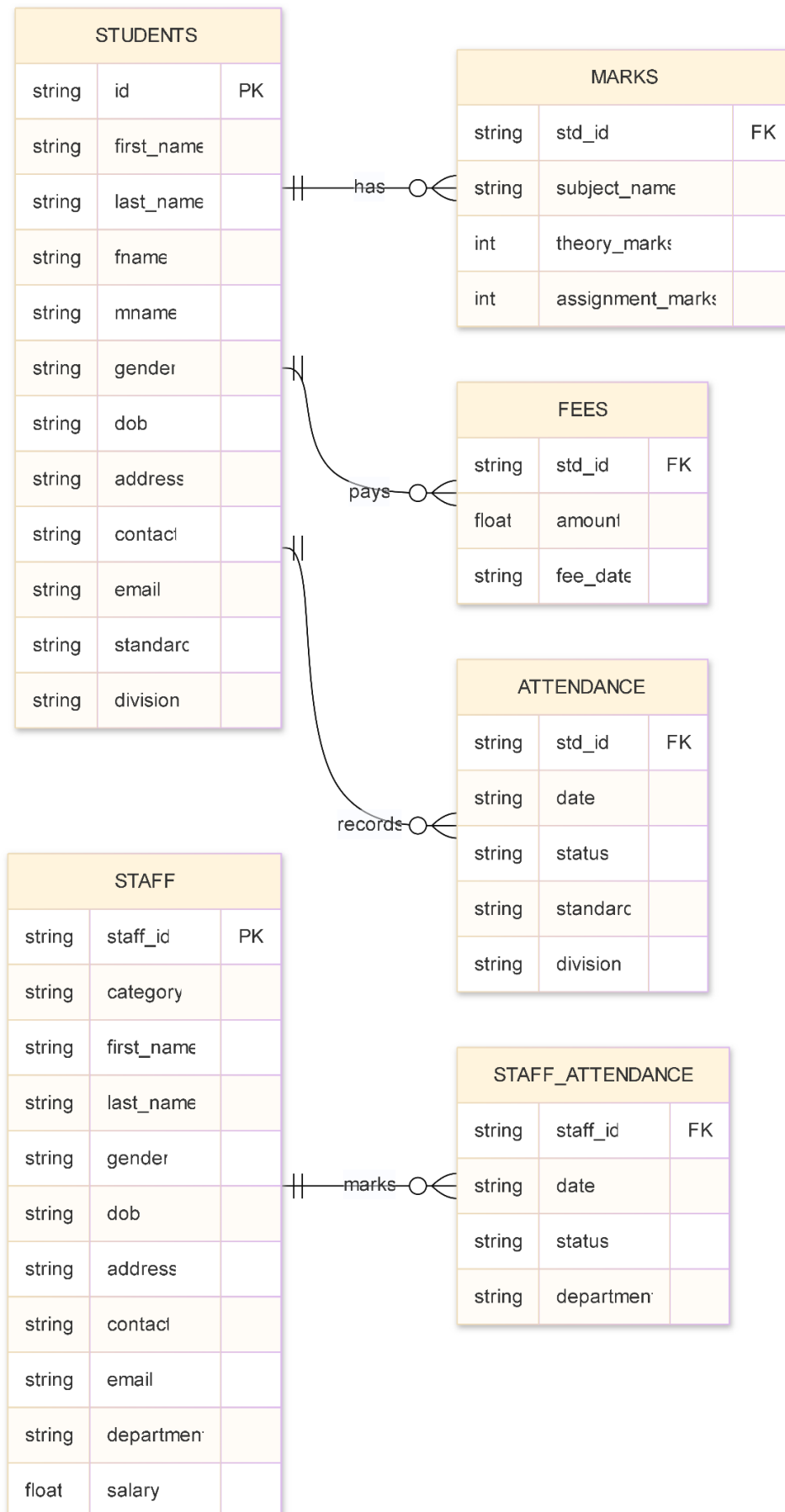
6.5 User Interface

The user interface of the system is designed using PySide6 and consists of multiple windows dedicated to different functionalities. These include the Admission Window, Staff Management Window, Attendance Window, Marks Management Window, and Fees Window.

Each window allows the user to interact with the system by entering data, viewing records, and generating reports. The interface is designed to be simple, intuitive, and user-friendly, ensuring ease of use for non-technical users.

7. ENTITY RELATIONSHIP (ER) DIAGRAM

7.1 ER Diagram



7.2 Entities

The system consists of multiple entities representing different components of school operations.

Main Entities:

- **Student** – Stores personal and academic details
- **Staff** – Stores teaching and non-teaching staff information
- **Marks** – Stores subject-wise performance of students
- **Fees** – Stores fee payment records
- **Attendance** – Stores student attendance
- **Staff Attendance** – Stores staff attendance

7.3 Relationships

The relationships define how entities are connected in the system.

- **Student** → **Marks** → **One-to-Many (1:N)**
- **Student** → **Fees** → **One-to-Many (1:N)**
- **Student** → **Attendance** → **One-to-Many (1:N)**
- **Staff** → **Staff Attendance** → **One-to-Many (1:N)**

Key Points:

- A single student can have multiple marks, fees, and attendance records
- A single staff member can have multiple attendance entries
- Relationships are maintained using:
 - **std_id** (for students)
 - **staff_id** (for staff)

7.4 Data Flow

Data flows through the system in a structured manner:

- User enters data via UI
- System processes and validates it
- Data is stored in the database
- Related records are linked using foreign keys
- Data is retrieved and displayed when required

Example:

Marks entered → linked using **std_id** → stored → shown in report card

7.5 Database Design

The database follows a relational model using SQLite.

Key Design Features:

- Separate tables for each entity
- Use of Primary Keys & Foreign Keys
- Maintains data consistency and integrity
- Supports multiple records per student/staff

Advantages:

- Avoids data redundancy

- **Improves performance**

8. DESIGN GOALS AND CONSTRAINTS

8.1 Design Goals

The design of the School Management System focuses on creating an efficient, reliable, and user-friendly application to manage school operations.

Key Design Goals:

- **Simplicity & Usability**
The system is designed with a clean and intuitive interface so that even non-technical users can operate it easily.
- **Modularity**
The application is divided into separate modules such as Student, Staff, Attendance, Marks, and Fees, making it easier to manage and extend.
- **Data Integrity & Accuracy**
Proper validation and structured database design ensure that data remains accurate and consistent.
- **Efficiency & Performance**
Use of SQLite ensures fast data access and minimal system resource usage.
- **Scalability (Basic Level)**
The system is designed in a way that new features (like authentication or online sync) can be added in the future.
- **Maintainability**
Code is organized into different classes and modules, making debugging and updates easier.

8.2 Design Constraints

While designing the system, certain limitations and constraints were considered:

- **Desktop-Based Limitation**
The application runs only on a local machine and is not accessible over the internet.
- **Single-User System**
The system is primarily designed for one admin user at a time, with no multi-user support.
- **SQLite Limitations**
SQLite is lightweight but not suitable for very large-scale or concurrent multi-user environments.
- **No Real-Time Synchronization**
Data is stored locally, so there is no cloud backup or real-time data sharing.
- **UI Limitations (PySide6)**
While functional, the UI may not be as modern as web-based interfaces.

9. METHODOLOGY

9.1 Prototype Model

The development of the School Management System follows the Prototype Model of the Software Development Life Cycle (SDLC). In this approach, an initial version (prototype) of the system is developed quickly to understand user requirements and gather feedback. Instead of building the entire system at once, the application was developed incrementally by creating and refining different modules such as Student Management, Staff Management, Attendance, Marks, and Fees.

Steps Followed in Prototype Model:

- **Requirement Gathering**
Basic requirements such as student registration, attendance tracking, and fee management were identified.
- **Quick Design (Prototype Creation)**
A basic UI using PySide6 was developed to visualize how the system would look and function.
- **Prototype Development**
Initial versions of modules (e.g., admission form, staff entry) were created with limited functionality.
- **User Feedback & Refinement**
The prototype was evaluated, and improvements were made based on usability and functionality requirements.
- **Final System Development**
After refining the prototype, the complete system was developed with full functionality and database integration.

9.2 Adopted Techniques and Tools

Techniques Used:

- **Prototyping Approach**
The system was built step-by-step, refining features after testing each module.
- **Modular Development**
Each feature (student, staff, attendance, etc.) was developed independently and then integrated.
- **Object-Oriented Programming (OOP)**
Classes and objects were used to organize code and improve reusability.
- **Database Normalization**
Data was structured into multiple tables to reduce redundancy and maintain consistency.

Tools & Technologies:

- **Python 3** – Core programming language
- **PySide6** – GUI development
- **SQLite3** – Database
- **docxtpl** – Report generation

- Qt Designer – UI design

10. PROJECT REQUIREMENTS

10.1 Hardware Requirements

The system does not require high-end hardware and can run on basic configurations.

- **Processor:** Intel i3 or higher
- **RAM:** Minimum 4 GB
- **Storage:** At least 500 MB free space
- **Display:** Standard monitor (1366×768 or higher)

10.2 Software Requirements

The application is developed using Python and requires the following software components:

- **Operating System:** Windows (*After some changes for paths we can port to Linux*)
- **Programming Language:** Python 3.12
- **GUI Framework:** PySide6
- **Database:** SQLite
- **Report Generation:** docxtpl

10.3 Functional Requirements

These define what the system should do:

- **Add, update, and view student records**
- **Manage staff details and departments**
- **Record and track attendance**
- **Enter and manage student marks**
- **Generate report cards**
- **Record and track fee payments**
- **Provide search and filtering options**

10.4 Non-Functional Requirements

These define system quality attributes:

- **Usability:** Simple and user-friendly interface
- **Performance:** Fast data processing using SQLite
- **Reliability:** Accurate data storage and retrieval
- **Security:** Basic data validation to prevent errors
- **Maintainability:** Modular code structure for easy updates

11. PROJECT STRUCTURE

11.1 At the Root Level

The root directory contains the main application entry point and core files required to run the system.

- **main.py**
Entry point of the application. Initializes the system and loads the main window.
- **database/**
Contains the SQLite database file (school.db) used for storing all records.
- **Ui_files/ (or individual window files)**
Contains different Python files for each ui module.
- **requirements.txt (optional)**
Lists all dependencies required to run the project.

11.2 Frontend-Related Components

The frontend is developed using PySide6 and consists of multiple windows and UI components.

- **Admission Window**
Handles student registration and data entry.
- **Staff Window**
Manages staff details and categorization.
- **Attendance Window**
Used to mark and track attendance.
- **Marksheet Window**
Handles marks entry and report generation.
- **Fees Window**
Manages fee collection and payment history.

Each window:

- Contains UI elements (forms, buttons, tables)
- Handles user interaction
- Communicates with backend logic

11.3 Backend Logic

The backend is implemented using Python and handles all processing tasks.

- Database operations (Insert, Update, Delete, Fetch)
- Business logic (ID generation, validation, calculations)
- Data processing and formatting

11.4 Database Layer

The system uses a relational database:

- **Database: SQLite**
- **Stores structured data in tables:**
 - **Students**
 - **Staff**
 - **Marks**
 - **Fees**
 - **Attendance**
 - **Staff Attendance**

Ensures:

- **Data consistency**
- **Fast access**
- **Lightweight storage**

11.5 Overall Architecture

The system follows a simple layered architecture:

User Interface (PySide6)



Application Logic (Python)



Database (SQLite)

Flow:

- **User interacts with UI**
- **Logic processes data**
- **Data is stored/retrieved from database**

12. PROJECT DEMONSTRATION

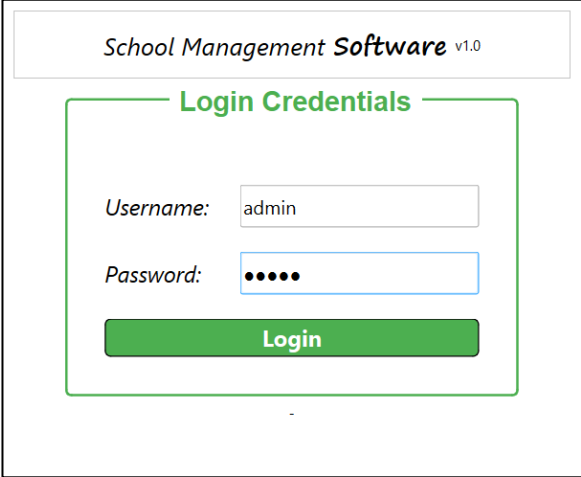
The School Management System demonstrates how various school operations can be performed efficiently using a single desktop application. The system is divided into multiple modules, each handling a specific functionality.

12.1 Application Startup

The application is launched using the *main.py* file, which acts as the entry point of the system. When the program starts, it initializes all required components and establishes a connection with the *SQLite* database.

The system checks for the availability of required tables and prepares the environment for execution. After successful initialization, the main interface or login window is displayed to the user.

12.2 Login Dialog

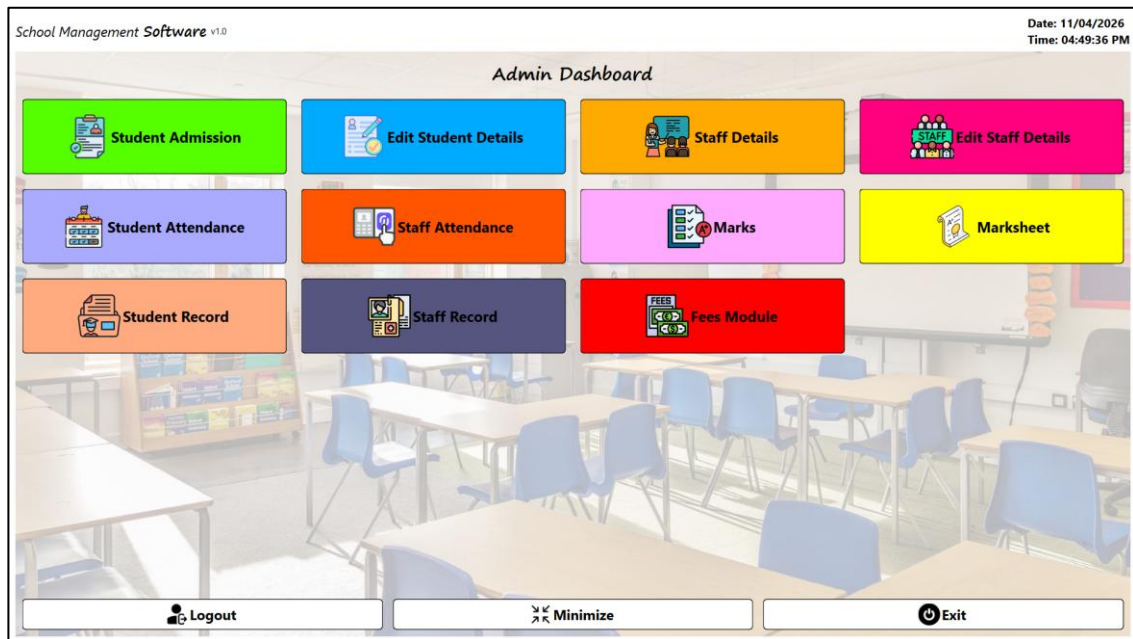


The login dialog is the first interactive screen presented to the user. It ensures that only authorized users can access the system.

Functions:

- Accepts username and password
- Validates user credentials
- Grants access to the system upon successful login

12.3 Admin Dashboard



After successful login, the user is directed to the Admin Dashboard. This acts as the central control panel of the system, providing access to all modules.

Key Features:

- Navigation to different modules
- Centralized control of operations
- Quick access to system functionalities

12.3.1 Student Admission

	Student ID	First Name	Last Name	ather's Nam	lother's Nam	Gender	Date of Birth	Address	ntact Numb	Email	Class	Section
1	S-1A2B3D	Aryan	Sharma	Ramesh ...	Pooja ...	Male	2016-01-15	10, MG Roa...	9876543200	aryan.shar...	Class 1	A
2	S-2D3E4G	Aditi	Verma	Sanjay Verma	Meena ...	Female	2016-03-22	12, ...	9876543201	aditi.verma...	Class 1	A
3	S-3G4H5J	Kabir	Gupta	Vikash Gupta	Nidhi Gupta	Male	2015-08-10	67, Navi ...	9876543202	kabir.gupta...	Class 1	B
4	S-4J5K6M	Sara	Mehta	Harish Mehta	Geeta Mehta	Female	2015-11-30	21, Salt Lak...	9876543203	sara.mehta...	Class 1	B
5	S-5M6N7P	Vihaan	Patel	Rajesh Patel	Anita Patel	Male	2016-05-25	10, Ashok ...	9876543204	vihaan.pate...	Class 1	A

This module is used to register new students in the system.

- Enter student details (name, contact, class, etc.)
- Automatic generation of Student ID
- Save data to the database

12.3.2 Edit Student Details

Edit Student Details

Student ID	S-1A2B3D	Address	10, MG Road, Delhi
First Name	Aryan	Contact No.	9876543200
Last Name	Sharma	Email	aryan.sharma@example.com
Father's Name	Ramesh Sharma	Standard	Class 1
Mother's Name	Pooja Sharma	Division	A
Gender	Male		
DOB	2016-01-15		

Fetch Data **Save Details**

This feature allows modification of existing student records.

- Search student using ID or filters
- Update necessary details
- Save updated information

12.3.3 Staff Details

Enter Staff Details

Staff ID	T-ED7R9	Address	
Category	Teaching Staff	Contact No.	
First Name		Email	
Last Name		Department	Primary - 1 to 5
Gender	Male	Salary	
DOB			

Save Details

	Staff ID	Category	First Name	Last Name	Gender	DOB	Address	Contact	Email	Department	Salary
1	T-101A	Teaching Staff	Vikram	Sharma	Male	1980-05-12	10, Karol ...	9876543301	vikram.shar...	Primary - 1 t...	70000.0
2	T-102B	Teaching Staff	Meera	Reddy	Female	1985-08-20	21, Banjara ...	9876543302	meera.reddy...	Primary - 1 t...	62000.0
3	T-103C	Teaching Staff	Arjun	Kapoor	Male	1982-11-03	30, Andheri ...	9876543303	arjun.kapoor...	Middle High ...	62000.0
4	T-104D	Teaching Staff	Nandini	Iyer	Female	1990-04-15	12, Adyar, ...	9876543304	nandini.iyer...	Middle High ...	58000.0
5	T-105E	Teaching Staff	Ramesh	Mishra	Male	1978-09-29	35, ...	9876543305	ramesh.mish...	Secondary - ...	55000.0

This module is used to add and manage staff information.

- Enter staff details
- Assign category (Teaching / Non-Teaching)
- Store records in database

12.3.4 Edit Staff Details

MainWindow

Edit Staff Details

Staff ID: T-101A Address: 10, Karol Bagh, Delhi

Category: Teaching Staff Contact No.: 9876543301

First Name: Vikram Email: vikram.sharma@example.com

Last Name: Sharma Department: Primary - 1 to 5

Gender: Male Salary: 70000.0

DOB: 1980-05-12

Fetch Data Update Details

Allows updating of staff information.

- Search staff records
- Modify details such as department or contact
- Save updated data

12.3.5 Student Attendance

Student Attendance

Standard: Class 1 Division: A

	Student ID	Name	Class	Status
1	S-1A2B3D	Aryan Sharma	Class 1 - A	Absent
2	S-2D3E4G	Aditi Verma	Class 1 - A	Present
3	S-5M6N7P	Vihaan Patel	Class 1 - A	Present

Fetch Attendance Save Attendance

This module is used to mark and track student attendance.

- Select class and date
- Mark attendance status
- Save attendance records

12.3.6 Staff Attendance

The screenshot shows a web application window titled "Staff Attendance". It features a "Staff Attendance" tab and a "Department" dropdown menu set to "Primary - 1 to 5". Below this is a table with columns: Staff ID, Staff Role, Staff Name, Date, and Attendance. Two rows are visible: Staff ID T-101A, Staff Role Vikram Sharma, Staff Name Primary - 1 to 5, Date Present, and Attendance; and Staff ID T-102B, Staff Role Meera Reddy, Staff Name Primary - 1 to 5, Date Late, and Attendance. To the right of the table is a calendar for April 2026, with the 11th highlighted. At the bottom right are two buttons: "Fetch Attendance" and "Save Attendance".

Staff ID	Staff Role	Staff Name	Date	Attendance
1 T-101A	Vikram Sharma	Primary - 1 to 5	Present	
2 T-102B	Meera Reddy	Primary - 1 to 5	Late	

This module manages attendance for staff members.

- Select department/date
- Mark attendance
- Store records for future reference

12.3.7 Marks

The screenshot shows a web application window titled "MainWindow". It features a "Marks Pages" tab. Below this is a "Student ID" input field with the value "S-1A2B3D". Below that is a "Subject Details" section with a dropdown menu set to "GK" and two input fields with values "50" and "10". A "Publish" button is to the right of these fields. Below this is a table with columns: Subject, Theory, and Practical. Four rows are visible: 1 Mathematics, 2 Science, 3 Social Science, and 4 GK. At the bottom right are two buttons: "Fetch Details" and "Save & Exit".

Subject	Theory	Practical
1 Mathematics	79	20
2 Science	70	0
3 Social Science	70	20
4 GK	50	10

This module allows entry of academic marks.

- Enter subject-wise marks
- Validate input data
- Store marks in database

12.3.8 Marksheet

Marksheet Generator

Student ID: S-1A2B3D

Fetch Data Print Marksheet

Student ID: S-1A2B3D
Student Name: Aryan Sharma
Class: Class 1
Section: A

	Subject	Theory Marks	Assignment Marks
1	Mathematics	79	20
2	Science	70	0
3	Social Science	70	20

AutoSave Off marksheet_S-1A2B3D.docx Saved to this PC Search

File Home Insert Draw Design Layout References Mailings Review View Help Easy Syntax Highlighter Table Design Table Layout Comments Editing Share

Calibri Light (Header) 13 A+ Aa B I U x x' A- Paragraph Styles Normal No Spacing Heading Heading 2 Title Find Replace Select Add-ins

Parent's Pride Public School
Add: Sadar Bazar, Karnal, Haryana, 133001
Ph: 098 15 946667

Student Name: Aryan Sharma Standard: Class 1
Father's Name: Ramesh Sharma Division: A
Mother's Name: Pooja Sharma DOB:
Student ID: S-1A2B3D

<<ACADEMIC REPORT>>

Subject Name	Theory	Maximum	Practical	Maximum	Total	Grade
Mathematics	79	70	20	30	99	A+
Science	70	70	0	30	70	B
Social Science	70	70	20	30	90	A+

Marks Obtained: 259
Percentage: 86.33
Result: Pass A

Parent's Sign Teacher's Sign Principal's Sign

Page 1 of 1 63 words English (India) Accessibility: Investigate Focus 90%

This feature generates report cards based on stored marks.

- Calculate total and percentage
- Display results
- Generate printable report

12.3.9 Student Record

MainWindow

Student Records

Student ID

Standard

Division

	Student ID	First Name	Last Name	Father's Name	Mother's Name	Gender	Date of Birth	Address	Contact Number	Email	Class	Section
1	S-1A2B3D	Aryan	Sharma	Ramesh ...	Pooja Sharma	Male	2016-01-15	10, MG Roa...	9876543200	aryan.shar...	Class 1	A
2	S-2D3E4G	Aditi	Verma	Sanjay Verma	Meena ...	Female	2016-03-22	12, ...	9876543201	aditi.verma...	Class 1	A
3	S-5M6N7P	Vihaan	Patel	Rajesh Patel	Anita Patel	Male	2016-05-25	10, Ashok ...	9876543204	vihaan.patel...	Class 1	A

Fetch Details

This module displays all student records.

- View complete student list
- Apply filters and search
- Access detailed information

12.3.10 Staff Record

MainWindow

Staff Records

Staff ID

Department

	Staff ID	Category	First Name	Last Name	Gender	DOB	Address	Contact	Email	Department	Salary
1	T-101A	Teaching Staff	Vikram	Sharma	Male	1980-05-12	10, Karol Bag...	9876543301	vikram.sharm...	Primary - 1 t...	70000.0
2	T-102B	Teaching Staff	Meera	Reddy	Female	1985-08-20	21, Banjara ...	9876543302	meera.reddy...	Primary - 1 t...	62000.0

Fetch Details

This module displays staff information.

- View staff details
- Filter by department
- Access complete records

12.3.11

Fees Module

Student Fees

Student ID

Amount

	Fee ID	Student ID	First Name	Last Name	Amount	Fee Date
1	S-1A2B3D	S-1A2B3D	Aryan	Sharma	1500.0	2023-06-15
2	S-2D3E4G	S-2D3E4G	Aditi	Verma	1500.0	2023-06-15
3	S-3G4H5J	S-3G4H5J	Kabir	Gupta	1600.0	2023-06-16
4	S-4J5K6M	S-4J5K6M	Sara	Mehta	1600.0	2023-06-16
5	S-5M6N7P	S-5M6N7P	Vihaan	Patel	1500.0	2023-06-15
6	S-6P7Q8S	S-6P7Q8S	Isha	Reddy	1700.0	2023-06-17
7	S-7S8T9U	S-7S8T9U	Krish	Singh	1700.0	2023-06-17
8	S-8V9W1Y	S-8V9W1Y	Tara	Nair	1800.0	2023-06-18

Fetch Fees History

Save & Exit

This module handles fee collection and tracking.

- Enter payment details
- Store transaction records
- View payment history

13. SOURCE CODE

13.1 Main Application File (main.py)

```
from PySide6.QtCore import (QCoreApplication, QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt, QTimer, QStringListModel)
from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
    QFont, QFontDatabase, QGradient, QIcon,
    QImage, QKeySequence, QLinearGradient, QPainter,
    QPalette, QPixmap, QRadialGradient, QTransform, QMouseEvent)
from PySide6.QtWidgets import (QApplication, QFormLayout, QGridLayout,
    QGroupBox,
    QHBoxLayout, QLabel, QLineEdit, QMainWindow,
    QPushButton, QSizePolicy, QSpacerItem, QStackedWidget,
    QVBoxLayout, QWidget, QMessageBox, QHeaderView,
    QTableWidgetItem, QComboBox)

from ui_login import Ui_MainWindow as login
from ui_main import Ui_MainWindow as main
from ui_admission import Ui_MainWindow as admission
from ui_edit_stud import Ui_MainWindow as editstud
from ui_stud_attend import Ui_MainWindow as std_attend
from ui_marksheet import Ui_MainWindow as marksheet
from ui_staff_attend import Ui_MainWindow as staff_attend

from ui_marks import Ui_MainWindow as marks
from ui_staff import Ui_MainWindow as staff
from ui_edit_staff import Ui_MainWindow as editstaff
from ui_stud_fees import Ui_MainWindow as fees

from ui_std_records import Ui_MainWindow as stud_rec
from ui_staff_rec import Ui_MainWindow as staff_rec

import sys
import string
import random
import subprocess
import os
import sqlite3
from docxtpl import DocxTemplate

class LoginWindow(QMainWindow):
    def __init__(self):
        super(LoginWindow,self).__init__()
        self.ui = login()
        self.ui.setupUi(self)
        self.setWindowFlags(Qt.FramelessWindowHint | Qt.Window)
        self.setWindowModality(Qt.WindowModality.ApplicationModal)
```



```

self.show()
self.ui.username_box.returnPressed.connect(self.ui.password_box.setFocus)
self.ui.password_box.returnPressed.connect(self.loginFunction)

self.conn = None
self.cursor = None

self.ui.login_btn.clicked.connect(self.loginFunction)

def loginFunction(self):
    username = self.ui.username_box.text()
    password = self.ui.password_box.text()
    self.conn = sqlite3.connect("database/school.db")
    self.cursor = self.conn.cursor()
    self.cursor.execute("""
        SELECT password FROM users WHERE username = ?
        """, (username,))
    result = self.cursor.fetchone()

    if result:
        password_g = result[0]
        # Verify the entered password against the stored hash
        if password_g == password:
            QMessageBox.information(self, "Login Successful", "Welcome!")
            self.ui.username_box.clear()
            self.ui.password_box.clear()
            self.close()
            self.conn.close()
            mainApp.showFullScreen()
        else:
            QMessageBox.warning(self, "Login Failed", "Invalid username or password.")
    else:
        QMessageBox.warning(self, "Login Failed", "No Username Found")

def mousePressEvent(self, event: QMouseEvent):
    """Store the position when the mouse is pressed."""
    if event.button() == Qt.LeftButton:
        self.old_pos = event.globalPosition()

def mouseMoveEvent(self, event: QMouseEvent):
    """Enable dragging the frameless window."""
    if self.old_pos is not None:
        delta = event.globalPosition() - self.old_pos
        self.move(self.x() + delta.x(), self.y() + delta.y())
        self.old_pos = event.globalPosition()

def mouseReleaseEvent(self, event: QMouseEvent):

```

```

        """Reset the position after releasing the mouse."""
        if event.button() == Qt.LeftButton:
            self.old_pos = None

class MainWindow(QMainWindow):
    def __init__(self):
        super(MainWindow,self).__init__()
        self.ui = main()
        self.ui.setupUi(self)

        self.windows = {}

        self.ui.logout_btn.clicked.connect(lambda : (mainApp.close(),loginApp.show()))
        self.ui.minimize_btn.clicked.connect(lambda : mainApp.showMinimized())
        self.ui.exit_btn.clicked.connect(lambda : mainApp.close())


        self.ui.admission_btn.clicked.connect(lambda: self.show_window("admission",
AdmissionWindow))
        self.ui.edit_btn.clicked.connect(lambda: self.show_window("edit_student",
EditStudentWindow))
        self.ui.marksheet_btn.clicked.connect(lambda: self.show_window("marksheet",
MarksheetWindow))
        self.ui.marks_btn.clicked.connect(lambda: self.show_window("marks",
MarksWindow))
        self.ui.student_attend_btn.clicked.connect(lambda:
self.show_window("student_attendance", StudentAttendanceWindow))
        self.ui.staff_attend_btn.clicked.connect(lambda:
self.show_window("staff_attendance", StaffAttendanceWindow))
        self.ui.teache_btn.clicked.connect(lambda: self.show_window("staff",
StaffWindow))
        self.ui.edit_staff_btn.clicked.connect(lambda: self.show_window("edit_staff",
EditStaffWindow))
        self.ui.fees_btn.clicked.connect(lambda: self.show_window("fees",FeesWindow))
        self.ui.std_rec_btn.clicked.connect(lambda:
self.show_window("stud_rec",StudentRecWindow))
        self.ui.staff_rec_btn.clicked.connect(lambda:
self.show_window("staff_rec",StaffRecWindow))


        self.timer = QTimer(self)
        self.timer.timeout.connect(self.update_time_date)
        self.timer.start(1000) # Update every 1000ms (1 second)

        # Initialize the display
        self.update_time_date()

    def update_time_date(self):

```

```

# Get the current time and date
current_time = QDateTime.currentDateTime()

# Format time and date
time_text = current_time.toString("hh:mm:ss AP") # 12-hour format with AM/PM
date_text = current_time.toString("dd/MM/yyyy") # Full date format

# Update the labels
self.ui.date_label.setText("Date: "+ date_text+" \nTime: "+time_text+"")

def close_window(self, window_name):
    """Close the existing window if it exists."""
    if window_name in self.windows:
        self.windows[window_name].deleteLater()
        del self.windows[window_name]

def show_window(self, window_name, window_class):
    """General method to close and show a fresh window."""
    self.close_window(window_name)
    self.windows[window_name] = window_class() # Create and store the window
    self.windows[window_name].showMaximized()

def set_light_mode(app):
    palette = QPalette()

    palette.setColor(QPalette.Window, QColor(255, 255, 255))
    palette.setColor(QPalette.WindowText, Qt.black)
    palette.setColor(QPalette.Base, QColor(245, 245, 245))
    palette.setColor(QPalette.AlternateBase, QColor(255, 255, 255))
    palette.setColor(QPalette.ToolTipBase, Qt.black)
    palette.setColor(QPalette.ToolTipText, Qt.black)
    palette.setColor(QPalette.Text, Qt.black)
    palette.setColor(QPalette.Button, QColor(240, 240, 240))
    palette.setColor(QPalette.ButtonText, Qt.black)
    palette.setColor(QPalette.Highlight, QColor(0, 120, 215))
    palette.setColor(QPalette.HighlightedText, Qt.white)

    app.setPalette(palette)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    app.setStyle("Fusion")
    set_light_mode(app)
    loginApp = LoginWindow()
    mainApp = MainWindow()
    loginApp.show()
    sys.exit(app.exec())

```

13.2 Student Management Module

```
class AdmissionWindow(QMainWindow):
    def __init__(self):
        super(AdmissionWindow, self).__init__()
        self.ui = admission()
        self.ui.setupUi(self)
        self.setWindowModality(Qt.WindowModality.ApplicationModal)

    self.ui.admission_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch
)

        self.ui.std_id_label.setText("S-"+self.generate_random_string())

        self.ui.save_btn.clicked.connect(self.save_data)
        self.load_data_into_table()
    def generate_random_string(self):
        characters = string.ascii_letters + string.digits # Include letters and numbers
        return ''.join(random.choice(characters) for _ in range(5)).upper()

    def save_data(self):
        # Retrieve data from UI elements
        std_id = self.ui.std_id_label.text()
        first_name = self.ui.std_firstname_box.text()
        last_name = self.ui.std_lastname_box.text()
        fname = self.ui.std_fname_box.text()
        mname = self.ui.std_mname_box.text()
        gender = self.ui.std_gender_box.currentText()
        dob = self.ui.std_dob_box.text()
        address = self.ui.std_address_box.toPlainText()
        contact = self.ui.std_contact_box.text()
        email = self.ui.std_email_box.text()
        standard = self.ui.std_class_box.currentText()
        div = self.ui.std_section_box.currentText()

        # Validate input fields (basic example)
        if not first_name or not last_name or not email or not contact:
            QMessageBox.warning(self, "Input Error", "Please fill in all required fields.")
            return

        try:
            # Connect to the SQLite database
            db = sqlite3.connect("database/school.db")
            cursor = db.cursor()

            # Insert or update the data in the database
            cursor.execute("""
            INSERT INTO students (id, first_name, last_name, fname, mname, gender, dob,
address, contact, email, standard, division)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            ON CONFLICT(id) DO UPDATE SET
            first_name=excluded.first_name,
```

```

        last_name=excluded.last_name,
        fname=excluded.fname,
        mname=excluded.mname,
        gender=excluded.gender,
        dob=excluded.dob,
        address=excluded.address,
        contact=excluded.contact,
        email=excluded.email,
        standard=excluded.standard,
        division=excluded.division
        """, (std_id, first_name, last_name, fname, mname, gender, dob, address,
contact, email, standard, div))

    # Commit changes and close the database
    db.commit()
    db.close()

    # Notify the user
    QMessageBox.information(self, "Success", "Student data saved successfully!")
    self.load_data_into_table()

except sqlite3.Error as e:
    QMessageBox.critical(self, "Database Error", f"An error occurred: {e}")
    db.rollback()
    db.close()

def load_data_into_table(self):
    try:
        # Connect to the SQLite database
        db = sqlite3.connect("database/school.db")
        cursor = db.cursor()

        # Query to fetch all data from the students table
        cursor.execute("SELECT * FROM students")
        data = cursor.fetchall() # Fetch all rows

        # Get the column names
        custom_headers = [
            "Student ID", "First Name", "Last Name", "Father's Name",
            "Mother's Name", "Gender", "Date of Birth", "Address",
            "Contact Number", "Email", "Class", "Section"
        ]

        # Set the row and column counts for the QTableWidgetItem
        self.ui.admission_table.setRowCount(len(data)) # Number of rows
        self.ui.admission_table.setColumnCount(len(custom_headers)) # Number of
columns
        self.ui.admission_table.setHorizontalHeaderLabels(custom_headers) # Set
custom headers

```

```

# Populate the table with data
for row_index, row_data in enumerate(data):
    for col_index, col_data in enumerate(row_data):
        # Convert data to string and add it to the table widget
        self.ui.admission_table.setItem(row_index, col_index,
QTableWidgetItem(str(col_data)))

# Close the database connection
db.close()

except sqlite3.Error as e:
    print(f"Database error: {e}")

class EditStudentWindow(QMainWindow):
    def __init__(self):
        super(EditStudentWindow,self).__init__()
        self.ui = editstud()
        self.ui.setupUi(self)
        self.setWindowModality(Qt.WindowModality.ApplicationModal)
        self.ui.std_age_box.setVisible(False)
        self.ui.label_8.setVisible(False)
        self.ui.save_btn.clicked.connect(self.update_data)
        self.ui.fetch_btn.clicked.connect(self.fetch_data)

    def fetch_data(self):
        # Retrieve the Student ID entered by the user (e.g., in a text box)
        std_id = self.ui.std_id_box.text()

        # Validate if Student ID is provided
        if not std_id:
            QMessageBox.warning(self, "Input Error", "Please enter a valid Student ID.")
            return

        try:
            # Connect to the SQLite database
            db = sqlite3.connect("database/school.db")
            cursor = db.cursor()

            # Query to fetch data from the students table based on the Student ID
            cursor.execute("SELECT * FROM students WHERE id = ?", (std_id,))
            student_data = cursor.fetchone() # Fetch the first matching row

            # If no record is found, show a message
            if not student_data:
                QMessageBox.warning(self, "No Data", f"No student found with ID {std_id}.")
                db.close()
                return

            # Populate the text fields with the fetched data

```

```

self.ui.std_firstname_box.setText(student_data[1]) # first_name
self.ui.std_lastname_box.setText(student_data[2]) # last_name
self.ui.std_fname_box.setText(student_data[3]) # fname
self.ui.std_mname_box.setText(student_data[4]) # mname
self.ui.std_gender_box.setCurrentText(student_data[5]) # gender
self.ui.std_dob_box.setText(student_data[6]) # dob
self.ui.std_address_box.setPlainText(student_data[7]) # address
self.ui.std_contact_box.setText(student_data[8]) # contact
self.ui.std_email_box.setText(student_data[9]) # email
self.ui.std_class_box.setCurrentText(student_data[10]) # standard
self.ui.std_section_box.setCurrentText(student_data[11]) # division

```

```

# Close the database connection
db.close()

```

```

except sqlite3.Error as e:

```

```

    QMessageBox.critical(self, "Database Error", f"An error occurred: {e}")

```

```

def update_data(self):

```

```

    # Retrieve data from UI elements

```

```

    std_id = self.ui.std_id_box.text() # Primary key to identify the record

```

```

    first_name = self.ui.std_firstname_box.text()

```

```

    last_name = self.ui.std_lastname_box.text()

```

```

    fname = self.ui.std_fname_box.text()

```

```

    mname = self.ui.std_mname_box.text()

```

```

    gender = self.ui.std_gender_box.currentText()

```

```

    dob = self.ui.std_dob_box.text()

```

```

    address = self.ui.std_address_box.toPlainText()

```

```

    contact = self.ui.std_contact_box.text()

```

```

    email = self.ui.std_email_box.text()

```

```

    standard = self.ui.std_class_box.currentText()

```

```

    div = self.ui.std_section_box.currentText()

```

```

    # Validate input fields

```

```

    if not std_id or not first_name or not last_name:

```

```

        QMessageBox.warning(self, "Input Error", "Student ID, First Name, and Last
Name are required.")

```

```

        return

```

```

    try:

```

```

        # Connect to the SQLite database

```

```

        db = sqlite3.connect("database/school.db")

```

```

        cursor = db.cursor()

```

```

        # SQL query to update the data

```

```

        cursor.execute("""

```

```

        UPDATE students

```

```

        SET first_name = ?, last_name = ?, fname = ?, mname = ?, gender = ?, dob = ?,

```

```

            address = ?, contact = ?, email = ?, standard = ?, division = ?

```

```

        WHERE id = ?

```

```

        "", (first_name, last_name, fname, mname, gender, dob, address, contact, email,
        standard, div, std_id))

    # Check if any row was updated
    if cursor.rowcount == 0:
        QMessageBox.warning(self, "Update Failed", f"No record found with Student
        ID {std_id}.")
    else:
        # Commit changes and provide feedback
        db.commit()
        QMessageBox.information(self, "Success", "Student data updated
        successfully.")

    # Close the database connection
    db.close()

except sqlite3.Error as e:
    QMessageBox.critical(self, "Database Error", f"An error occurred: {e}")
    db.rollback()
db.close()

class StudentRecWindow(QMainWindow):
    def __init__(self):
        super(StudentRecWindow, self).__init__()
        self.ui = stud_rec()
        self.ui.setupUi(self)
        self.setWindowModality(Qt.WindowModality.ApplicationModal)
        self.ui.std_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self.ui.fetch_btn.clicked.connect(self.load_data_into_table)
        self.load_data_into_table()

    def load_data_into_table(self):
        try:
            # Connect to the SQLite database
            db = sqlite3.connect("database/school.db")
            cursor = db.cursor()

            # Get values entered by the user
            std_id = self.ui.std_id_box.text()
            std_class = self.ui.std_class_box.currentText()
            std_div = self.ui.std_section_box.currentText()

            # Handle the case when no filters are provided
            if not std_id :
                std_id = 'None'

            # Construct the WHERE clause dynamically based on the available filters

```



```

        cursor.execute('SELECT * FROM students WHERE id=? OR standard=? AND
division=?', (std_id,std_class,std_div) )
        data = cursor.fetchall() # Fetch all rows

        # Check if any data was returned
        if not data:
            QMessageBox.warning(self, "No Data", "No student found matching the given
criteria.")
            db.close()
            return

        # Define custom column headers
        custom_headers = [
            "Student ID", "First Name", "Last Name", "Father's Name",
            "Mother's Name", "Gender", "Date of Birth", "Address",
            "Contact Number", "Email", "Class", "Section"
        ]

        # Set the row and column counts for the QTableWidgetItem
        self.ui.std_table.setRowCount(len(data)) # Number of rows
        self.ui.std_table.setColumnCount(len(custom_headers)) # Number of columns
        self.ui.std_table.setHorizontalHeaderLabels(custom_headers) # Set custom
headers

        # Populate the table with data
        for row_index, row_data in enumerate(data):
            for col_index, col_data in enumerate(row_data):
                # Convert data to string and add it to the table widget
                self.ui.std_table.setItem(row_index, col_index,
QTableWidgetItem(str(col_data)))

        # Close the database connection
        db.close()

    except sqlite3.Error as e:
        # Handle any database errors
        print(f"Database error: {e}")
        QMessageBox.critical(self, "Database Error", f"An error occurred while fetching
data: {e}")

```

13.3 Staff Management Module

```
class EditStaffWindow(QMainWindow):
    def __init__(self):
        super(EditStaffWindow,self).__init__()
        self.ui = editstaff()
        self.ui.setupUi(self)
        self.setWindowModality(Qt.WindowModality.ApplicationModal)
        self.ui.fetch_btn.clicked.connect(self.fetch_staff_details)
        self.ui.update_btn.clicked.connect(self.update_staff_details)
    def fetch_staff_details(self):
        # Retrieve the staff ID entered by the user (for example, in a text box)
        staff_id = self.ui.lineEdit.text()

        # Validate if staff ID is provided
        if not staff_id:
            QMessageBox.warning(self, "Input Error", "Please enter a valid Staff ID.")
            return

        try:
            # Connect to the SQLite database
            db = sqlite3.connect("database/school.db")
            cursor = db.cursor()

            # Query to fetch data from the staff table based on the Staff ID
            cursor.execute("SELECT * FROM staff WHERE staff_id = ?", (staff_id,))
            staff_data = cursor.fetchone() # Fetch the first matching row

            # If no record is found, show a message
            if not staff_data:
                QMessageBox.warning(self, "No Data", f"No staff found with ID {staff_id}.")
                db.close()
                return

            # Populate the text fields with the fetched data
            self.ui.staff_fname_box.setText(staff_data[2]) # first_name
            self.ui.staff_lname_box.setText(staff_data[3]) # last_name
            self.ui.staff_category_box.setCurrentText(staff_data[1]) # category
            self.ui.staff_gen_box.setCurrentText(staff_data[4]) # gender
            self.ui.staff_dob_box.setText(staff_data[5]) # dob
            self.ui.staff_address_box.setPlainText(staff_data[6]) # address
            self.ui.staff_contact_box.setText(staff_data[7]) # contact
            self.ui.staff_email_box.setText(staff_data[8]) # email
            self.ui.staff_dept_box.setCurrentText(staff_data[9]) # department
            self.ui.staff_sal_box.setText(str(staff_data[10])) # salary

            # Close the database connection
            db.close()

        except sqlite3.Error as e:
```

```

    QMessageBox.critical(self, "Database Error", f"An error occurred: {e}")

def update_staff_details(self):
    # Retrieve data from the UI elements
    staff_id = self.ui.lineEdit.text() # Staff ID (primary key)
    category = self.ui.staff_category_box.currentText() # Staff Category
    first_name = self.ui.staff_fname_box.text() # First Name
    last_name = self.ui.staff_lname_box.text() # Last Name
    gender = self.ui.staff_gen_box.currentText() # Gender
    dob = self.ui.staff_dob_box.text() # Date of Birth
    address = self.ui.staff_address_box.toPlainText() # Address
    contact = self.ui.staff_contact_box.text() # Contact Number
    email = self.ui.staff_email_box.text() # Email
    dept = self.ui.staff_dept_box.currentText() # Department
    salary = self.ui.staff_sal_box.text() # Salary

    # Validate required fields
    if not staff_id or not first_name or not last_name or not contact or not email:
        QMessageBox.warning(self, "Input Error", "Please fill in all required fields.")
        return

    try:
        # Connect to the SQLite database
        db = sqlite3.connect("database/school.db")
        cursor = db.cursor()

        # SQL query to update the staff details in the staff table
        cursor.execute("""
        UPDATE staff
        SET category = ?, first_name = ?, last_name = ?, gender = ?, dob = ?,
            address = ?, contact = ?, email = ?, department = ?, salary = ?
        WHERE staff_id = ?
        """, (category, first_name, last_name, gender, dob, address, contact, email, dept,
            salary, staff_id))

        # Check if any row was updated
        if cursor.rowcount == 0:
            QMessageBox.warning(self, "Update Failed", f"No record found with Staff ID {staff_id}.")
        else:
            # Commit the changes and provide feedback
            db.commit()
            QMessageBox.information(self, "Success", "Staff details updated successfully.")

        # Close the database connection
        db.close()

    except sqlite3.Error as e:
        QMessageBox.critical(self, "Database Error", f"An error occurred: {e}")

```

```

        db.rollback()
    db.close()

class StaffRecWindow(QMainWindow):
    def __init__(self):
        super(StaffRecWindow,self).__init__()
        self.ui = staff_rec()
        self.ui.setupUi(self)
        self.setWindowModality(Qt.WindowModality.ApplicationModal)
        self.ui.staff_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)

        self.load_data_into_table()
        self.ui.fetch_btn.clicked.connect(self.load_data_into_table)
    def load_data_into_table(self):
        try:
            # Connect to the SQLite database
            db = sqlite3.connect("database/school.db")
            cursor = db.cursor()

            # Get the values entered by the user
            staff_id = self.ui.staff_id_box.text()
            staff_dept = self.ui.staff_dept_box.currentText()

            # Handle case where both filters are empty
            if not staff_id:
                staff_id='None'

            # If staff_id is provided, filter by staff_id, otherwise filter by department
            cursor.execute("SELECT * FROM staff WHERE (staff_id = ? OR department =
?) ", (staff_id, staff_dept))

            data = cursor.fetchall() # Fetch all rows from the query

            # If no data is returned, show a message
            if not data:
                QMessageBox.warning(self, "No Data", "No staff found matching the given
criteria.")
                db.close()
                return

            # Get the column names for header
            headers = ["Staff ID", "Category", "First Name", "Last Name", "Gender",
"DOB",
                "Address", "Contact", "Email", "Department", "Salary"]

            # Set the row and column counts for the QTableWidgetItem
            self.ui.staff_table.setRowCount(len(data)) # Number of rows
            self.ui.staff_table.setColumnCount(len(headers)) # Number of columns
            self.ui.staff_table.setHorizontalHeaderLabels(headers) # Set custom headers

```

```

    # Populate the table with the data
    for row_index, row_data in enumerate(data):
        for col_index, col_data in enumerate(row_data):
            # Convert data to string and add it to the table widget
            self.ui.staff_table.setItem(row_index, col_index,
            QTableWidgetItem(str(col_data)))

    # Resize the columns to fit the content

    # Close the database connection
    db.close()

except sqlite3.Error as e:
    QMessageBox.critical(self, "Database Error", f"An error occurred while fetching
data: {e}")
class StaffWindow(QMainWindow):
    def __init__(self):
        super(StaffWindow,self).__init__()
        self.ui = staff()
        self.ui.setupUi(self)
        self.setWindowModality(Qt.WindowModality.ApplicationModal)
        self.ui.staff_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self.ui.staff_id_label.setText("T-"+self.generate_random_string())
        self.ui.staff_category_box.currentIndexChanged.connect(self.ToggleFunc)

        self.ui.save_btn.clicked.connect(self.save_details)
        self.load_data_into_table()

    def ToggleFunc(self,x):
        if x == 0:
            self.ui.staff_id_label.setText("T-"+self.generate_random_string())
        else:
            self.ui.staff_id_label.setText("NT-"+self.generate_random_string())

    def generate_random_string(self):
        characters = string.ascii_letters + string.digits # Include letters and numbers
        return ''.join(random.choice(characters) for _ in range(5)).upper()

    def save_details(self):
        # Retrieve data from the UI elements
        staff_id = self.ui.staff_id_label.text() # Staff ID
        category = self.ui.staff_category_box.currentText() # Staff Category
        first_name = self.ui.staff_fname_box.text() # First Name
        last_name = self.ui.staff_lname_box.text() # Last Name
        gender = self.ui.staff_gen_box.currentText() # Gender
        dob = self.ui.staff_dob_box.text() # Date of Birth
        address = self.ui.staff_address_box.toPlainText() # Address
        contact = self.ui.staff_contact_box.text() # Contact Number

```

```

email = self.ui.staff_email_box.text() # Email
dept = self.ui.staff_dept_box.currentText() # Department
salary = self.ui.staff_sal_box.text() # Salary

# Validate required fields
if not staff_id or not first_name or not last_name or not contact or not email:
    QMessageBox.warning(self, "Input Error", "Please fill in all required fields.")
    return

try:
    # Connect to the SQLite database
    db = sqlite3.connect("database/school.db")
    cursor = db.cursor()

    # SQL query to insert the staff details into the staff table
    cursor.execute("""
INSERT INTO staff (staff_id, category, first_name, last_name, gender, dob,
                    address, contact, email, department, salary)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
""", (staff_id, category, first_name, last_name, gender, dob, address, contact,
email, dept, salary))

    # Commit the changes
    db.commit()

    # Provide feedback to the user
    QMessageBox.information(self, "Success", "Staff details saved successfully.")

    # Close the database connection
    db.close()
    self.load_data_into_table()

except sqlite3.Error as e:
    # Handle any database errors
    QMessageBox.critical(self, "Database Error", f"An error occurred: {e}")

def load_data_into_table(self):
    try:
        # Connect to the SQLite database
        db = sqlite3.connect("database/school.db")
        cursor = db.cursor()

        # Query to fetch all data from the students table
        cursor.execute("SELECT * FROM staff")
        data = cursor.fetchall() # Fetch all rows

        # Get the column names
        headers = ["Staff ID", "Category", "First Name", "Last Name", "Gender",
"DOB",
"Address", "Contact", "Email", "Department", "Salary"]

```

```
# Set the row and column counts for the QTableWidgetItem
self.ui.staff_table.setRowCount(len(data)) # Number of rows
self.ui.staff_table.setColumnCount(len(headers)) # Number of columns
self.ui.staff_table.setHorizontalHeaderLabels(headers) # Set custom headers

# Populate the table with data
for row_index, row_data in enumerate(data):
    for col_index, col_data in enumerate(row_data):
        # Convert data to string and add it to the table widget
        self.ui.staff_table.setItem(row_index, col_index,
QTableWidgetItem(str(col_data)))

# Close the database connection
db.close()

except sqlite3.Error as e:
    print(f"Database error: {e}")
```

13.4 Attendance Module

```
class StudentAttendanceWindow(QMainWindow):
    def __init__(self):
        super(StudentAttendanceWindow, self).__init__()
        self.ui = std_attend() # Assuming std_attend() is your form's class
        self.ui.setupUi(self)
        self.setWindowModality(Qt.WindowModality.ApplicationModal)

self.ui.std_attend_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch
)

    # Connect buttons to functions
    self.ui.fetch_btn.clicked.connect(self.load_students_for_attendance)
    self.ui.save_btn.clicked.connect(self.save_attendance)

    self.ui.std_attend_table.setColumnCount(4) # Columns: Student ID, Name, Class,
Status
    self.ui.std_attend_table.setHorizontalHeaderLabels(["Student ID", "Name",
"Class", "Status"])

    def check_attendance_exists(self, selected_date, standard, division):
        # Check if attendance exists for the selected date, class, and division
        cursor = sqlite3.connect('database/school.db').cursor()
        cursor.execute("""
            SELECT COUNT(*) FROM attendance
            WHERE date = ? AND standard = ? AND division = ?
            """, (selected_date, standard, division))
        result = cursor.fetchone()
        return result[0] > 0 # Returns True if attendance data exists, False otherwise

    def load_students_for_attendance(self):
        try:
            selected_date = self.ui.std_calender_box.selectedDate().toString("yyyy-MM-dd")
            standard = self.ui.std_class_box.currentText()
            division = self.ui.std_section_box.currentText()

            # Check if attendance exists
            if not self.check_attendance_exists(selected_date, standard, division):
                # Fetch students for the given class and division
                db = sqlite3.connect('database/school.db')
                cursor = db.cursor()
                cursor.execute("""
                    SELECT id, first_name || ' ' || last_name, standard, division
                    FROM students
                    WHERE standard = ? AND division = ?
                    """, (standard, division))
                students = cursor.fetchall()

                # Populate TableWidget with student details and default status
```



```

        self.populate_attendance_table_with_default_status(students, selected_date,
standard, division)
    else:
        # Fetch and display the existing attendance
        self.fetch_attendance()
except sqlite3.Error as e:
    QMessageBox.critical(self, "Error", f"Database error: {e}")

def populate_attendance_table_with_default_status(self, students, selected_date,
standard, division):
    # Populate the attendance table with the students' information
    self.ui.std_attend_table.setRowCount(len(students))
    self.ui.std_attend_table.setColumnCount(4) # Student ID, Name, Class, Status
    self.ui.std_attend_table.setHorizontalHeaderLabels(["Student ID", "Name",
"Class", "Status"])

    for row, student in enumerate(students):
        self.ui.std_attend_table.setItem(row, 0, QTableWidgetItem(student[0])) #
Student ID
        self.ui.std_attend_table.setItem(row, 1, QTableWidgetItem(student[1])) # Name
        self.ui.std_attend_table.setItem(row, 2, QTableWidgetItem(f"{student[2]} -
{student[3]}")) # Class - Division

        # Default attendance status is "Absent"
        combo = QComboBox()
        combo.addItem(["Present", "Absent", "Late", "Excused"])
        combo.setCurrentText("Absent") # Default status
        self.ui.std_attend_table.setCellWidget(row, 3, combo)

    # Add default attendance entries to the database
    self.save_default_attendance(selected_date, students, standard, division)

def save_default_attendance(self, selected_date, students, standard, division):
    try:
        # Save default attendance status for each student
        db = sqlite3.connect('database/school.db')
        cursor = db.cursor()
        for student in students:
            std_id = student[0] # Student ID
            status = "Absent" # Default status

            cursor.execute("""
                INSERT INTO attendance (std_id, date, status, standard, division)
                VALUES (?, ?, ?, ?, ?)
            """, (std_id, selected_date, status, standard, division))

            db.commit() # Commit the changes
    except sqlite3.Error as e:
        QMessageBox.critical(self, "Error", f"Database error: {e}")

```

```

def fetch_attendance(self):
    try:
        selected_date = self.ui.std_calender_box.selectedDate().toString("yyyy-MM-dd")
        standard = self.ui.std_class_box.currentText()
        division = self.ui.std_section_box.currentText()

        # Connect to the database
        db = sqlite3.connect('database/school.db')
        cursor = db.cursor()

        # Fetch attendance data from the database
        cursor.execute("""
            SELECT a.std_id, s.first_name || ' ' || s.last_name AS full_name,
                   a.standard, a.division, a.status
            FROM attendance a
            JOIN students s ON a.std_id = s.id
            WHERE a.date = ? AND a.standard = ? AND a.division = ?
            """, (selected_date, standard, division))

        records = cursor.fetchall()

        # Set up the table widget: setting rows and columns dynamically based on the
        # fetched data
        self.ui.std_attend_table.setRowCount(len(records))
        self.ui.std_attend_table.setColumnCount(4) # Columns: Student ID, Name,
        # Class, Status
        self.ui.std_attend_table.setHorizontalHeaderLabels(["Student ID", "Name",
        "Class", "Status"])

        # Loop through the fetched records and populate the table
        for row, record in enumerate(records):
            # Add student data to the first three columns
            self.set_student_data(row, record)

            # Add a combo box for the attendance status in the last column
            self.set_attendance_status(row, record[4])

    except sqlite3.Error as e:
        QMessageBox.critical(self, "Error", f"Database error: {e}")

def set_student_data(self, row, record):
    """
    Set student information (ID, Name, Class) in the table.
    """
    # Student ID
    self.ui.std_attend_table.setItem(row, 0, QTableWidgetItem(str(record[0])))

    # Student Name
    self.ui.std_attend_table.setItem(row, 1, QTableWidgetItem(str(record[1])))

```

```

# Class and Division (Standard and Division)
self.ui.std_attend_table.setItem(row, 2, QTableWidgetItem(f'{record[2]} - {record[3]}'))

def set_attendance_status(self, row, status):
    """
    Set attendance status using a combo box.
    """

    # Create combo box for attendance status
    combo = QComboBox()
    combo.addItem("Present", "Absent", "Late", "Excused")

    # Set the current status from the database
    combo.setCurrentText(status) # Set the current status based on the fetched record

    # Add the combo box to the last column (Status column)
    self.ui.std_attend_table.setCellWidget(row, 3, combo)

def save_attendance(self):
    try:
        selected_date = self.ui.std_calender_box.selectedDate().toString("yyyy-MM-dd")
        standard = self.ui.std_class_box.currentText()
        division = self.ui.std_section_box.currentText()
        db = sqlite3.connect('database/school.db')
        cursor = db.cursor()

        for row in range(self.ui.std_attend_table.rowCount()):
            std_id = self.ui.std_attend_table.item(row, 0).text() # Student ID
            status = self.ui.std_attend_table.cellWidget(row, 3).currentText() # Attendance

Status

            # Check if attendance for this student and date already exists in the DB
            cursor.execute("""
                SELECT COUNT(*) FROM attendance
                WHERE std_id = ? AND date = ? AND standard = ? AND division = ?
                """, (std_id, selected_date, standard, division))
            exists = cursor.fetchone()[0]

            if exists:
                # If attendance exists, update it
                cursor.execute("""
                    UPDATE attendance
                    SET status = ?
                    WHERE std_id = ? AND date = ? AND standard = ? AND division = ?
                    """, (status, std_id, selected_date, standard, division))
            else:
                # If attendance doesn't exist, insert a new record

```

```

        cursor.execute("""
            INSERT INTO attendance (std_id, date, status, standard, division)
            VALUES (?, ?, ?, ?, ?)
            """, (std_id, selected_date, status, standard, division))

        db.commit() # Commit the changes
        QMessageBox.information(self, "Success", "Attendance saved successfully!")
    except sqlite3.Error as e:
        QMessageBox.critical(self, "Error", f"Database error: {e}")
class StaffAttendanceWindow(QMainWindow):
    def __init__(self):
        super(StaffAttendanceWindow, self).__init__()
        self.ui = staff_attend() # Assuming staff_attend() is your form's class
        self.ui.setupUi(self)
        self.setWindowModality(Qt.WindowModality.ApplicationModal)
        self.ui.staff_attend_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)

        # Connect buttons to functions
        self.ui.fetch_btn.clicked.connect(self.load_staff_for_attendance)
        self.ui.save_btn.clicked.connect(self.save_attendance)

    def check_staff_attendance_exists(self, selected_date, department):
        # Check if staff attendance exists for the selected date and department
        cursor = sqlite3.connect('database/school.db').cursor()
        cursor.execute("""
            SELECT COUNT(*) FROM staff_attendance
            WHERE date = ? AND department = ?
            """, (selected_date, department))
        result = cursor.fetchone()
        return result[0] > 0 # Returns True if attendance data exists, False otherwise

    def load_staff_for_attendance(self):
        try:
            selected_date = self.ui.staff_calender_box.selectedDate().toString("yyyy-MM-dd")
            department = self.ui.staff_dept_box.currentText()

            # Check if attendance exists for the selected date and department
            if not self.check_staff_attendance_exists(selected_date, department):
                # Fetch staff members from the selected department
                db = sqlite3.connect('database/school.db')
                cursor = db.cursor()
                cursor.execute("""
                    SELECT staff_id, first_name || ' ' || last_name, department
                    FROM staff
                    WHERE department = ?
                    """, (department,))
                staff_members = cursor.fetchall()

```

```

        # Populate TableWidget with staff details and default status
        self.populate_staff_attendance_table_with_default_status(staff_members,
selected_date, department)
    else:
        # Fetch and display the existing staff attendance
        self.fetch_staff_attendance()
except sqlite3.Error as e:
    QMessageBox.critical(self, "Error", f"Database error: {e}")

def populate_staff_attendance_table_with_default_status(self, staff_members,
selected_date, department):
    # Populate the attendance table with the staff members' information
    self.ui.staff_attend_table.setRowCount(len(staff_members))
    self.ui.staff_attend_table.setColumnCount(4) # Staff ID, Name, Department, Status
    self.ui.staff_attend_table.setHorizontalHeaderLabels(["Staff ID", "Name",
"Department", "Status"])

    for row, staff in enumerate(staff_members):
        self.ui.staff_attend_table.setItem(row, 0, QTableWidgetItem(str(staff[0]))) #
Staff ID
        self.ui.staff_attend_table.setItem(row, 1, QTableWidgetItem(staff[1])) # Name
        self.ui.staff_attend_table.setItem(row, 2, QTableWidgetItem(staff[2])) #
Department

        # Default attendance status is "Absent"
        combo = QComboBox()
        combo.addItem(["Present", "Absent", "Late", "Excused"])
        combo.setCurrentText("Absent") # Default status
        self.ui.staff_attend_table.setCellWidget(row, 3, combo)

    # Add default attendance entries to the database
    self.save_default_staff_attendance(selected_date, staff_members, department)

def save_default_staff_attendance(self, selected_date, staff_members, department):
    try:
        # Save default attendance status for each staff member
        db = sqlite3.connect('database/school.db')
        cursor = db.cursor()
        for staff in staff_members:
            staff_id = staff[0] # Staff ID
            status = "Absent" # Default status

            cursor.execute("""
                INSERT INTO staff_attendance (staff_id, date, status, department)
                VALUES (?, ?, ?, ?)
            """, (staff_id, selected_date, status, department))

        db.commit() # Commit the changes
    except sqlite3.Error as e:
        QMessageBox.critical(self, "Error", f"Database error: {e}")

```

```

def fetch_staff_attendance(self):
    try:
        selected_date = self.ui.staff_calender_box.selectedDate().toString("yyyy-MM-
dd")
        department = self.ui.staff_dept_box.currentText()
        db = sqlite3.connect('database/school.db')
        cursor = db.cursor()
        cursor.execute("""
            SELECT sa.staff_id, s.first_name || ' ' || s.last_name AS full_name,
                sa.department, sa.status
            FROM staff_attendance sa
            JOIN staff s ON sa.staff_id = s.staff_id
            WHERE sa.date = ? AND sa.department = ?
        """, (selected_date, department))
        records = cursor.fetchall()

        # Populate TableWidget with staff attendance data
        self.ui.staff_attend_table.setRowCount(len(records))
        for row, record in enumerate(records):
            for col, data in enumerate(record):
                if col != 3: # Attendance status (column 3) will have a combo box
                    self.ui.staff_attend_table.setItem(row, col, QTableWidgetItem(str(data)))
                else:
                    # Add combo box for attendance status
                    combo = QComboBox()
                    combo.addItem("Present", "Absent", "Late", "Excused")
                    combo.setCurrentText(data) # Set the current status from the DB
                    self.ui.staff_attend_table.setCellWidget(row, col, combo)

    except sqlite3.Error as e:
        QMessageBox.critical(self, "Error", f"Database error: {e}")

def save_attendance(self):
    try:
        selected_date = self.ui.staff_calender_box.selectedDate().toString("yyyy-MM-
dd")
        department = self.ui.staff_dept_box.currentText()
        db = sqlite3.connect('database/school.db')
        cursor = db.cursor()
        for row in range(self.ui.staff_attend_table.rowCount()):
            staff_id = self.ui.staff_attend_table.item(row, 0).text() # Staff ID
            status = self.ui.staff_attend_table.cellWidget(row, 3).currentText() #
Attendance Status

            # Update attendance status in the database
            cursor.execute("""
                UPDATE staff_attendance
                SET status = ?
                WHERE staff_id = ? AND date = ? AND department = ?
            """, (status, staff_id, selected_date, department))
    
```

```
"" , (status, staff_id, selected_date, department))
```

```
db.commit() # Commit the changes
```

```
QMessageBox.information(self, "Success", "Attendance saved successfully!")
```

```
except sqlite3.Error as e:
```

```
QMessageBox.critical(self, "Error", f"Database error: {e}")
```

13.5 Marks Management Module

```
class MarksheetWindow(QMainWindow):
    def __init__(self):
        super(MarksheetWindow, self).__init__()

        self.ui = marksheet() # Assuming marksheet is the UI class generated by pyuic
        self.ui.setupUi(self)
        self.setWindowModality(Qt.WindowModality.ApplicationModal)

self.ui.std_marks_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch
)

        self.details = {}
        # Connect fetch button to function
        self.ui.fetch_btn.clicked.connect(self.fetch_marks)
        self.ui.print_btn.clicked.connect(self.print_marksheet)

    def fetch_marks(self):
        """Fetch marks for a student and populate the list view with student details and the
        table with marks."""
        try:
            std_id = self.ui.std_id_box.text() # Get student ID from input field or combo box
            db = sqlite3.connect('database/school.db')
            cursor = db.cursor()

            # Fetch student details
            cursor.execute("""
                SELECT first_name || ' ' || last_name, standard, division,dob,fname,mname
                FROM students
                WHERE id = ?
            """, (std_id,))
            student_details = cursor.fetchone()

            if student_details:
                # Extract details from fetched data
                student_name, student_class, student_section, dob, fname, mname =
student_details
                student_info = [
                    f"Student ID: {std_id}",
                    f"Student Name: {student_name}",
                    f"Class: {student_class}",
                    f"Section: {student_section}"
                ]

                # Assuming `std_details_box` is a QListWidget
                self.ui.std_details_box.clear() # Clear the list widget first
                self.ui.std_details_box.addItem(student_info) # Add student details to the list
widget

            # Fetch marks for the student
```



```

cursor.execute("""
    SELECT subject_name, theory_marks, assignment_marks
    FROM marks
    WHERE std_id = ?
""", (std_id,))
records = cursor.fetchall()

# Clear the previous data from the table
self.ui.std_marks_table.setRowCount(0)

# Populate the table with fetched data
self.ui.std_marks_table.setRowCount(len(records))
self.ui.std_marks_table.setColumnCount(3) # 4 columns: Subject, Theory
Marks, Assignment Marks
self.ui.std_marks_table.setHorizontalHeaderLabels(["Subject", "Theory
Marks", "Assignment Marks"])
marks = []
# Populate table with data
for row, record in enumerate(records):
    self.ui.std_marks_table.setItem(row, 0, QTableWidgetItem(record[0])) #
Subject Name
    self.ui.std_marks_table.setItem(row, 1, QTableWidgetItem(str(record[1])))
# Theory Marks
    self.ui.std_marks_table.setItem(row, 2, QTableWidgetItem(str(record[2])))
# Assignment Marks

total_marks_obtained = 0 # Total obtained marks across all subjects
total_max_marks = 0 # Total maximum marks across all subjects
marks = [] # List to store individual subject marks and grades

for record in records:
    subject_name = record[0]
    theory_marks = record[1]
    theory_max = 70
    assignment_marks = record[2]
    assignment_max = 30
    total_subject_marks = theory_marks + assignment_marks
    max_subject_marks = theory_max + assignment_max

# Add to cumulative totals
total_marks_obtained += total_subject_marks
total_max_marks += max_subject_marks

# Append subject-specific details with grade
marks.append([
    subject_name, # Subject Name
    theory_marks, # Theory Marks
    theory_max, # Total Theory Marks
    assignment_marks, # Assignment Marks

```

```

        assignment_max,          # Total Assignment Marks
        total_subject_marks,     # Total Obtained Marks (Theory +
Assignment)
        self.assign_grade(total_subject_marks) # Grade for the subject
    )

    # Calculate overall percentage and grade
    percentage = round((total_marks_obtained / total_max_marks) * 100,2)
    final_grade = self.assign_grade(total_marks_obtained / len(records)) #
Average grade
    result = "Pass" if percentage >= 35 else "Fail"
    self.details = {"name":student_name,
                    "fname":fname,
                    "mname":mname,
                    "id":std_id,
                    "class":student_class,
                    "sec":student_section,
                    "dob":dob,
                    "marks":marks,
                    "total": total_marks_obtained,
                    "per":percentage,
                    "grade":final_grade,
                    "result":result
                    }

    return self.details
else:
    QMessageBox.warning(self, "No Student Found", "Student ID not found!")

except sqlite3.Error as e:
    QMessageBox.critical(self, "Error", f"Database error: {e}")

def assign_grade(self, total_marks):
    """Assign a grade based on total obtained marks."""
    if total_marks >= 90:
        return 'A+'
    elif 80 <= total_marks < 90:
        return 'A'
    elif 70 <= total_marks < 80:
        return 'B'
    elif 60 <= total_marks < 70:
        return 'C'
    elif 50 <= total_marks < 60:
        return 'D'
    elif 35 <= total_marks < 50:
        return 'E'
    else:
        return 'Fail'

def generate_marksheet(self):

```

```
"""Generate a marksheet using docxtpl and save it."""
```

```
try:
```

```
    # Load the Word template
```

```
    doc = DocxTemplate("report.docx")
```

```
    # Prepare context (data to fill in the template)
```

```
    context = {
```

```
        'name': self.details['name'],
```

```
        'fname': self.details['fname'],
```

```
        'mname': self.details['mname'],
```

```
        'std_id': self.details['id'],
```

```
        'std': self.details['class'],
```

```
        'div': self.details['sec'],
```

```
        'marks': self.details['marks'],
```

```
        'per':self.details['per'],
```

```
        'result':self.details['result'] + " " + self.details['grade'] ,
```

```
        'total':self.details['total']
```

```
    }
```

```
    # Render the template with the student data
```

```
    doc.render(context)
```

```
    # Save the generated marksheet
```

```
    file_name = f"report_card/marksheet_{self.details['id']}.docx"
```

```
    doc.save(file_name)
```

```
    return file_name
```

```
except Exception as e:
```

```
    QMessageBox.critical(self, "Error", f"Error generating marksheet: {e}")
```

```
def print_marksheet(self):
```

```
    """Print the marksheet for the selected student."""
```

```
    student_id = self.ui.std_id_box.text() # Assume the student ID is entered in a text box
```

```
    if not student_id:
```

```
        QMessageBox.warning(self, "Input Error", "Please enter a valid student ID.")
```

```
        return
```

```
    # Fetch student data
```

```
    student_data = self.fetch_marks()
```

```
    if not student_data:
```

```
        return # Student not found
```

```
    # Generate the marksheet
```

```
    file_name = self.generate_marksheet()
```

```
    if file_name:
```

```

    QMessageBox.information(self, "Success", f"Marksheets have been generated
and saved as {file_name}")
    try:
        subprocess.Popen(['start', file_name], shell=True)
    except Exception as e:
        print(f"Error: {e}")
    except FileNotFoundError:
        print("The specified application or file was not found.")
    except subprocess.CalledProcessError as e:
        print(f"An error occurred: {e}")
    else:
        QMessageBox.critical(self, "Error", "Error generating the marksheet.")

class MarksWindow(QMainWindow):
    def __init__(self):
        super(MarksWindow, self).__init__()
        self.ui = marks() # Assuming marks() is your form's UI class
        self.ui.setupUi(self)
        self.setWindowModality(Qt.WindowModality.ApplicationModal)
        self.ui.sub_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self.ui.sub_table.setRowCount(0)
        custom_headers = ["Subject", "Theory", "Practical"]

        # Set the row and column counts for the QTableWidgetItem
        self.ui.sub_table.setColumnCount(len(custom_headers)) # Number of columns
        self.ui.sub_table.setHorizontalHeaderLabels(custom_headers) # Set custom
headers

        # Connect buttons to functions
        self.ui.sub_btn.clicked.connect(self.publish_marks)
        self.ui.save_btn.clicked.connect(self.save_marks)
        self.ui.fetch_btn.clicked.connect(self.fetch_marks)

    def publish_marks(self):
        """Publish entered subject marks to the table widget."""
        try:
            # Get input values from UI elements
            std_id = self.ui.std_id_box.text() # Student ID input field
            subject_name = self.ui.sub_name.text() # Subject Name combo box
            theory_marks = self.ui.sub_m1_box.text() # Theory Marks spinner
            assignment_marks = self.ui.sub_m2_box.text() # Assignment Marks spinner

            # Add the entered details into the table widget
            row_position = self.ui.sub_table.rowCount()
            self.ui.sub_table.insertRow(row_position)

            # Set the values into the table cells
            self.ui.sub_table.setItem(row_position, 0, QTableWidgetItem(subject_name)) #
Subject Name

```

```

        self.ui.sub_table.setItem(row_position, 1, QTableWidgetItem(str(theory_marks)))
# Theory Marks
        self.ui.sub_table.setItem(row_position, 2,
QTableWidgetItem(str(assignment_marks))) # Assignment Marks

    except Exception as e:
        QMessageBox.critical(self, "Error", f"An error occurred while publishing
marks: {e}")

def save_marks(self):
    """Save or update marks for a student."""
    try:
        db = sqlite3.connect('database/school.db')
        cursor = db.cursor()

        std_id = self.ui.std_id_box.text() # Get the student ID from input field

        # Loop through each row in the table and update or insert marks
        for row in range(self.ui.sub_table.rowCount()):
            subject_name = self.ui.sub_table.item(row, 0).text() # Subject
            theory_marks = self.ui.sub_table.item(row, 1).text() # Theory Marks
            assignment_marks = self.ui.sub_table.item(row, 2).text() # Assignment Marks

            # Check if the record already exists in the database
            cursor.execute("""
                SELECT COUNT(*) FROM marks
                WHERE std_id = ? AND subject_name = ?
            """, (std_id, subject_name))
            exists = cursor.fetchone()[0]

            if exists > 0:
                # If the record exists, update the marks
                cursor.execute("""
                    UPDATE marks
                    SET theory_marks = ?, assignment_marks = ?
                    WHERE std_id = ? AND subject_name = ?
                """, (theory_marks, assignment_marks, std_id, subject_name))
            else:
                # If the record does not exist, insert a new record
                cursor.execute("""
                    INSERT INTO marks (std_id, subject_name, theory_marks,
assignment_marks)
                    VALUES (?, ?, ?, ?)
                """, (std_id, subject_name, theory_marks, assignment_marks))

            db.commit() # Commit the changes to the database
            QMessageBox.information(self, "Success", "Marks saved successfully!")

    except sqlite3.Error as e:
        QMessageBox.critical(self, "Error", f"Database error: {e}")

```

```

def fetch_marks(self):
    """Fetch marks for a student and populate the table."""
    try:
        db = sqlite3.connect('database/school.db')
        cursor = db.cursor()

        std_id = self.ui.std_id_box.text() # Get the student ID from input field
        cursor.execute("""
            SELECT std_id, subject_name, theory_marks, assignment_marks
            FROM marks
            WHERE std_id = ?
        """, (std_id,))
        records = cursor.fetchall()

        # Populate the table widget with fetched data
        self.ui.sub_table.setRowCount(len(records))
        for row, record in enumerate(records):
            self.ui.sub_table.setItem(row, 0, QTableWidgetItem(str(record[1]))) # Subject
            self.ui.sub_table.setItem(row, 1, QTableWidgetItem(str(record[2]))) # Theory
            self.ui.sub_table.setItem(row, 2, QTableWidgetItem(str(record[3]))) #
            self.ui.sub_table.setItem(row, 3, QTableWidgetItem(str(record[4]))) #
        Marks
        Assignment Marks

    except sqlite3.Error as e:
        QMessageBox.critical(self, "Error", f"Database error: {e}")

```

13.6 Fees Management Module

```
class FeesWindow(QMainWindow):
    def __init__(self):
        super(FeesWindow, self).__init__()
        self.ui = fees()
        self.ui.setupUi(self)
        self.setWindowModality(Qt.WindowModality.ApplicationModal)
        self.ui.fees_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self.ui.entry_btn.setVisible(False)
        self.load_fees_data()
        self.ui.save_btn.clicked.connect(self.save_fees)
        self.ui.fetch_btn.clicked.connect(self.fetch_fees_data)

    def fetch_fees_data(self):
        try:
            # Retrieve the student ID entered by the user
            std_id = self.ui.fees_id_box.text()

            # Validate the student ID input
            if not std_id:
                QMessageBox.warning(self, "Input Error", "Please enter a Student ID to fetch fees.")
                return

            # Connect to the SQLite database
            db = sqlite3.connect("database/school.db")
            cursor = db.cursor()

            # Query to fetch fee records for the given student ID
            cursor.execute("""
                SELECT f.std_id, s.id AS std_id, s.first_name, s.last_name, f.amount,
f.fee_date
                FROM fees f
                JOIN students s ON f.std_id = s.id
                WHERE f.std_id = ?
            """, (std_id,))

            data = cursor.fetchall() # Fetch all rows

            # Check if data is returned
            if not data:
                QMessageBox.warning(self, "No Data", f"No fee records found for Student ID {std_id}.")
                db.close()
                return

            # Define custom column headers
            custom_headers = [
```

```

        "Fee ID", "Student ID", "First Name", "Last Name", "Amount", "Fee Date"
    ]

    # Set the row and column counts for the QTableWidgetItem
    self.ui.fees_table.setRowCount(len(data)) # Number of rows
    self.ui.fees_table.setColumnCount(len(custom_headers)) # Number of columns
    self.ui.fees_table.setHorizontalHeaderLabels(custom_headers) # Set custom
headers

    # Populate the table with fee data
    for row_index, row_data in enumerate(data):
        for col_index, col_data in enumerate(row_data):
            # Convert data to string and add it to the table widget
            self.ui.fees_table.setItem(row_index, col_index,
QTableWidgetItem(str(col_data)))

    # Close the database connection
    db.close()

except sqlite3.Error as e:
    # Handle any database errors
    print(f"Database error: {e}")
    QMessageBox.critical(self, "Database Error", f"An error occurred while fetching
fee data: {e}")

def save_fees(self):
    try:
        # Retrieve the inputs
        std_id = self.ui.fees_id_box.text()
        amt = self.ui.lineEdit.text()
        date = QDate.currentDate().toString("dd-MM-yyyy") # Current date in the
format "dd-MM-yyyy"

        # Validate inputs
        if not std_id or not amt or not date:
            QMessageBox.warning(self, "Input Error", "Please fill in all the fields.")
            return

        if not amt.isdigit():
            QMessageBox.warning(self, "Input Error", "Please enter a valid fee amount.")
            return

        # Connect to the database
        db = sqlite3.connect("database/school.db")
        cursor = db.cursor()

        # Query to check if the student ID exists
        cursor.execute("SELECT * FROM students WHERE id=?", (std_id,))
        student = cursor.fetchone()

```



```

        if not student:
            QMessageBox.warning(self, "Invalid Student", f"No student found with ID {std_id}.")
            db.close()
            return

        # Insert the fee data into the fees table
        cursor.execute("""
            INSERT INTO fees (std_id, amount, fee_date)
            VALUES (?, ?, ?)
            """, (std_id, amt, date))

        # Commit the changes to the database
        db.commit()

        # Show a success message
        QMessageBox.information(self, "Success", "Fee record saved successfully.")

        # Clear the input fields after saving the data
        self.ui.fees_id_box.clear()
        self.ui.lineEdit.clear()

        # Close the database connection
        db.close()
        self.load_fees_data()

    except sqlite3.Error as e:
        # Handle any database errors
        print(f"Database error: {e}")
        QMessageBox.critical(self, "Database Error", f"An error occurred while saving the fee: {e}")

    def load_fees_data(self):
        try:
            # Connect to the SQLite database
            db = sqlite3.connect("database/school.db")
            cursor = db.cursor()

            # Query to fetch all fee records
            cursor.execute("SELECT f.std_id, s.id AS std_id, s.first_name, s.last_name, f.amount, f.fee_date "
                           "FROM fees f "
                           "JOIN students s ON f.std_id = s.id")
            data = cursor.fetchall() # Fetch all rows

            # Check if data is returned
            if not data:
                QMessageBox.warning(self, "No Data", "No fee records found.")
                db.close()

```

```

    return

    # Define custom column headers
    custom_headers = [
        "Fee ID", "Student ID", "First Name", "Last Name", "Amount", "Fee Date"
    ]

    # Set the row and column counts for the QTableWidgetItem
    self.ui.fees_table.setRowCount(len(data)) # Number of rows
    self.ui.fees_table.setColumnCount(len(custom_headers)) # Number of columns
    self.ui.fees_table.setHorizontalHeaderLabels(custom_headers) # Set custom
headers

    # Populate the table with fee data
    for row_index, row_data in enumerate(data):
        for col_index, col_data in enumerate(row_data):
            # Convert data to string and add it to the table widget
            self.ui.fees_table.setItem(row_index, col_index,
QTableWidgetItem(str(col_data)))

    # Close the database connection
    db.close()

except sqlite3.Error as e:
    # Handle any database errors
    print(f"Database error: {e}")
    QMessageBox.critical(self, "Database Error", f"An error occurred while fetching fee
{e}

```

13.7 Database SQL Table

```
CREATE TABLE students (  
  id TEXT PRIMARY KEY,  
  first_name TEXT,  
  last_name TEXT,  
  fname TEXT,  
  mname TEXT,  
  gender TEXT,  
  dob TEXT,  
  address TEXT,  
  contact TEXT,  
  email TEXT,  
  standard TEXT,  
  division TEXT  
);
```

```
CREATE TABLE staff (  
  staff_id TEXT PRIMARY KEY,  
  category TEXT,  
  first_name TEXT,  
  last_name TEXT,  
  gender TEXT,  
  dob TEXT,  
  address TEXT,  
  contact TEXT,  
  email TEXT,  
  department TEXT,  
  salary REAL  
);
```

```
CREATE TABLE marks (  
  std_id TEXT,  
  subject_name TEXT,  
  theory_marks INTEGER,  
  assignment_marks INTEGER,  
  PRIMARY KEY (std_id, subject_name),  
  FOREIGN KEY (std_id) REFERENCES students(id)  
);
```

```
CREATE TABLE fees (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  std_id TEXT,  
  amount REAL,  
  fee_date TEXT,  
  FOREIGN KEY (std_id) REFERENCES students(id)  
);
```

```

CREATE TABLE attendance (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    std_id TEXT,
    date TEXT,
    status TEXT,
    standard TEXT,
    division TEXT,
    FOREIGN KEY (std_id) REFERENCES students(id)
);

CREATE TABLE staff_attendance (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    staff_id TEXT,
    date TEXT,
    status TEXT,
    department TEXT,
    FOREIGN KEY (staff_id) REFERENCES staff(staff_id)
);

CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE,
    password TEXT
);

```

13.8 icons_res.qrc

```

<RCC>
<qresource>
<file>assets/classroom.png</file>
<file>assets/setting.png</file>
<file>assets/staff_record.png</file>
<file>assets/std_record.png</file>
<file>assets/admission_icon.png</file>
<file>assets/attendance.png</file>
<file>assets/calendar.png</file>
<file>assets/close.png</file>
<file>assets/edit_stud.png</file>
<file>assets/fees.png</file>
<file>assets/logout.png</file>
<file>assets/marks.png</file>
<file>assets/minimize.png</file>
<file>assets/result.png</file>
<file>assets/school.jpg</file>
<file>assets/staff.png</file>
<file>assets/staff-attend.png</file>
<file>assets/teaching.png</file>
</qresource>
</RCC>

```



```

# -*- coding: utf-8 -*-

#####
#####
## Form generated from reading UI file 'admissionVxrdGo.ui'
##
## Created by: Qt User Interface Compiler version 6.4.0
##
## WARNING! All changes made in this file will be lost when recompiling UI file!
#####
#####

from PySide6.QtCore import (QCoreApplication, QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt)
from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
    QFont, QFontDatabase, QGradient, QIcon,
    QImage, QKeySequence, QLinearGradient, QPainter,
    QPalette, QPixmap, QRadialGradient, QTransform)
from PySide6.QtWidgets import (QApplication, QComboBox, QFormLayout,
    QFrame,
    QHBoxLayout, QHeaderView, QLabel, QLineEdit,
    QMainWindow, QPlainTextEdit, QPushButton, QSizePolicy,
    QSpacerItem, QTabWidget, QTableWidget, QTableWidgetItem,
    QVBoxLayout, QWidget)

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.setObjectName():
            MainWindow.setObjectName(u"MainWindow")
        MainWindow.resize(800, 600)
        self.centralwidget = QWidget(MainWindow)
        self.centralwidget.setObjectName(u"centralwidget")
        self.verticalLayout_2 = QVBoxLayout(self.centralwidget)
        self.verticalLayout_2.setObjectName(u"verticalLayout_2")
        self.tabWidget = QTabWidget(self.centralwidget)
        self.tabWidget.setObjectName(u"tabWidget")
        self.tabWidget.setStyleSheet(u"/* Styling the Tab Widget */\n"
"QTabWidget {\n"
"    background-color: #f0f0f0;\n"
"    border: 2px solid black;\n"
"    border-radius: 10px;\n"
"    padding: 5px;\n"
"}\n"
"\n"
"\n"
"/* Styling the Tabs */\n"
"QTabBar::tab {\n"
"    background-color: #2196F3; /* Tab background color */\n"

```

```

" color: white;\n"
" border: 1px solid #c0c0c0;\n"
" padding: 10px;\n"
" border-radius: 5px;\n"
" min-width: 80px;\n"
" min-height: 30px;\n"
"\n"
" font: 14pt \"Segoe UI\";\n"
"}\n"
"\n"
""
    self.tab = QWidget()
    self.tab.setObjectName(u"tab")
    self.tab.setStyleSheet(u"")
    self.verticalLayout = QVBoxLayout(self.tab)
    self.verticalLayout.setObjectName(u"verticalLayout")
    self.horizontalLayout_2 = QHBoxLayout()
    self.horizontalLayout_2.setObjectName(u"horizontalLayout_2")
    self.horizontalLayout_2.setContentsMargins(-1, 0, -1, -1)
    self.formFrame = QFrame(self.tab)
    self.formFrame.setObjectName(u"formFrame")
    self.formFrame.setStyleSheet(u"*{\n"
"font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/* Styling the QLineEdit */\n"
"QLineEdit {\n"
" background-color: #f4f4f4; /* Light grey background */\n"
" color: #333333; /* Dark text color */\n"
" border: 2px solid #c0c0c0; /* Border color */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 5px;\n"
"}\n"
"\n"
"/* Focused state (when the QLineEdit is selected) */\n"
"QLineEdit:focus {\n"
" border: 2px solid #2196F3; /* Blue border when focused */\n"
" background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state */\n"
"QLineEdit:disabled {\n"
" background-color: #e0e0e0; /* Grey background when disabled */\n"
" color: #888888; /* Light grey text when disabled */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Placeholder text style */\n"
"QLineEdit::placeholder {\n"

```

```

"    color: #888888; /* Light grey color for placeholder text */\n"
"    font-style: italic; /* Italicize the placeholder text */\n"
""

        "}\n"
"\n"
"/\n" Styling the text cursor */\n"
"QLineEdit::cursor {\n"
"    color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
""
)

    self.formLayout = QFormLayout(self.formFrame)
    self.formLayout.setObjectName(u"formLayout")
    self.formLayout.setContentsMargins(-1, 0, 0, 0)
    self.label_2 = QLabel(self.formFrame)
    self.label_2.setObjectName(u"label_2")

    self.formLayout.addWidget(0, QFormLayout.LabelRole, self.label_2)

    self.std_id_label = QLabel(self.formFrame)
    self.std_id_label.setObjectName(u"std_id_label")
    self.std_id_label.setStyleSheet(u"color: rgb(255, 0, 0);\n"
"font: 700 14pt \"Segoe UI\";")

    self.formLayout.addWidget(0, QFormLayout.FieldRole, self.std_id_label)

    self.label = QLabel(self.formFrame)
    self.label.setObjectName(u"label")

    self.formLayout.addWidget(1, QFormLayout.LabelRole, self.label)

    self.std_firstname_box = QLineEdit(self.formFrame)
    self.std_firstname_box.setObjectName(u"std_firstname_box")

    self.formLayout.addWidget(1, QFormLayout.FieldRole,
self.std_firstname_box)

    self.label_3 = QLabel(self.formFrame)
    self.label_3.setObjectName(u"label_3")

    self.formLayout.addWidget(2, QFormLayout.LabelRole, self.label_3)

    self.std_lastname_box = QLineEdit(self.formFrame)
    self.std_lastname_box.setObjectName(u"std_lastname_box")

    self.formLayout.addWidget(2, QFormLayout.FieldRole,
self.std_lastname_box)

    self.label_4 = QLabel(self.formFrame)
    self.label_4.setObjectName(u"label_4")

```



```

self.formLayout.addWidget(3, QFormLayout.LabelRole, self.label_4)

self.std_fname_box = QLineEdit(self.formFrame)
self.std_fname_box.setObjectName(u"std_fname_box")

self.formLayout.addWidget(3, QFormLayout.FieldRole, self.std_fname_box)

self.label_5 = QLabel(self.formFrame)
self.label_5.setObjectName(u"label_5")

self.formLayout.addWidget(4, QFormLayout.LabelRole, self.label_5)

self.std_mname_box = QLineEdit(self.formFrame)
self.std_mname_box.setObjectName(u"std_mname_box")

self.formLayout.addWidget(4, QFormLayout.FieldRole, self.std_mname_box)

self.label_6 = QLabel(self.formFrame)
self.label_6.setObjectName(u"label_6")

self.formLayout.addWidget(5, QFormLayout.LabelRole, self.label_6)

self.std_gender_box = QComboBox(self.formFrame)
self.std_gender_box.addItem("")
self.std_gender_box.addItem("")
self.std_gender_box.setObjectName(u"std_gender_box")
self.std_gender_box.setStyleSheet(u"/* Styling the QComboBox */\n"
"QComboBox {\n"
"  background-color: #f4f4f4; /* Light grey background */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 2px solid #c0c0c0; /* Border color */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 5px;\n"
"  font: 14pt \"Segoe UI\";\n"
"\n"
"  min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
"/* Styling the combo box when focused */\n"
"QComboBox:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
"  background-color: #e0e0e0; /* Grey background when disabled */\n"
"  color: #888888; /* Light grey text when disabled */\n"
"  border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"

```



```

"/ * Styling the QLineEdit */\n"
"QLineEdit {\n"
"    background-color: #f4f4f4; /* Light grey background */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 2px solid #c0c0c0; /* Border color */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 5px;\n"
"}\n"
"\n"
"/ * Focused state (when the QLineEdit is selected) */\n"
"QLineEdit:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/ * Disabled state */\n"
"QLineEdit:disabled {\n"
"    background-color: #e0e0e0; /* Grey background when disabled */\n"
"    color: #888888; /* Light grey text when disabled */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/ * Placeholder text style */\n"
"QLineEdit::placeholder {\n"
"    color: #888888; /* Light grey color for placeholder text */\n"
"    font-style: italic; /* Italicize the placeholder text */\n"
""
    "}\n"
"\n"
"/ * Styling the text cursor */\n"
"QLineEdit::cursor {\n"
"    color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
""
    self.formLayout_2 = QFormLayout(self.formFrame_2)
    self.formLayout_2.setObjectName(u"formLayout_2")
    self.formLayout_2.setContentsMargins(-1, 0, 0, 0)
    self.label_11 = QLabel(self.formFrame_2)
    self.label_11.setObjectName(u"label_11")

    self.formLayout_2.setWidget(0, QFormLayout.LabelRole, self.label_11)

    self.label_12 = QLabel(self.formFrame_2)
    self.label_12.setObjectName(u"label_12")

    self.formLayout_2.setWidget(2, QFormLayout.LabelRole, self.label_12)

    self.std_contact_box = QLineEdit(self.formFrame_2)
    self.std_contact_box.setObjectName(u"std_contact_box")

```

```

self.formLayout_2.setWidget(2, QFormLayout.FieldRole,
self.std_contact_box)

self.label_17 = QLabel(self.formFrame_2)
self.label_17.setObjectName(u"label_17")

self.formLayout_2.setWidget(3, QFormLayout.LabelRole, self.label_17)

self.std_email_box = QLineEdit(self.formFrame_2)
self.std_email_box.setObjectName(u"std_email_box")

self.formLayout_2.setWidget(3, QFormLayout.FieldRole, self.std_email_box)

self.label_13 = QLabel(self.formFrame_2)
self.label_13.setObjectName(u"label_13")

self.formLayout_2.setWidget(4, QFormLayout.LabelRole, self.label_13)

self.label_14 = QLabel(self.formFrame_2)
self.label_14.setObjectName(u"label_14")

self.formLayout_2.setWidget(7, QFormLayout.LabelRole, self.label_14)

self.std_address_box = QPlainTextEdit(self.formFrame_2)
self.std_address_box.setObjectName(u"std_address_box")
self.std_address_box.setMaximumSize(QSize(16777215, 80))
self.std_address_box.setStyleSheet(u"/* Styling the QPlainTextEdit */\n"
"QPlainTextEdit {\n"
"  background-color: #f4f4f4; /* Light grey background */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 2px solid #c0c0c0; /* Border color */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 8px;\n"
"  font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/* Focused state (when the QPlainTextEdit is selected) */\n"
"QPlainTextEdit:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state */\n"
"QPlainTextEdit:disabled {\n"
"  background-color: #e0e0e0; /* Grey background when disabled */\n"
"  color: #888888; /* Light grey text when disabled */\n"
"  border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"

```

```

"/ * Styling the text cursor */\n"
"QPlainTextEdit::cursor {\n"
"    color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
"\n"
"/ * Styling the selected text */\n"
"QPlainTextEdit::"
"    selection {\n"
"        background-color: #64B5F6; /* Light blue background for selected text */\n"
"        color: #ffffff; /* White text color for selected text */\n"
"}\n"
"\n"
"/ * Styling the scroll bar */\n"
"QPlainTextEdit QScrollBar:vertical {\n"
"    border: none;\n"
"    background: #f4f4f4;\n"
"    width: 12px;\n"
"    margin: 0px, 0px, 0px, 0px;\n"
"    border-radius: 6px;\n"
"}\n"
"\n"
"QPlainTextEdit QScrollBar::handle:vertical {\n"
"    background: #2196F3;\n"
"    border-radius: 6px;\n"
"}\n"
"\n"
"QPlainTextEdit QScrollBar::add-line:vertical, \n"
"QPlainTextEdit QScrollBar::sub-line:vertical {\n"
"    border: none;\n"
"    background: none;\n"
"}\n"
""
)

```

```

self.formLayout_2.addWidget(0, QFormLayout.FieldRole,
self.std_address_box)

```

```

self.std_class_box = QComboBox(self.formFrame_2)
self.std_class_box.addItem("")
self.std_class_box.addItem("")
self.std_class_box.addItem("")
self.std_class_box.addItem("")
self.std_class_box.addItem("")
self.std_class_box.addItem("")
self.std_class_box.addItem("")
self.std_class_box.addItem("")
self.std_class_box.addItem("")
self.std_class_box.addItem("")
self.std_class_box.addItem("")
self.std_class_box.setObjectName(u"std_class_box")

```

```

        self.std_class_box.setStyleSheet(u"/* Styling the QComboBox */\n"
"QComboBox {\n"
"    background-color: #f4f4f4; /* Light grey background */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 2px solid #c0c0c0; /* Border color */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 5px;\n"
"    font: 14pt \"Segoe UI\";\n"
"\n"
"    min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
"/* Styling the combo box when focused */\n"
"QComboBox:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
"    background-color: #e0e0e0; /* Grey background when disabled */\n"
"    color: #888888; /* Light grey text when disabled */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Arrow button of the combo box */\n"
"\n"
"\n"
"/* Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
"    background-color\n"
"        : #ffffff; /* White background for the dropdown list */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
"    border-radius: 5px;\n"
"    padding: 5px;\n"
"    min-width: 150px;\n"
"}\n"
"\n"
"/* Hover effect on the dropdown list items */\n"
"QComboBox QAbstractItemView::item:hover {\n"
"    background-color: #2196F3; /* Blue background on hover */\n"
"    color: #ffffff; /* White text on hover */\n"
"}\n"
"\n"
"/* Selected item in the dropdown list */\n"
"QComboBox QAbstractItemView::item:selected {\n"
"    background-color: #64B5F6; /* Light blue background when selected */\n"
"    color: #ffffff; /* White text when selected */\n"
"}\n"

```

""))

```
self.formLayout_2.addWidget(4, QFormLayout.FieldRole, self.std_class_box)

self.std_section_box = QComboBox(self.formFrame_2)
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.setObjectName(u"std_section_box")
self.std_section_box.setStyleSheet(u"/* Styling the QComboBox */\n"
"QComboBox {\n"
"  background-color: #f4f4f4; /* Light grey background */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 2px solid #c0c0c0; /* Border color */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 5px;\n"
"  font: 14pt \"Segoe UI\";\n"
"\n"
"  min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
"/* Styling the combo box when focused */\n"
"QComboBox:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
"  background-color: #e0e0e0; /* Grey background when disabled */\n"
"  color: #888888; /* Light grey text when disabled */\n"
"  border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Arrow button of the combo box */\n"
"\n"
"\n"
"/* Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
"  background-color\n"
"    : #ffffff; /* White background for the dropdown list */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
"  border-radius: 5px;\n"
"  padding: 5px;\n"
"  min-width: 150px;\n"
"}
```

```

"}\n"
"\n"
"/\n"
"QComboBox QAbstractItemView::item: hover {\n"
"    background-color: #2196F3; /\n"
"    color: #ffffff; /\n"
"}\n"
"\n"
"/\n"
"QComboBox QAbstractItemView::item:selected {\n"
"    background-color: #64B5F6; /\n"
"    color: #ffffff; /\n"
"}\n"
""
)

```

```

self.formLayout_2.addWidget(7, QFormLayout.FieldRole,
self.std_section_box)

```

```

self.horizontalLayout_2.addWidget(self.formFrame_2, 0, Qt.AlignTop)

```

```

self.verticalLayout.addLayout(self.horizontalLayout_2)

```

```

self.horizontalLayout_3 = QHBoxLayout()
self.horizontalLayout_3.setObjectName(u"horizontalLayout_3")
self.horizontalLayout_3.setContentsMargins(-1, 0, -1, -1)
self.horizontalSpacer = QSpacerItem(40, 20, QSizePolicy.Expanding,
QSizePolicy.Minimum)

```

```

self.horizontalLayout_3.addItem(self.horizontalSpacer)

```

```

self.save_btn = QPushButton(self.tab)
self.save_btn.setObjectName(u"save_btn")
self.save_btn.setStyleSheet(u"/\n"
"QPushButton {\n"
"    background-color: #4CAF50; /\n"
"    color: white; /\n"
"    border: 2px solid #4CAF50; /\n"
"    border-radius: 5px; /\n"
"    padding: 10px 20px; /\n"
"    \n"
"    font: 14pt \"Segoe UI\"; /\n"
"    min-width: 100px; /\n"
"}\n"
"\n"
"/\n"
"QPushButton: hover {\n"
"    background-color: #45a049; /\n"
"    border-color: #45a049; /\n"
}

```



```

"}\n"
"\n"
"/ * Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
"    background-color: #388e3c; /* Even darker green */\n"
"    border-color: #388e3c; /* Dark border */\n"
"    padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
"    padding-left: 18px; /* Shift the button to the left slightly */\n"
"}\n"
"\n"
"/ * Focused state (when the button is focused) */\n"
"QPushButton:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    outline: none; /* Removes the default outline */\n"
"}\n"
"\n"
"/ * Disabled state for the QPushButton */\n"
"QPushButton:disabled {\n"
"    background-color: #e0e0e0; /* Light grey background */\n"
"    color: #888888; /* Light grey text */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/ * Styling the text of the QPushButton */\n"
"QPushButton {\n"
"    font-weight: bold; /* Make the text bold */\n"
"}\n"
"\n"
"/ * Styling the QPushButton icon (if it has one) */\n"
"QPushButton::icon {\n"
"    padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"
"\n"
"/ * Border effect when the button is in a focus or pressed state */\n"
"QPushButton:focus, QPushButton:pressed {\n"
"    border-style: inset;\n"
"}\n"
""

```

```

self.horizontalLayout_3.addWidget(self.save_btn)

```

```

self.verticalLayout.addLayout(self.horizontalLayout_3)

```

```

self.admission_table = QTableWidgetItem(self.tab)
if (self.admission_table.columnCount() < 6):
    self.admission_table.setColumnCount(6)
    _qtablewidgetitem = QTableWidgetItem()

```

```

self.admission_table.setHorizontalHeaderItem(0, __qtablewidgetitem)
__qtablewidgetitem1 = QTableWidgetItem()
self.admission_table.setHorizontalHeaderItem(1, __qtablewidgetitem1)
__qtablewidgetitem2 = QTableWidgetItem()
self.admission_table.setHorizontalHeaderItem(2, __qtablewidgetitem2)
__qtablewidgetitem3 = QTableWidgetItem()
self.admission_table.setHorizontalHeaderItem(3, __qtablewidgetitem3)
__qtablewidgetitem4 = QTableWidgetItem()
self.admission_table.setHorizontalHeaderItem(4, __qtablewidgetitem4)
__qtablewidgetitem5 = QTableWidgetItem()
self.admission_table.setHorizontalHeaderItem(5, __qtablewidgetitem5)
if (self.admission_table.rowCount() < 3):
    self.admission_table.setRowCount(3)
self.admission_table.setObjectName(u"admission_table")
font = QFont()
font.setPointSize(14)
font.setBold(False)
self.admission_table.setFont(font)
self.admission_table.setStyleSheet(u"QHeaderView::section {\n"
"background-color: rgb(0, 120, 212);\n"
" /* Green background for headers */\n"
" color: white; /* White text color */\n"
" font-weight: bold; /* Bold font for header text */\n"
" text-align: center; /* Center-align text */\n"
" \n"
" font: 700 14pt \"Segoe UI\";\n"
" border: 1px solid #ccc; /* Light gray border around header sections */\n"
" padding: 5px; /* Padding inside header cells */\n"
"}\n"
"")

```

```

self.verticalLayout.addWidget(self.admission_table)

```

```

self.verticalSpacer = QSpacerItem(20, 40, QSizePolicy.Minimum,
QSizePolicy.Expanding)

```

```

self.verticalLayout.addItem(self.verticalSpacer)

```

```

self.tabWidget.addTab(self.tab, "")

```

```

self.verticalLayout_2.addWidget(self.tabWidget)

```

```

MainWindow.setCentralWidget(self.centralwidget)
QWidget.setTabOrder(self.std_firstname_box, self.std_lastname_box)
QWidget.setTabOrder(self.std_lastname_box, self.std_fname_box)
QWidget.setTabOrder(self.std_fname_box, self.std_mname_box)
QWidget.setTabOrder(self.std_mname_box, self.std_gender_box)
QWidget.setTabOrder(self.std_gender_box, self.std_dob_box)
QWidget.setTabOrder(self.std_dob_box, self.std_address_box)
QWidget.setTabOrder(self.std_address_box, self.std_contact_box)

```

```

QWidget.setTabOrder(self.std_contact_box, self.std_email_box)
QWidget.setTabOrder(self.std_email_box, self.std_class_box)
QWidget.setTabOrder(self.std_class_box, self.std_section_box)
QWidget.setTabOrder(self.std_section_box, self.save_btn)
QWidget.setTabOrder(self.save_btn, self.tabWidget)

self.retranslateUi(MainWindow)

self.tabWidget.setCurrentIndex(0)

QMetaObject.connectSlotsByName(MainWindow)
# setupUi

def retranslateUi(self, MainWindow):
    MainWindow.setWindowTitle(QCoreApplication.translate("MainWindow",
u"Admission Page", None))
    self.label_2.setText(QCoreApplication.translate("MainWindow", u"Student
ID", None))
    self.std_id_label.setText(QCoreApplication.translate("MainWindow",
u"RG-1209", None))
    self.label.setText(QCoreApplication.translate("MainWindow", u"First
Name", None))
    self.label_3.setText(QCoreApplication.translate("MainWindow", u"Last
Name", None))
    self.label_4.setText(QCoreApplication.translate("MainWindow", u"Father's
Name", None))
    self.label_5.setText(QCoreApplication.translate("MainWindow",
u"Mother's Name", None))
    self.label_6.setText(QCoreApplication.translate("MainWindow",
u"Gender", None))
    self.std_gender_box.setItemText(0,
QCoreApplication.translate("MainWindow", u"Male", None))
    self.std_gender_box.setItemText(1,
QCoreApplication.translate("MainWindow", u"Female", None))

    self.label_7.setText(QCoreApplication.translate("MainWindow", u"DOB",
None))
    self.label_11.setText(QCoreApplication.translate("MainWindow",
u"Address", None))
    self.label_12.setText(QCoreApplication.translate("MainWindow",
u"Contact No.", None))
    self.label_17.setText(QCoreApplication.translate("MainWindow",
u"Email", None))
    self.label_13.setText(QCoreApplication.translate("MainWindow",
u"Standard", None))
    self.label_14.setText(QCoreApplication.translate("MainWindow",
u"Division", None))
    self.std_class_box.setItemText(0,
QCoreApplication.translate("MainWindow", u"Class 1", None))

```

```

        self.std_class_box.setItemText(1,
QtCoreApplication.translate("MainWindow", u"Class 2", None))
        self.std_class_box.setItemText(2,
QtCoreApplication.translate("MainWindow", u"Class 3", None))
        self.std_class_box.setItemText(3,
QtCoreApplication.translate("MainWindow", u"Class 4", None))
        self.std_class_box.setItemText(4,
QtCoreApplication.translate("MainWindow", u"Class 5", None))
        self.std_class_box.setItemText(5,
QtCoreApplication.translate("MainWindow", u"Class 6", None))
        self.std_class_box.setItemText(6,
QtCoreApplication.translate("MainWindow", u"Class 7", None))
        self.std_class_box.setItemText(7,
QtCoreApplication.translate("MainWindow", u"Class 8", None))
        self.std_class_box.setItemText(8,
QtCoreApplication.translate("MainWindow", u"Class 9", None))
        self.std_class_box.setItemText(9,
QtCoreApplication.translate("MainWindow", u"Class 10", None))
        self.std_class_box.setItemText(10,
QtCoreApplication.translate("MainWindow", u"Class 11", None))
        self.std_class_box.setItemText(11,
QtCoreApplication.translate("MainWindow", u"Class 12", None))

```

```

        self.std_section_box.setItemText(0,
QtCoreApplication.translate("MainWindow", u"A", None))
        self.std_section_box.setItemText(1,
QtCoreApplication.translate("MainWindow", u"B", None))
        self.std_section_box.setItemText(2,
QtCoreApplication.translate("MainWindow", u"C", None))
        self.std_section_box.setItemText(3,
QtCoreApplication.translate("MainWindow", u"D", None))
        self.std_section_box.setItemText(4,
QtCoreApplication.translate("MainWindow", u"E", None))
        self.std_section_box.setItemText(5,
QtCoreApplication.translate("MainWindow", u"F", None))
        self.std_section_box.setItemText(6,
QtCoreApplication.translate("MainWindow", u"G", None))

```

```

        self.save_btn.setText(QtCoreApplication.translate("MainWindow", u"Save
Details", None))
        __qtablewidgetitem = self.admission_table.horizontalHeaderItem(0)
        __qtablewidgetitem.setText(QtCoreApplication.translate("MainWindow",
u"Student ID", None));
        __qtablewidgetitem1 = self.admission_table.horizontalHeaderItem(1)
        __qtablewidgetitem1.setText(QtCoreApplication.translate("MainWindow",
u"Standard", None));
        __qtablewidgetitem2 = self.admission_table.horizontalHeaderItem(2)
        __qtablewidgetitem2.setText(QtCoreApplication.translate("MainWindow",
u"Division", None));
        __qtablewidgetitem3 = self.admission_table.horizontalHeaderItem(3)

```

```
____qtablewidgetitem3.setText(QCoreApplication.translate("MainWindow",  
u"Student Name", None));  
____qtablewidgetitem4 = self.admission_table.horizontalHeaderItem(4)  
____qtablewidgetitem4.setText(QCoreApplication.translate("MainWindow",  
u"Gender", None));  
____qtablewidgetitem5 = self.admission_table.horizontalHeaderItem(5)  
____qtablewidgetitem5.setText(QCoreApplication.translate("MainWindow",  
u"DOB", None));  
self.tabWidget.setTabText(self.tabWidget.indexOf(self.tab),  
QCoreApplication.translate("MainWindow", u"Enter Student Details", None))  
# retranslateUi
```

13.10 ui_edit_staff.py

```
# -*- coding: utf-8 -*-

#####
#####
## Form generated from reading UI file 'edit_staffQFlpGQ.ui'
##
## Created by: Qt User Interface Compiler version 6.4.0
##
## WARNING! All changes made in this file will be lost when recompiling UI file!
#####
#####

from PySide6.QtCore import (QCoreApplication, QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt)
from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
    QFont, QFontDatabase, QGradient, QIcon,
    QImage, QKeySequence, QLinearGradient, QPainter,
    QPalette, QPixmap, QRadialGradient, QTransform)
from PySide6.QtWidgets import (QApplication, QComboBox, QFormLayout,
    QFrame,
    QHBoxLayout, QLabel, QLineEdit, QMainWindow,
    QPlainTextEdit, QPushButton, QSizePolicy, QSpacerItem,
    QTabWidget, QVBoxLayout, QWidget)

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.setObjectName():
            MainWindow.setObjectName(u"MainWindow")
        MainWindow.resize(800, 600)
        self.centralwidget = QWidget(MainWindow)
        self.centralwidget.setObjectName(u"centralwidget")
        self.horizontalLayout = QHBoxLayout(self.centralwidget)
        self.horizontalLayout.setObjectName(u"horizontalLayout")
        self.tabWidget = QTabWidget(self.centralwidget)
        self.tabWidget.setObjectName(u"tabWidget")
        self.tabWidget.setStyleSheet(u"/* Styling the Tab Widget */\n"
"QTabWidget {\n"
"    background-color: #f0f0f0;\n"
"    border: 2px solid black;\n"
"    border-radius: 10px;\n"
"    padding: 5px;\n"
"}\n"
"\n"
"\n"
"/* Styling the Tabs */\n"
"QTabBar::tab {\n"
"    background-color: #2196F3; /* Tab background color */\n"
"    color: white;\n"
"
```

```

" border: 1px solid #c0c0c0;\n"
" padding: 10px;\n"
" border-radius: 5px;\n"
" min-width: 80px;\n"
" min-height: 30px;\n"
"\n"
" font: 14pt \"Segoe UI\";\n"
"}\n"
"\n"
""))
    self.tab = QWidget()
    self.tab.setObjectName(u"tab")
    self.tab.setStyleSheet(u"")
    self.verticalLayout = QVBoxLayout(self.tab)
    self.verticalLayout.setObjectName(u"verticalLayout")
    self.horizontalLayout_2 = QHBoxLayout()
    self.horizontalLayout_2.setObjectName(u"horizontalLayout_2")
    self.horizontalLayout_2.setContentsMargins(-1, 0, -1, -1)
    self.formFrame = QFrame(self.tab)
    self.formFrame.setObjectName(u"formFrame")
    self.formFrame.setStyleSheet(u"*{\n"
"font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/* Styling the QLineEdit */\n"
"QLineEdit {\n"
" background-color: #f4f4f4; /* Light grey background */\n"
" color: #333333; /* Dark text color */\n"
" border: 2px solid #c0c0c0; /* Border color */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 5px;\n"
"}\n"
"\n"
"/* Focused state (when the QLineEdit is selected) */\n"
"QLineEdit:focus {\n"
" border: 2px solid #2196F3; /* Blue border when focused */\n"
" background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state */\n"
"QLineEdit:disabled {\n"
" background-color: #e0e0e0; /* Grey background when disabled */\n"
" color: #888888; /* Light grey text when disabled */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Placeholder text style */\n"
"QLineEdit::placeholder {\n"
" color: #888888; /* Light grey color for placeholder text */\n"

```



```

" font-style: italic; /* Italicize the placeholder text */\n"
""

        "}\n"
"\n"
/* Styling the text cursor */\n"
"QLineEdit::cursor {\n"
" color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
""
)

    self.formLayout = QFormLayout(self.formFrame)
    self.formLayout.setObjectName(u"formLayout")
    self.formLayout.setContentsMargins(-1, 0, 0, 0)
    self.label_2 = QLabel(self.formFrame)
    self.label_2.setObjectName(u"label_2")

    self.formLayout.addWidget(1, QFormLayout.LabelRole, self.label_2)

    self.label_4 = QLabel(self.formFrame)
    self.label_4.setObjectName(u"label_4")

    self.formLayout.addWidget(2, QFormLayout.LabelRole, self.label_4)

    self.staff_category_box = QComboBox(self.formFrame)
    self.staff_category_box.addItem("")
    self.staff_category_box.addItem("")
    self.staff_category_box.setObjectName(u"staff_category_box")
    self.staff_category_box.setStyleSheet(u"/* Styling the QComboBox */\n"
"QComboBox {\n"
" background-color: #f4f4f4; /* Light grey background */\n"
" color: #333333; /* Dark text color */\n"
" border: 2px solid #c0c0c0; /* Border color */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 5px;\n"
" font: 14pt \"Segoe UI\";\n"
"\n"
" min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
/* Styling the combo box when focused */\n"
"QComboBox:focus {\n"
" border: 2px solid #2196F3; /* Blue border when focused */\n"
" background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
/* Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
" background-color: #e0e0e0; /* Grey background when disabled */\n"
" color: #888888; /* Light grey text when disabled */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"

```



```

"\n"
"/ * Arrow button of the combo box */\n"
"\n"
"\n"
"/ * Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
"    background-color"
        ": #ffffff; /* White background for the dropdown list */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
"    border-radius: 5px;\n"
"    padding: 5px;\n"
"    min-width: 150px;\n"
"}\n"
"\n"
"/ * Hover effect on the dropdown list items */\n"
"QComboBox QAbstractItemView::item:hover {\n"
"    background-color: #2196F3; /* Blue background on hover */\n"
"    color: #ffffff; /* White text on hover */\n"
"}\n"
"\n"
"/ * Selected item in the dropdown list */\n"
"QComboBox QAbstractItemView::item:selected {\n"
"    background-color: #64B5F6; /* Light blue background when selected */\n"
"    color: #ffffff; /* White text when selected */\n"
"}\n"
""
)

    self.formLayout.setWidget(2, QFormLayout.FieldRole,
self.staff_category_box)

    self.label = QLabel(self.formFrame)
    self.label.setObjectName(u"label")

    self.formLayout.setWidget(3, QFormLayout.LabelRole, self.label)

    self.staff_fname_box = QLineEdit(self.formFrame)
    self.staff_fname_box.setObjectName(u"staff_fname_box")

    self.formLayout.setWidget(3, QFormLayout.FieldRole, self.staff_fname_box)

    self.label_3 = QLabel(self.formFrame)
    self.label_3.setObjectName(u"label_3")

    self.formLayout.setWidget(4, QFormLayout.LabelRole, self.label_3)

    self.staff_lname_box = QLineEdit(self.formFrame)
    self.staff_lname_box.setObjectName(u"staff_lname_box")

    self.formLayout.setWidget(4, QFormLayout.FieldRole, self.staff_lname_box)

```

```

self.label_6 = QLabel(self.formFrame)
self.label_6.setObjectName(u"label_6")

self.formLayout.addWidget(5, QFormLayout.LabelRole, self.label_6)

self.staff_gen_box = QComboBox(self.formFrame)
self.staff_gen_box.addItem("")
self.staff_gen_box.addItem("")
self.staff_gen_box.setObjectName(u"staff_gen_box")
self.staff_gen_box.setStyleSheet(u"/** Styling the QComboBox */\n"
"QComboBox {\n"
"  background-color: #f4f4f4; /* Light grey background */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 2px solid #c0c0c0; /* Border color */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 5px;\n"
"  font: 14pt \"Segoe UI\";\n"
"\n"
"  min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
"/** Styling the combo box when focused */\n"
"QComboBox:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/** Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
"  background-color: #e0e0e0; /* Grey background when disabled */\n"
"  color: #888888; /* Light grey text when disabled */\n"
"  border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/** Arrow button of the combo box */\n"
"\n"
"/** Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
"  background-color\n"
"      : #ffffff; /* White background for the dropdown list */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
"  border-radius: 5px;\n"
"  padding: 5px;\n"
"  min-width: 150px;\n"
"}\n"
"\n"
"/** Hover effect on the dropdown list items */\n"

```

```

"QComboBox QAbstractItemView::item:hover {\n"
"  background-color: #2196F3; /* Blue background on hover */\n"
"  color: #ffffff; /* White text on hover */\n"
"}\n"
"\n"
"/* Selected item in the dropdown list */\n"
"QComboBox QAbstractItemView::item:selected {\n"
"  background-color: #64B5F6; /* Light blue background when selected */\n"
"  color: #ffffff; /* White text when selected */\n"
"}\n"
""")

self.formLayout.addWidget(5, QFormLayout.FieldRole, self.staff_gen_box)

self.label_7 = QLabel(self.formFrame)
self.label_7.setObjectName(u"label_7")

self.formLayout.addWidget(7, QFormLayout.LabelRole, self.label_7)

self.staff_dob_box = QLineEdit(self.formFrame)
self.staff_dob_box.setObjectName(u"staff_dob_box")

self.formLayout.addWidget(7, QFormLayout.FieldRole, self.staff_dob_box)

self.lineEdit = QLineEdit(self.formFrame)
self.lineEdit.setObjectName(u"lineEdit")

self.formLayout.addWidget(1, QFormLayout.FieldRole, self.lineEdit)

self.horizontalLayout_2.addWidget(self.formFrame, 0, Qt.AlignTop)

self.formFrame_2 = QFrame(self.tab)
self.formFrame_2.setObjectName(u"formFrame_2")
self.formFrame_2.setStyleSheet(u"{\n"
"font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/* Styling the QLineEdit */\n"
"QLineEdit {\n"
"  background-color: #f4f4f4; /* Light grey background */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 2px solid #c0c0c0; /* Border color */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 5px;\n"
"}\n"
"\n"
"/* Focused state (when the QLineEdit is selected) */\n"
"QLineEdit:focus {\n"

```

```

"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
/* Disabled state */\n"
"QLineEdit:disabled {\n"
"    background-color: #e0e0e0; /* Grey background when disabled */\n"
"    color: #888888; /* Light grey text when disabled */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
/* Placeholder text style */\n"
"QLineEdit::placeholder {\n"
"    color: #888888; /* Light grey color for placeholder text */\n"
"    font-style: italic; /* Italicize the placeholder text */\n"
""
    "}\n"
"\n"
/* Styling the text cursor */\n"
"QLineEdit::cursor {\n"
"    color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
""
)

self.formLayout_2 = QFormLayout(self.formFrame_2)
self.formLayout_2.setObjectName(u"formLayout_2")
self.formLayout_2.setContentsMargins(-1, 0, 0, 0)
self.label_11 = QLabel(self.formFrame_2)
self.label_11.setObjectName(u"label_11")

self.formLayout_2.addWidget(0, QFormLayout.LabelRole, self.label_11)

self.label_12 = QLabel(self.formFrame_2)
self.label_12.setObjectName(u"label_12")

self.formLayout_2.addWidget(2, QFormLayout.LabelRole, self.label_12)

self.staff_contact_box = QLineEdit(self.formFrame_2)
self.staff_contact_box.setObjectName(u"staff_contact_box")

self.formLayout_2.addWidget(2, QFormLayout.FieldRole,
self.staff_contact_box)

self.label_17 = QLabel(self.formFrame_2)
self.label_17.setObjectName(u"label_17")

self.formLayout_2.addWidget(3, QFormLayout.LabelRole, self.label_17)

self.staff_email_box = QLineEdit(self.formFrame_2)
self.staff_email_box.setObjectName(u"staff_email_box")

```

```

self.formLayout_2.addWidget(3, QFormLayout.FieldRole,
self.staff_email_box)

self.label_13 = QLabel(self.formFrame_2)
self.label_13.setObjectName(u"label_13")

self.formLayout_2.addWidget(4, QFormLayout.LabelRole, self.label_13)

self.label_14 = QLabel(self.formFrame_2)
self.label_14.setObjectName(u"label_14")

self.formLayout_2.addWidget(8, QFormLayout.LabelRole, self.label_14)

self.staff_address_box = QPlainTextEdit(self.formFrame_2)
self.staff_address_box.setObjectName(u"staff_address_box")
self.staff_address_box.setMaximumSize(QSize(16777215, 80))
self.staff_address_box.setStyleSheet(u"/* Styling the QPlainTextEdit */\n"
"QPlainTextEdit {\n"
"  background-color: #f4f4f4; /* Light grey background */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 2px solid #c0c0c0; /* Border color */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 8px;\n"
"  font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/* Focused state (when the QPlainTextEdit is selected) */\n"
"QPlainTextEdit:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state */\n"
"QPlainTextEdit:disabled {\n"
"  background-color: #e0e0e0; /* Grey background when disabled */\n"
"  color: #888888; /* Light grey text when disabled */\n"
"  border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Styling the text cursor */\n"
"QPlainTextEdit::cursor {\n"
"  color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
"\n"
"/* Styling the selected text */\n"
"QPlainTextEdit::"
    "selection {\n"
"  background-color: #64B5F6; /* Light blue background for selected text */\n"
"  color: #ffffff; /* White text color for selected text */\n"

```

```

"}\n"
"\n"
"/** Styling the scroll bar */\n"
"QPlainTextEdit QScrollBar:vertical {\n"
"    border: none;\n"
"    background: #f4f4f4;\n"
"    width: 12px;\n"
"    margin: 0px, 0px, 0px, 0px;\n"
"    border-radius: 6px;\n"
"}\n"
"\n"
"QPlainTextEdit QScrollBar::handle:vertical {\n"
"    background: #2196F3;\n"
"    border-radius: 6px;\n"
"}\n"
"\n"
"QPlainTextEdit QScrollBar::add-line:vertical, \n"
"QPlainTextEdit QScrollBar::sub-line:vertical {\n"
"    border: none;\n"
"    background: none;\n"
"}\n"
""")

```

```

self.formLayout_2.setWidget(0, QFormLayout.FieldRole,
self.staff_address_box)

```

```

self.staff_dept_box = QComboBox(self.formFrame_2)
self.staff_dept_box.addItem("")
self.staff_dept_box.addItem("")
self.staff_dept_box.addItem("")
self.staff_dept_box.addItem("")
self.staff_dept_box.addItem("")
self.staff_dept_box.addItem("")
self.staff_dept_box.addItem("")
self.staff_dept_box.addItem("")
self.staff_dept_box.addItem("")
self.staff_dept_box.addItem("")
self.staff_dept_box.setObjectName(u"staff_dept_box")
self.staff_dept_box.setStyleSheet(u"/** Styling the QComboBox */\n"
"QComboBox {\n"
"    background-color: #f4f4f4; /* Light grey background */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 2px solid #c0c0c0; /* Border color */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 5px;\n"
"    font: 14pt \"Segoe UI\";\n"
"\n"
"    min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"

```

```

"/* Styling the combo box when focused */\n"
"QComboBox:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
"    background-color: #e0e0e0; /* Grey background when disabled */\n"
"    color: #888888; /* Light grey text when disabled */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Arrow button of the combo box */\n"
"\n"
"/* Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
"    background-color\n"
"        ": #ffffff; /* White background for the dropdown list */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
"    border-radius: 5px;\n"
"    padding: 5px;\n"
"    min-width: 150px;\n"
"}\n"
"\n"
"/* Hover effect on the dropdown list items */\n"
"QComboBox QAbstractItemView::item:hover {\n"
"    background-color: #2196F3; /* Blue background on hover */\n"
"    color: #ffffff; /* White text on hover */\n"
"}\n"
"\n"
"/* Selected item in the dropdown list */\n"
"QComboBox QAbstractItemView::item:selected {\n"
"    background-color: #64B5F6; /* Light blue background when selected */\n"
"    color: #ffffff; /* White text when selected */\n"
"}\n"
""
)

```

```

        self.formLayout_2.setWidget(4, QFormLayout.FieldRole,
self.staff_dept_box)

```

```

        self.staff_sal_box = QLineEdit(self.formFrame_2)
        self.staff_sal_box.setObjectName(u"staff_sal_box")

```

```

        self.formLayout_2.setWidget(8, QFormLayout.FieldRole, self.staff_sal_box)

```

```

        self.horizontalLayout_2.addWidget(self.formFrame_2, 0, Qt.AlignTop)

```

```

self.verticalLayout.addLayout(self.horizontalLayout_2)

self.horizontalLayout_3 = QHBoxLayout()
self.horizontalLayout_3.setObjectName(u"horizontalLayout_3")
self.horizontalLayout_3.setContentsMargins(-1, 0, -1, -1)
self.horizontalSpacer = QSpacerItem(40, 20, QSizePolicy.Expanding,
QSizePolicy.Minimum)

self.horizontalLayout_3.addItem(self.horizontalSpacer)

self.fetch_btn = QPushButton(self.tab)
self.fetch_btn.setObjectName(u"fetch_btn")
self.fetch_btn.setStyleSheet(u"/* Styling the QPushButton */\n"
"QPushButton {\n"
"  background-color: #4CAF50; /* Green background */\n"
"  color: white; /* White text color */\n"
"  border: 2px solid #4CAF50; /* Border color matches background */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 10px 20px; /* Vertical and horizontal padding */\n"
"  \n"
"  font: 14pt \"Segoe UI\"; /* Font size */\n"
"  min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
"/* Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
"  background-color: #45a049; /* Darker green background */\n"
"  border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
"/* Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
"  background-color: #388e3c; /* Even darker green */\n"
"  border-color: #388e3c; /* Dark border */\n"
"  padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
"  padding-left: 18px; /* Shift the button to the left slightly */\n"
"}\n"
"\n"
"/* Focused state (when the button is focused) */\n"
"QPushButton:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  outline: none; /* Removes the default outline */\n"
"}\n"
"\n"
"/* Disabled state for the QPushButton */\n"
"QPushButton:disabled {\n"

```



```

" background-color: #e0e0e0; /* Light grey background */\n"
" color: #888888; /* Light grey text */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
/* Styling the text of the QPushButton */\n"
"QPushButton {\n"
" font-weight: bold; /* Make the text bold */\n"
"}\n"
"\n"
/* Styling the QPushButton icon (if it has one) */\n"
"QPushButton::icon {\n"
" padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"
"\n"
/* Border effect when the button is in a focus or pressed state */\n"
"QPushButton:focus, QPushButton:pressed {\n"
" border-style: inset;\n"
"}\n"
""
)

self.horizontalLayout_3.addWidget(self.fetch_btn)

self.update_btn = QPushButton(self.tab)
self.update_btn.setObjectName(u"update_btn")
self.update_btn.setStyleSheet(u"/* Styling the QPushButton */\n"
"QPushButton {\n"
" background-color: #4CAF50; /* Green background */\n"
" color: white; /* White text color */\n"
" border: 2px solid #4CAF50; /* Border color matches background */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 10px 20px; /* Vertical and horizontal padding */\n"
" \n"
" font: 14pt \"Segoe UI\"; /* Font size */\n"
" min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
/* Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
" background-color: #45a049; /* Darker green background */\n"
" border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
/* Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
" background-color: #388e3c; /* Even darker green */\n"
" border-color: #388e3c; /* Dark border */\n"
" padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
" padding-left: 18px; /* Shift the button to t"

```

```

        "he left slightly */\n"
    }\n"
\n"
    /* Focused state (when the button is focused) */\n"
    QPushButton:focus {\n"
    "    border: 2px solid #2196F3; /* Blue border when focused */\n"
    "    outline: none; /* Removes the default outline */\n"
    }\n"
\n"
    /* Disabled state for the QPushButton */\n"
    QPushButton:disabled {\n"
    "    background-color: #e0e0e0; /* Light grey background */\n"
    "    color: #888888; /* Light grey text */\n"
    "    border: 2px solid #cccccc; /* Light grey border */\n"
    }\n"
\n"
    /* Styling the text of the QPushButton */\n"
    QPushButton {\n"
    "    font-weight: bold; /* Make the text bold */\n"
    }\n"
\n"
    /* Styling the QPushButton icon (if it has one) */\n"
    QPushButton::icon {\n"
    "    padding-right: 10px; /* Space between the icon and the text */\n"
    }\n"
\n"
    /* Border effect when the button is in a focus or pressed state */\n"
    QPushButton:focus, QPushButton:pressed {\n"
    "    border-style: inset;\n"
    }\n"
    """)

    self.horizontalLayout_3.addWidget(self.update_btn)

    self.verticalLayout.addLayout(self.horizontalLayout_3)

    self.verticalSpacer = QSpacerItem(20, 40, QSizePolicy.Minimum,
    QSizePolicy.Expanding)

    self.verticalLayout.addItem(self.verticalSpacer)

    self.tabWidget.addTab(self.tab, "")

    self.horizontalLayout.addWidget(self.tabWidget)

    MainWindow.setCentralWidget(self.centralwidget)

    self.retranslateUi(MainWindow)

```

```

self.tabWidget.setCurrentIndex(0)

QMetaObject.connectSlotsByName(MainWindow)
# setupUi

def retranslateUi(self, MainWindow):
    MainWindow.setWindowTitle(QCoreApplication.translate("MainWindow",
u"MainWindow", None))
    self.label_2.setText(QCoreApplication.translate("MainWindow", u"Staff
ID", None))
    self.label_4.setText(QCoreApplication.translate("MainWindow",
u"Category", None))
    self.staff_category_box.setItemText(0,
QCoreApplication.translate("MainWindow", u"Teaching Staff", None))
    self.staff_category_box.setItemText(1,
QCoreApplication.translate("MainWindow", u"Non-Teaching Staff", None))

    self.label.setText(QCoreApplication.translate("MainWindow", u"First
Name", None))
    self.label_3.setText(QCoreApplication.translate("MainWindow", u"Last
Name", None))
    self.label_6.setText(QCoreApplication.translate("MainWindow",
u"Gender", None))
    self.staff_gen_box.setItemText(0,
QCoreApplication.translate("MainWindow", u"Male", None))
    self.staff_gen_box.setItemText(1,
QCoreApplication.translate("MainWindow", u"Female", None))

    self.label_7.setText(QCoreApplication.translate("MainWindow", u"DOB",
None))
    self.label_11.setText(QCoreApplication.translate("MainWindow",
u"Address", None))
    self.label_12.setText(QCoreApplication.translate("MainWindow",
u"Contact No.", None))
    self.label_17.setText(QCoreApplication.translate("MainWindow",
u"Email", None))
    self.label_13.setText(QCoreApplication.translate("MainWindow",
u"Department", None))
    self.label_14.setText(QCoreApplication.translate("MainWindow",
u"Salary", None))
    self.staff_dept_box.setItemText(0,
QCoreApplication.translate("MainWindow", u"Primary - 1 to 5", None))
    self.staff_dept_box.setItemText(1,
QCoreApplication.translate("MainWindow", u"Middle High - 6 to 8", None))
    self.staff_dept_box.setItemText(2,
QCoreApplication.translate("MainWindow", u"Secondary - 9 to 10", None))
    self.staff_dept_box.setItemText(3,
QCoreApplication.translate("MainWindow", u"Science", None))

```

```

        self.staff_dept_box.setItemText(4,
QtCoreApplication.translate("MainWindow", u"Commerce", None))
        self.staff_dept_box.setItemText(5,
QtCoreApplication.translate("MainWindow", u"Humanities", None))
        self.staff_dept_box.setItemText(6,
QtCoreApplication.translate("MainWindow", u"Utilities", None))
        self.staff_dept_box.setItemText(7,
QtCoreApplication.translate("MainWindow", u"Lab Assistant", None))
        self.staff_dept_box.setItemText(8,
QtCoreApplication.translate("MainWindow", u"Finance", None))
        self.staff_dept_box.setItemText(9,
QtCoreApplication.translate("MainWindow", u"Office", None))

        self.fetch_btn.setText(QtCoreApplication.translate("MainWindow", u"Fetch
Data", None))
        self.update_btn.setText(QtCoreApplication.translate("MainWindow",
u"Update Details", None))
        self.tabWidget.setTabText(self.tabWidget.indexOf(self.tab),
QtCoreApplication.translate("MainWindow", u"Edit Staff Details", None))
# retranslateUi

```

13.11 ui_edit_stud.py

```

# -*- coding: utf-8 -*-

#####
#####
## Form generated from reading UI file 'edit_studLPYmQp.ui'
##
## Created by: Qt User Interface Compiler version 6.4.0
##
## WARNING! All changes made in this file will be lost when recompiling UI file!
#####
#####

from PySide6.QtCore import (QtCoreApplication, QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt)
from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
    QFont, QFontDatabase, QGradient, QIcon,
    QImage, QKeySequence, QLinearGradient, QPainter,
    QPalette, QPixmap, QRadialGradient, QTransform)
from PySide6.QtWidgets import (QApplication, QComboBox, QFormLayout,
    QFrame,
    QHBoxLayout, QLabel, QLineEdit, QMainWindow,
    QTextEdit, QPushButton, QSizePolicy, QSpacerItem,
    QTabWidget, QVBoxLayout, QWidget)

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.setObjectName():
            MainWindow.setObjectName(u"MainWindow")

```

```

MainWindow.resize(800, 600)
self.centralwidget = QWidget(MainWindow)
self.centralwidget.setObjectName(u"centralwidget")
self.horizontalLayout = QHBoxLayout(self.centralwidget)
self.horizontalLayout.setObjectName(u"horizontalLayout")
self.tabWidget = QTabWidget(self.centralwidget)
self.tabWidget.setObjectName(u"tabWidget")
self.tabWidget.setStyleSheet(u"/* Styling the Tab Widget */\n"
"QTabWidget {\n"
"  background-color: #f0f0f0;\n"
"  border: 2px solid black;\n"
"  border-radius: 10px;\n"
"  padding: 5px;\n"
"}\n"
"\n"
"\n"
"/* Styling the Tabs */\n"
"QTabBar::tab {\n"
"  background-color: #2196F3; /* Tab background color */\n"
"  color: white;\n"
"  border: 1px solid #c0c0c0;\n"
"  padding: 10px;\n"
"  border-radius: 5px;\n"
"  min-width: 80px;\n"
"  min-height: 30px;\n"
"\n"
"  font: 14pt \"Segoe UI\";\n"
"}\n"
"\n"
"")
self.tab = QWidget()
self.tab.setObjectName(u"tab")
self.tab.setStyleSheet(u"")
self.verticalLayout = QVBoxLayout(self.tab)
self.verticalLayout.setObjectName(u"verticalLayout")
self.horizontalLayout_2 = QHBoxLayout()
self.horizontalLayout_2.setObjectName(u"horizontalLayout_2")
self.horizontalLayout_2.setContentsMargins(-1, 0, -1, -1)
self.formFrame = QFrame(self.tab)
self.formFrame.setObjectName(u"formFrame")
self.formFrame.setStyleSheet(u"*{\n"
"font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/* Styling the QLineEdit */\n"
"QLineEdit {\n"
"  background-color: #f4f4f4; /* Light grey background */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 2px solid #c0c0c0; /* Border color */\n"

```

```

"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 5px;\n"
"}\n"
"\n"
/* Focused state (when the QLineEdit is selected) */\n"
"QLineEdit:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
/* Disabled state */\n"
"QLineEdit:disabled {\n"
"    background-color: #e0e0e0; /* Grey background when disabled */\n"
"    color: #888888; /* Light grey text when disabled */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
/* Placeholder text style */\n"
"QLineEdit::placeholder {\n"
"    color: #888888; /* Light grey color for placeholder text */\n"
"    font-style: italic; /* Italicize the placeholder text */\n"
""
        "}\n"
"\n"
/* Styling the text cursor */\n"
"QLineEdit::cursor {\n"
"    color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
""
)

self.formLayout = QFormLayout(self.formFrame)
self.formLayout.setObjectName(u"formLayout")
self.formLayout.setContentsMargins(-1, 0, 0, 0)
self.label_2 = QLabel(self.formFrame)
self.label_2.setObjectName(u"label_2")

self.formLayout.addWidget(0, QFormLayout.LabelRole, self.label_2)

self.label = QLabel(self.formFrame)
self.label.setObjectName(u"label")

self.formLayout.addWidget(2, QFormLayout.LabelRole, self.label)

self.std_firstname_box = QLineEdit(self.formFrame)
self.std_firstname_box.setObjectName(u"std_firstname_box")

self.formLayout.addWidget(2, QFormLayout.FieldRole,
self.std_firstname_box)

self.label_3 = QLabel(self.formFrame)
self.label_3.setObjectName(u"label_3")

```

```

self.formLayout.addWidget(3, QFormLayout.LabelRole, self.label_3)

self.std_lastname_box = QLineEdit(self.formFrame)
self.std_lastname_box.setObjectName(u"std_lastname_box")

self.formLayout.addWidget(3, QFormLayout.FieldRole,
self.std_lastname_box)

self.std_fname_box = QLineEdit(self.formFrame)
self.std_fname_box.setObjectName(u"std_fname_box")

self.formLayout.addWidget(4, QFormLayout.FieldRole, self.std_fname_box)

self.label_5 = QLabel(self.formFrame)
self.label_5.setObjectName(u"label_5")

self.formLayout.addWidget(5, QFormLayout.LabelRole, self.label_5)

self.std_mname_box = QLineEdit(self.formFrame)
self.std_mname_box.setObjectName(u"std_mname_box")

self.formLayout.addWidget(5, QFormLayout.FieldRole, self.std_mname_box)

self.label_6 = QLabel(self.formFrame)
self.label_6.setObjectName(u"label_6")

self.formLayout.addWidget(6, QFormLayout.LabelRole, self.label_6)

self.label_7 = QLabel(self.formFrame)
self.label_7.setObjectName(u"label_7")

self.formLayout.addWidget(8, QFormLayout.LabelRole, self.label_7)

self.std_dob_box = QLineEdit(self.formFrame)
self.std_dob_box.setObjectName(u"std_dob_box")

self.formLayout.addWidget(8, QFormLayout.FieldRole, self.std_dob_box)

self.label_8 = QLabel(self.formFrame)
self.label_8.setObjectName(u"label_8")

self.formLayout.addWidget(9, QFormLayout.LabelRole, self.label_8)

self.std_age_box = QLineEdit(self.formFrame)
self.std_age_box.setObjectName(u"std_age_box")

self.formLayout.addWidget(9, QFormLayout.FieldRole, self.std_age_box)

self.label_4 = QLabel(self.formFrame)

```



```

self.label_4.setObjectName(u"label_4")

self.formLayout.addWidget(4, QFormLayout.LabelRole, self.label_4)

self.std_gender_box = QComboBox(self.formFrame)
self.std_gender_box.addItem("")
self.std_gender_box.addItem("")
self.std_gender_box.setObjectName(u"std_gender_box")
self.std_gender_box.setStyleSheet(u"/* Styling the QComboBox */\n"
"QComboBox {\n"
"  background-color: #f4f4f4; /* Light grey background */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 2px solid #c0c0c0; /* Border color */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 5px;\n"
"  font: 14pt \"Segoe UI\";\n"
"}\n"
"  min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
"/* Styling the combo box when focused */\n"
"QComboBox:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
"  background-color: #e0e0e0; /* Grey background when disabled */\n"
"  color: #888888; /* Light grey text when disabled */\n"
"  border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Arrow button of the combo box */\n"
"\n"
"\n"
"/* Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
"  background-color\n"
"    : #ffffff; /* White background for the dropdown list */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
"  border-radius: 5px;\n"
"  padding: 5px;\n"
"  min-width: 150px;\n"
"}\n"
"\n"
"/* Hover effect on the dropdown list items */\n"
"QComboBox QAbstractItemView::item:hover {\n"
"  background-color: #2196F3; /* Blue background on hover */\n"

```



```

" color: #ffffff; /* White text on hover */\n"
"}\n"
"\n"
"/\n"
"QComboBox QAbstractItemView::item:selected {\n"
" background-color: #64B5F6; /* Light blue background when selected */\n"
" color: #ffffff; /* White text when selected */\n"
"}\n"
""")

self.formLayout.addWidget(6, QFormLayout.FieldRole, self.std_gender_box)

self.std_id_box = QLineEdit(self.formFrame)
self.std_id_box.setObjectName(u"std_id_box")

self.formLayout.addWidget(0, QFormLayout.FieldRole, self.std_id_box)

self.horizontalLayout_2.addWidget(self.formFrame)

self.formFrame_2 = QFrame(self.tab)
self.formFrame_2.setObjectName(u"formFrame_2")
self.formFrame_2.setStyleSheet(u"*\n"
"font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/\n"
"QLineEdit {\n"
" background-color: #f4f4f4; /* Light grey background */\n"
" color: #333333; /* Dark text color */\n"
" border: 2px solid #c0c0c0; /* Border color */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 5px;\n"
"}\n"
"\n"
"/\n"
"QLineEdit:focus {\n"
" border: 2px solid #2196F3; /* Blue border when focused */\n"
" background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/\n"
"QLineEdit:disabled {\n"
" background-color: #e0e0e0; /* Grey background when disabled */\n"
" color: #888888; /* Light grey text when disabled */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/\n"
"Placeholder text style */\n"

```

```

"QLineEdit::placeholder {\n"
"    color: #888888; /* Light grey color for placeholder text */\n"
"    font-style: italic; /* Italicize the placeholder text */\n"
""
        "}\n"
"\n"
"/* Styling the text cursor */\n"
"QLineEdit::cursor {\n"
"    color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
""
)

self.formLayout_2 = QFormLayout(self.formFrame_2)
self.formLayout_2.setObjectName(u"formLayout_2")
self.formLayout_2.setContentsMargins(-1, 0, 0, 0)
self.label_11 = QLabel(self.formFrame_2)
self.label_11.setObjectName(u"label_11")

self.formLayout_2.addWidget(0, QFormLayout.LabelRole, self.label_11)

self.label_12 = QLabel(self.formFrame_2)
self.label_12.setObjectName(u"label_12")

self.formLayout_2.addWidget(2, QFormLayout.LabelRole, self.label_12)

self.std_contact_box = QLineEdit(self.formFrame_2)
self.std_contact_box.setObjectName(u"std_contact_box")

self.formLayout_2.addWidget(2, QFormLayout.FieldRole,
self.std_contact_box)

self.label_17 = QLabel(self.formFrame_2)
self.label_17.setObjectName(u"label_17")

self.formLayout_2.addWidget(3, QFormLayout.LabelRole, self.label_17)

self.std_email_box = QLineEdit(self.formFrame_2)
self.std_email_box.setObjectName(u"std_email_box")

self.formLayout_2.addWidget(3, QFormLayout.FieldRole, self.std_email_box)

self.label_13 = QLabel(self.formFrame_2)
self.label_13.setObjectName(u"label_13")

self.formLayout_2.addWidget(4, QFormLayout.LabelRole, self.label_13)

self.label_14 = QLabel(self.formFrame_2)
self.label_14.setObjectName(u"label_14")

self.formLayout_2.addWidget(7, QFormLayout.LabelRole, self.label_14)

```

```

        self.std_address_box = QPlainTextEdit(self.formFrame_2)
        self.std_address_box.setObjectName(u"std_address_box")
        self.std_address_box.setMaximumSize(QSize(16777215, 80))
        self.std_address_box.setStyleSheet(u"/* Styling the QPlainTextEdit */\n"
"QPlainTextEdit {\n"
"  background-color: #f4f4f4; /* Light grey background */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 2px solid #c0c0c0; /* Border color */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 8px;\n"
"  font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/* Focused state (when the QPlainTextEdit is selected) */\n"
"QPlainTextEdit:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state */\n"
"QPlainTextEdit:disabled {\n"
"  background-color: #e0e0e0; /* Grey background when disabled */\n"
"  color: #888888; /* Light grey text when disabled */\n"
"  border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Styling the text cursor */\n"
"QPlainTextEdit::cursor {\n"
"  color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
"\n"
"/* Styling the selected text */\n"
"QPlainTextEdit::"
        "selection {\n"
"  background-color: #64B5F6; /* Light blue background for selected text */\n"
"  color: #ffffff; /* White text color for selected text */\n"
"}\n"
"\n"
"/* Styling the scroll bar */\n"
"QPlainTextEdit QScrollBar:vertical {\n"
"  border: none;\n"
"  background: #f4f4f4;\n"
"  width: 12px;\n"
"  margin: 0px, 0px, 0px, 0px;\n"
"  border-radius: 6px;\n"
"}\n"
"\n"
"QPlainTextEdit QScrollBar::handle:vertical {\n"
"  background: #2196F3;\n"

```

```

"    border-radius: 6px;\n"
"}\n"
"\n"
"QPlainTextEdit QScrollBar::add-line:vertical, \n"
"QPlainTextEdit QScrollBar::sub-line:vertical {\n"
"    border: none;\n"
"    background: none;\n"
"}\n"
""")

    self.formLayout_2.setWidget(0, QFormLayout.FieldRole,
self.std_address_box)

    self.std_class_box = QComboBox(self.formFrame_2)
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.setObjectName(u"std_class_box")
    self.std_class_box.setStyleSheet(u"/* Styling the QComboBox */\n"
"QComboBox {\n"
"    background-color: #f4f4f4; /* Light grey background */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 2px solid #c0c0c0; /* Border color */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 5px;\n"
"    font: 14pt \"Segoe UI\";\n"
"\n"
"    min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
"/* Styling the combo box when focused */\n"
"QComboBox:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
"    background-color: #e0e0e0; /* Grey background when disabled */\n"
"    color: #888888; /* Light grey text when disabled */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"

```



```

"}\n"
"\n"
"/\n" Styling the combo box when focused */\n"
"QComboBox:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/\n" Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
"    background-color: #e0e0e0; /* Grey background when disabled */\n"
"    color: #888888; /* Light grey text when disabled */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/\n" Arrow button of the combo box */\n"
"\n"
"\n"
"/\n" Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
"    background-color\n"
"        ": #ffffff; /* White background for the dropdown list */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
"    border-radius: 5px;\n"
"    padding: 5px;\n"
"    min-width: 150px;\n"
"}\n"
"\n"
"/\n" Hover effect on the dropdown list items */\n"
"QComboBox QAbstractItemView::item:hover {\n"
"    background-color: #2196F3; /* Blue background on hover */\n"
"    color: #ffffff; /* White text on hover */\n"
"}\n"
"\n"
"/\n" Selected item in the dropdown list */\n"
"QComboBox QAbstractItemView::item:selected {\n"
"    background-color: #64B5F6; /* Light blue background when selected */\n"
"    color: #ffffff; /* White text when selected */\n"
"}\n"
""
)

```

```

        self.formLayout_2.addWidget(7, QFormLayout.FieldRole,
self.std_section_box)

```

```

        self.horizontalLayout_2.addWidget(self.formFrame_2, 0, Qt.AlignTop)

```

```

        self.verticalLayout.addLayout(self.horizontalLayout_2)

```

```

self.horizontalLayout_3 = QHBoxLayout()
self.horizontalLayout_3.setObjectName(u"horizontalLayout_3")
self.horizontalLayout_3.setContentsMargins(-1, 0, -1, -1)
self.horizontalSpacer = QSpacerItem(40, 20, QSizePolicy.Expanding,
QSizePolicy.Minimum)

self.horizontalLayout_3.addItem(self.horizontalSpacer)

self.fetch_btn = QPushButton(self.tab)
self.fetch_btn.setObjectName(u"fetch_btn")
self.fetch_btn.setStyleSheet(u"/ * Styling the QPushButton */\n"
"QPushButton {\n"
"  background-color: #4CAF50; /* Green background */\n"
"  color: white; /* White text color */\n"
"  border: 2px solid #4CAF50; /* Border color matches background */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 10px 20px; /* Vertical and horizontal padding */\n"
"  \n"
"  font: 14pt \"Segoe UI\"; /* Font size */\n"
"  min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
"/ * Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
"  background-color: #45a049; /* Darker green background */\n"
"  border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
"/ * Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
"  background-color: #388e3c; /* Even darker green */\n"
"  border-color: #388e3c; /* Dark border */\n"
"  padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
"  padding-left: 18px; /* Shift the button to the left slightly */\n"
"}\n"
"\n"
"/ * Focused state (when the button is focused) */\n"
"QPushButton:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  outline: none; /* Removes the default outline */\n"
"}\n"
"\n"
"/ * Disabled state for the QPushButton */\n"
"QPushButton:disabled {\n"
"  background-color: #e0e0e0; /* Light grey background */\n"
"  color: #888888; /* Light grey text */\n"
"  border: 2px solid #cccccc; /* Light grey border */\n"

```

```

"}\n"
"\n"
"/ * Styling the text of the QPushButton */\n"
"QPushButton {\n"
"    font-weight: bold; /* Make the text bold */\n"
"}\n"
"\n"
"/ * Styling the QPushButton icon (if it has one) */\n"
"QPushButton::icon {\n"
"    padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"
"\n"
"/ * Border effect when the button is in a focus or pressed state */\n"
"QPushButton:focus, QPushButton:pressed {\n"
"    border-style: inset;\n"
"}\n"
""")

self.horizontalLayout_3.addWidget(self.fetch_btn)

self.save_btn = QPushButton(self.tab)
self.save_btn.setObjectName(u"save_btn")
self.save_btn.setStyleSheet(u"/ * Styling the QPushButton */\n"
"QPushButton {\n"
"    background-color: #4CAF50; /* Green background */\n"
"    color: white; /* White text color */\n"
"    border: 2px solid #4CAF50; /* Border color matches background */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 10px 20px; /* Vertical and horizontal padding */\n"
"    \n"
"    font: 14pt \"Segoe UI\"; /* Font size */\n"
"    min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
"/ * Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
"    background-color: #45a049; /* Darker green background */\n"
"    border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
"/ * Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
"    background-color: #388e3c; /* Even darker green */\n"
"    border-color: #388e3c; /* Dark border */\n"
"    padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
"    padding-left: 18px; /* Shift the button to the left slightly */\n"
"}\n"
"\n"

```



```

"/ * Focused state (when the button is focused) */\n"
"QPushButton:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    outline: none; /* Removes the default outline */\n"
"}\n"
"\n"
"/ * Disabled state for the QPushButton */\n"
"QPushButton:disabled {\n"
"    background-color: #e0e0e0; /* Light grey background */\n"
"    color: #888888; /* Light grey text */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/ * Styling the text of the QPushButton */\n"
"QPushButton {\n"
"    font-weight: bold; /* Make the text bold */\n"
"}\n"
"\n"
"/ * Styling the QPushButton icon (if it has one) */\n"
"QPushButton::icon {\n"
"    padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"
"\n"
"/ * Border effect when the button is in a focus or pressed state */\n"
"QPushButton:focus, QPushButton:pressed {\n"
"    border-style: inset;\n"
"}\n"
""
)

self.horizontalLayout_3.addWidget(self.save_btn)

self.verticalLayout.addLayout(self.horizontalLayout_3)

self.verticalSpacer = QSpacerItem(20, 40, QSizePolicy.Minimum,
QSizePolicy.Expanding)

self.verticalLayout.addItem(self.verticalSpacer)

self.tabWidget.addTab(self.tab, "")

self.horizontalLayout.addWidget(self.tabWidget)

MainWindow.setCentralWidget(self.centralwidget)

self.retranslateUi(MainWindow)

self.tabWidget.setCurrentIndex(0)

```

```

QMetaObject.connectSlotsByName(MainWindow)
# setupUi

def retranslateUi(self, MainWindow):
    MainWindow.setWindowTitle(QCoreApplication.translate("MainWindow",
u"Edit Student Details", None))
    self.label_2.setText(QCoreApplication.translate("MainWindow", u"Student
ID", None))
    self.label.setText(QCoreApplication.translate("MainWindow", u"First
Name", None))
    self.label_3.setText(QCoreApplication.translate("MainWindow", u"Last
Name", None))
    self.label_5.setText(QCoreApplication.translate("MainWindow",
u"Mother's Name", None))
    self.label_6.setText(QCoreApplication.translate("MainWindow",
u"Gender", None))
    self.label_7.setText(QCoreApplication.translate("MainWindow", u"DOB",
None))
    self.label_8.setText(QCoreApplication.translate("MainWindow", u"Age",
None))
    self.label_4.setText(QCoreApplication.translate("MainWindow", u"Father's
Name", None))
    self.std_gender_box.setItemText(0,
QCoreApplication.translate("MainWindow", u"Male", None))
    self.std_gender_box.setItemText(1,
QCoreApplication.translate("MainWindow", u"Female", None))

    self.label_11.setText(QCoreApplication.translate("MainWindow",
u"Address", None))
    self.label_12.setText(QCoreApplication.translate("MainWindow",
u"Contact No.", None))
    self.label_17.setText(QCoreApplication.translate("MainWindow",
u"Email", None))
    self.label_13.setText(QCoreApplication.translate("MainWindow",
u"Standard", None))
    self.label_14.setText(QCoreApplication.translate("MainWindow",
u"Division", None))
    self.std_class_box.setItemText(0,
QCoreApplication.translate("MainWindow", u"Class 1", None))
    self.std_class_box.setItemText(1,
QCoreApplication.translate("MainWindow", u"Class 2", None))
    self.std_class_box.setItemText(2,
QCoreApplication.translate("MainWindow", u"Class 3", None))
    self.std_class_box.setItemText(3,
QCoreApplication.translate("MainWindow", u"Class 4", None))
    self.std_class_box.setItemText(4,
QCoreApplication.translate("MainWindow", u"Class 5", None))
    self.std_class_box.setItemText(5,
QCoreApplication.translate("MainWindow", u"Class 6", None))

```

```
        self.std_class_box.setItemText(6,
QtCoreApplication.translate("MainWindow", u"Class 7", None))
        self.std_class_box.setItemText(7,
QtCoreApplication.translate("MainWindow", u"Class 8", None))
        self.std_class_box.setItemText(8,
QtCoreApplication.translate("MainWindow", u"Class 9", None))
        self.std_class_box.setItemText(9,
QtCoreApplication.translate("MainWindow", u"Class 10", None))
        self.std_class_box.setItemText(10,
QtCoreApplication.translate("MainWindow", u"Class 11", None))
        self.std_class_box.setItemText(11,
QtCoreApplication.translate("MainWindow", u"Class 12", None))
```

```
        self.std_section_box.setItemText(0,
QtCoreApplication.translate("MainWindow", u"A", None))
        self.std_section_box.setItemText(1,
QtCoreApplication.translate("MainWindow", u"B", None))
        self.std_section_box.setItemText(2,
QtCoreApplication.translate("MainWindow", u"C", None))
        self.std_section_box.setItemText(3,
QtCoreApplication.translate("MainWindow", u"D", None))
        self.std_section_box.setItemText(4,
QtCoreApplication.translate("MainWindow", u"E", None))
        self.std_section_box.setItemText(5,
QtCoreApplication.translate("MainWindow", u"F", None))
        self.std_section_box.setItemText(6,
QtCoreApplication.translate("MainWindow", u"G", None))
```

```
        self.fetch_btn.setText(QtCoreApplication.translate("MainWindow", u"Fetch
Data", None))
        self.save_btn.setText(QtCoreApplication.translate("MainWindow", u"Save
Details", None))
        self.tabWidget.setTabText(self.tabWidget.indexOf(self.tab),
QtCoreApplication.translate("MainWindow", u"Edit Student Details", None))
        # retranslateUi
```

13.12 ui_login.py

```
# -*- coding: utf-8 -*-

#####
#####
## Form generated from reading UI file 'loginJVydE.ui'
##
## Created by: Qt User Interface Compiler version 6.4.0
##
## WARNING! All changes made in this file will be lost when recompiling UI file!
#####
#####

from PySide6.QtCore import (QCoreApplication, QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt)
from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
    QFont, QFontDatabase, QGradient, QIcon,
    QImage, QKeySequence, QLinearGradient, QPainter,
    QPalette, QPixmap, QRadialGradient, QTransform)
from PySide6.QtWidgets import (QApplication, QFormLayout, QFrame,
    QGroupBox,
    QHBoxLayout, QLabel, QLineEdit, QMainWindow,
    QPushButton, QSizePolicy, QSpacerItem, QVBoxLayout,
    QWidget)

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.setObjectName():
            MainWindow.setObjectName(u"MainWindow")
        MainWindow.resize(484, 399)
        MainWindow.setMaximumSize(QSize(484, 399))
        MainWindow.setStyleSheet(u"background-color: rgb(255, 255, 255);")
        self.centralwidget = QWidget(MainWindow)
        self.centralwidget.setObjectName(u"centralwidget")
        self.verticalLayout = QVBoxLayout(self.centralwidget)
        self.verticalLayout.setObjectName(u"verticalLayout")
        self.frame_2 = QFrame(self.centralwidget)
        self.frame_2.setObjectName(u"frame_2")
        self.frame_2 setFrameShape(QFrame.StyledPanel)
        self.frame_2 setFrameShadow(QFrame.Raised)
        self.horizontalLayout_2 = QHBoxLayout(self.frame_2)
        self.horizontalLayout_2.setObjectName(u"horizontalLayout_2")
        self.horizontalSpacer_2 = QSpacerItem(40, 20, QSizePolicy.Expanding,
            QSizePolicy.Minimum)

        self.horizontalLayout_2.addItem(self.horizontalSpacer_2)

        self.label = QLabel(self.frame_2)
        self.label.setObjectName(u"label")
```

```

font = QFont()
font.setPointSize(16)
font.setItalic(True)
self.label.setFont(font)

self.horizontalLayout_2.addWidget(self.label)

self.label_6 = QLabel(self.frame_2)
self.label_6.setObjectName(u"label_6")
font1 = QFont()
font1.setFamilies([u"Segoe Print"])
font1.setPointSize(16)
font1.setBold(True)
self.label_6.setFont(font1)

self.horizontalLayout_2.addWidget(self.label_6)

self.label_7 = QLabel(self.frame_2)
self.label_7.setObjectName(u"label_7")

self.horizontalLayout_2.addWidget(self.label_7)

self.horizontalSpacer = QSpacerItem(40, 20, QSizePolicy.Expanding,
QSizePolicy.Minimum)

self.horizontalLayout_2.addItem(self.horizontalSpacer)

self.verticalLayout.addWidget(self.frame_2)

self.groupBox = QGroupBox(self.centralwidget)
self.groupBox.setObjectName(u"groupBox")
font2 = QFont()
font2.setFamilies([u"Arial"])
font2.setPointSize(17)
font2.setBold(True)
font2.setItalic(False)
self.groupBox.setFont(font2)
self.groupBox.setStyleSheet(u"QGroupBox {\n"
" border: 2px solid #4CAF50; /* Green border */\n"
" border-radius: 5px; /* Rounded corners */\n"
" margin-top: 10px; /* Space between the title and the border */\n"
" background-color: #FFFFFF; /* Light gray background */\n"
" font: bold 17pt 'Arial'; /* Font style for the title */\n"
" color: #333333; /* Dark gray title text color */\n"
"}\n"
"\n"
"QGroupBox::title {\n"
" subcontrol-origin: margin; /* Place the title outside the border */\n"
" subcontrol-position: top center; /* Center the title at the top */\n"

```

```

" padding: 0 5px; /* Padding around the title text */\n"
" background-color: #FFFFFF; /* White background behind the title */\n"
" color: #4CAF50; /* Green title text color */\n"
" font: bold 20pt \"Arial\"; /* Font style for the title */\n"
"}\n"
""))

self.verticalLayout_2 = QVBoxLayout(self.groupBox)
self.verticalLayout_2.setSpacing(0)
self.verticalLayout_2.setObjectName(u"verticalLayout_2")
self.verticalLayout_2.setContentsMargins(1, 40, 1, 1)
self.frame = QFrame(self.groupBox)
self.frame.setObjectName(u"frame")
self.frame.setStyleSheet(u"background-color: rgb(255, 255, 255);")
self.formLayout = QFormLayout(self.frame)
self.formLayout.setObjectName(u"formLayout")
self.formLayout.setHorizontalSpacing(20)
self.formLayout.setVerticalSpacing(20)
self.formLayout.setContentsMargins(30, 30, 30, 30)
self.password_box = QLineEdit(self.frame)
self.password_box.setObjectName(u"password_box")
self.password_box.setMinimumSize(QSize(200, 0))
font3 = QFont()
font3.setFamilies([u"Segoe UI"])
font3.setPointSize(12)
font3.setBold(False)
font3.setItalic(False)
self.password_box.setFont(font3)
self.password_box.setStyleSheet(u"QLineEdit{\n"
" height:30px;\n"
" font: 12pt \"Segoe UI\";\n"
" \n"
" background-color: rgb(255, 255, 255);\n"
"}")
self.password_box.setEchoMode(QLineEdit.Password)

self.formLayout.addWidget(1, QFormLayout.FieldRole, self.password_box)

self.username_box = QLineEdit(self.frame)
self.username_box.setObjectName(u"username_box")
self.username_box.setMinimumSize(QSize(200, 0))
self.username_box.setFont(font3)
self.username_box.setStyleSheet(u"QLineEdit{\n"
" height:30px;\n"
" font: 12pt \"Segoe UI\";\n"
" \n"
" background-color: rgb(255, 255, 255);\n"
"}")

self.formLayout.addWidget(0, QFormLayout.FieldRole, self.username_box)

```

```

self.label_3 = QLabel(self.frame)
self.label_3.setObjectName(u"label_3")
font4 = QFont()
font4.setFamilies([u"Segoe UI"])
font4.setPointSize(14)
font4.setBold(False)
font4.setItalic(True)
self.label_3.setFont(font4)
self.label_3.setStyleSheet(u"background-color: rgb(255, 255, 255);")

self.formLayout.addWidget(0, QFormLayout.LabelRole, self.label_3)

self.label_4 = QLabel(self.frame)
self.label_4.setObjectName(u"label_4")
self.label_4.setFont(font4)
self.label_4.setStyleSheet(u"background-color: rgb(255, 255, 255);")

self.formLayout.addWidget(1, QFormLayout.LabelRole, self.label_4)

self.login_btn = QPushButton(self.frame)
self.login_btn.setObjectName(u"login_btn")
font5 = QFont()
font5.setFamilies([u"Segoe UI"])
font5.setPointSize(14)
font5.setBold(True)
font5.setItalic(False)
self.login_btn.setFont(font5)
self.login_btn.setCursor(QCursor(Qt.PointingHandCursor))
self.login_btn.setStyleSheet(u"QPushButton{\n"
"    color: rgb(255, 255, 255);\n"
"    background-color: rgb(76, 175, 80);\n"
"    border-radius:5px;\n"
"    border: 1px solid black;\n"
"    height:30px;\n"
"    font: 700 14pt \"Segoe UI\";\n"
"}")

self.formLayout.addWidget(2, QFormLayout.SpanningRole, self.login_btn)

self.verticalLayout_2.addWidget(self.frame)

self.verticalLayout.addWidget(self.groupBox, 0,
Qt.AlignHCenter|Qt.AlignTop)

self.label_2 = QLabel(self.centralwidget)
self.label_2.setObjectName(u"label_2")

self.verticalLayout.addWidget(self.label_2, 0, Qt.AlignHCenter)

```



```

        self.verticalSpacer = QSpacerItem(20, 40, QSizePolicy.Minimum,
        QSizePolicy.Expanding)

        self.verticalLayout.addItem(self.verticalSpacer)

        MainWindow.setCentralWidget(self.centralwidget)
        QWidget.setTabOrder(self.username_box, self.password_box)
        QWidget.setTabOrder(self.password_box, self.login_btn)

        self.retranslateUi(MainWindow)

        QMetaObject.connectSlotsByName(MainWindow)
        # setupUi

        def retranslateUi(self, MainWindow):
            MainWindow.setWindowTitle(QCoreApplication.translate("MainWindow",
            u"Login Window", None))
            self.label.setText(QCoreApplication.translate("MainWindow", u"School
            Management", None))
            self.label_6.setText(QCoreApplication.translate("MainWindow",
            u"Software", None))
            self.label_7.setText(QCoreApplication.translate("MainWindow", u"v1.0",
            None))
            self.groupBox.setTitle(QCoreApplication.translate("MainWindow", u"Login
            Credentials", None))

        self.password_box.setPlaceholderText(QCoreApplication.translate("MainWindo
        w", u"pass$word", None))

        self.username_box.setPlaceholderText(QCoreApplication.translate("MainWindo
        w", u"user@name", None))
            self.label_3.setText(QCoreApplication.translate("MainWindow",
            u"Username: ", None))
            self.label_4.setText(QCoreApplication.translate("MainWindow",
            u"Password:", None))
            self.login_btn.setText(QCoreApplication.translate("MainWindow",
            u"Login", None))
            self.label_2.setText(QCoreApplication.translate("MainWindow", u"-",
            None)) # Developer Name
        # retranslateUi

```


13.13 ui_main.py

```
# -*- coding: utf-8 -*-

#####
#####
## Form generated from reading UI file 'mainbphGdB.ui'
##
## Created by: Qt User Interface Compiler version 6.4.0
##
## WARNING! All changes made in this file will be lost when recompiling UI file!
#####
#####

from PySide6.QtCore import (QCoreApplication, QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt)
from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
    QFont, QFontDatabase, QGradient, QIcon,
    QImage, QKeySequence, QLinearGradient, QPainter,
    QPalette, QPixmap, QRadialGradient, QTransform)
from PySide6.QtWidgets import (QApplication, QGridLayout, QHBoxLayout,
    QLabel,
    QMainWindow, QPushButton, QSizePolicy, QSpacerItem,
    QStackedWidget, QVBoxLayout, QWidget)
import icons_res_rc

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.setObjectName():
            MainWindow.setObjectName(u"MainWindow")
        MainWindow.resize(1264, 600)
        self.centralwidget = QWidget(MainWindow)
        self.centralwidget.setObjectName(u"centralwidget")
        self.centralwidget.setStyleSheet(u"")
        self.verticalLayout = QVBoxLayout(self.centralwidget)
        self.verticalLayout.setObjectName(u"verticalLayout")
        self.horizontalLayout = QHBoxLayout()
        self.horizontalLayout.setObjectName(u"horizontalLayout")
        self.horizontalLayout.setContentsMargins(-1, 0, -1, -1)
        self.horizontalLayout_2 = QHBoxLayout()
        self.horizontalLayout_2.setObjectName(u"horizontalLayout_2")
        self.label = QLabel(self.centralwidget)
        self.label.setObjectName(u"label")
        font = QFont()
        font.setPointSize(14)
        font.setItalic(True)
        self.label.setFont(font)

        self.horizontalLayout_2.addWidget(self.label, 0, Qt.AlignLeft)
```

```

self.label_6 = QLabel(self.centralwidget)
self.label_6.setObjectName(u"label_6")
font1 = QFont()
font1.setFamilies([u"Segoe Print"])
font1.setPointSize(14)
font1.setBold(True)
self.label_6.setFont(font1)

self.horizontalLayout_2.addWidget(self.label_6, 0, Qt.AlignLeft)

self.label_7 = QLabel(self.centralwidget)
self.label_7.setObjectName(u"label_7")

self.horizontalLayout_2.addWidget(self.label_7)

self.horizontalSpacer = QSpacerItem(40, 20, QSizePolicy.Expanding,
QSizePolicy.Minimum)

self.horizontalLayout_2.addItem(self.horizontalSpacer)

self.horizontalLayout.addLayout(self.horizontalLayout_2)

self.date_label = QLabel(self.centralwidget)
self.date_label.setObjectName(u"date_label")
font2 = QFont()
font2.setPointSize(12)
font2.setBold(True)
self.date_label.setFont(font2)

self.horizontalLayout.addWidget(self.date_label, 0, Qt.AlignRight)

self.verticalLayout.addLayout(self.horizontalLayout)

self.stackedWidget = QStackedWidget(self.centralwidget)
self.stackedWidget.setObjectName(u"stackedWidget")
self.stackedWidget.setStyleSheet(u"QStackedWidget {\n"
"    background-image: url(:/assets/classroom.png);\n"
"    background-position: center;\n"
"\n"
"}\n"
"")
self.main_pag = QWidget()
self.main_pag.setObjectName(u"main_pag")
self.verticalLayout_4 = QVBoxLayout(self.main_pag)
self.verticalLayout_4.setSpacing(14)
self.verticalLayout_4.setObjectName(u"verticalLayout_4")
self.label_5 = QLabel(self.main_pag)
self.label_5.setObjectName(u"label_5")

```

```

font3 = QFont()
font3.setFamilies([u"Segoe Print"])
font3.setPointSize(18)
font3.setBold(True)
self.label_5.setFont(font3)

self.verticalLayout_4.addWidget(self.label_5, 0,
Qt.AlignHCenter|Qt.AlignTop)

self.gridLayout = QGridLayout()
self.gridLayout.setSpacing(20)
self.gridLayout.setObjectName(u"gridLayout")
self.edit_staff_btn = QPushButton(self.main_pag)
self.edit_staff_btn.setObjectName(u"edit_staff_btn")
self.edit_staff_btn.setStyleSheet(u"QPushButton{\n"
"    background-color: rgb(255, 0, 127);\n"
"    height:100px;\n"
"    width: 100px;\n"
"    font: 700 15pt \"Segoe UI\";border-radius:5px;\n"
"    border: 1px solid black;\n"
"}")
icon = QIcon()
icon.addFile(u":/assets/staff.png", QSize(), QIcon.Normal, QIcon.Off)
self.edit_staff_btn.setIcon(icon)
self.edit_staff_btn.setIconSize(QSize(60, 60))

self.gridLayout.addWidget(self.edit_staff_btn, 1, 4, 1, 1)

self.staff_attend_btn = QPushButton(self.main_pag)
self.staff_attend_btn.setObjectName(u"staff_attend_btn")
self.staff_attend_btn.setStyleSheet(u"QPushButton{\n"
"    background-color: rgb(255, 85, 0);\n"
"    height:100px;\n"
"    width: 100px;\n"
"    font: 700 15pt \"Segoe UI\";border-radius:5px;\n"
"    border: 1px solid black;\n"
"}")
icon1 = QIcon()
icon1.addFile(u":/assets/staff-attend.png", QSize(), QIcon.Normal,
QIcon.Off)
self.staff_attend_btn.setIcon(icon1)
self.staff_attend_btn.setIconSize(QSize(60, 60))

self.gridLayout.addWidget(self.staff_attend_btn, 3, 2, 1, 1)

self.edit_btn = QPushButton(self.main_pag)
self.edit_btn.setObjectName(u"edit_btn")
self.edit_btn.setStyleSheet(u"QPushButton{\n"
"    background-color: rgb(0, 170, 255);\n"
"    height:100px;\n"

```

```

"    width: 100px;\n"
"    font: 700 15pt \"Segoe UI\";border-radius:5px;\n"
"    border: 1px solid black;\n"
"}")
    icon2 = QIcon()
    icon2.addFile(u":/assets/edit_stud.png", QSize(), QIcon.Normal, QIcon.Off)
    self.edit_btn.setIcon(icon2)
    self.edit_btn.setIconSize(QSize(60, 60))

    self.gridLayout.addWidget(self.edit_btn, 1, 2, 1, 1)

    self.teache_btn = QPushButton(self.main_pag)
    self.teache_btn.setObjectName(u"teache_btn")
    self.teache_btn.setStyleSheet(u"QPushButton{\n"
"    background-color: rgb(255, 170, 0);\n"
"    height:100px;\n"
"    width: 100px;\n"
"    font: 700 15pt \"Segoe UI\";border-radius:5px;\n"
"    border: 1px solid black;\n"
"}")
    icon3 = QIcon()
    icon3.addFile(u":/assets/teaching.png", QSize(), QIcon.Normal, QIcon.Off)
    self.teache_btn.setIcon(icon3)
    self.teache_btn.setIconSize(QSize(60, 60))

    self.gridLayout.addWidget(self.teache_btn, 1, 3, 1, 1)

    self.student_attend_btn = QPushButton(self.main_pag)
    self.student_attend_btn.setObjectName(u"student_attend_btn")
    self.student_attend_btn.setStyleSheet(u"QPushButton{\n"
"    background-color: rgb(170, 170, 255);\n"
"    height:100px;\n"
"    width: 100px;\n"
"    font: 700 15pt \"Segoe UI\";border-radius:5px;\n"
"    border: 1px solid black;\n"
"}")
    icon4 = QIcon()
    icon4.addFile(u":/assets/attendance.png", QSize(), QIcon.Normal,
QIcon.Off)
    self.student_attend_btn.setIcon(icon4)
    self.student_attend_btn.setIconSize(QSize(60, 60))

    self.gridLayout.addWidget(self.student_attend_btn, 3, 1, 1, 1)

    self.marksheet_btn = QPushButton(self.main_pag)
    self.marksheet_btn.setObjectName(u"marksheet_btn")
    self.marksheet_btn.setStyleSheet(u"QPushButton{\n"
"    background-color: rgb(255, 255, 0);\n"
"    height:100px;\n"
"    width: 100px;\n"

```

```

"    font: 700 15pt \"Segoe UI\";border-radius:5px;\n"
"    border: 1px solid black;\n"
"}")
    icon5 = QIcon()
    icon5.addFile(u":/assets/result.png", QSize(), QIcon.Normal, QIcon.Off)
    self.marksheet_btn.setIcon(icon5)
    self.marksheet_btn.setIconSize(QSize(60, 60))

    self.gridLayout.addWidget(self.marksheet_btn, 3, 4, 1, 1)

    self.admission_btn = QPushButton(self.main_pag)
    self.admission_btn.setObjectName(u"admission_btn")
    self.admission_btn.setStyleSheet(u"QPushButton{\n"
"    height:100px;\n"
"    width: 100px;\n"
"    font: 700 15pt \"Segoe UI\";\n"
"    border-radius:5px;\n"
"    border: 1px solid black;\n"
"    background-color: rgb(85, 255, 0);\n"
"}")
    icon6 = QIcon()
    icon6.addFile(u":/assets/admission_icon.png", QSize(), QIcon.Normal,
QIcon.Off)
    self.admission_btn.setIcon(icon6)
    self.admission_btn.setIconSize(QSize(60, 60))

    self.gridLayout.addWidget(self.admission_btn, 1, 1, 1, 1)

    self.marks_btn = QPushButton(self.main_pag)
    self.marks_btn.setObjectName(u"marks_btn")
    self.marks_btn.setStyleSheet(u"QPushButton{\n"
"    background-color: rgb(255, 170, 255);\n"
"    height:100px;\n"
"    width: 100px;\n"
"    font: 700 15pt \"Segoe UI\";border-radius:5px;\n"
"    border: 1px solid black;\n"
"}")
    icon7 = QIcon()
    icon7.addFile(u":/assets/marks.png", QSize(), QIcon.Normal, QIcon.Off)
    self.marks_btn.setIcon(icon7)
    self.marks_btn.setIconSize(QSize(60, 60))

    self.gridLayout.addWidget(self.marks_btn, 3, 3, 1, 1)

    self.std_rec_btn = QPushButton(self.main_pag)
    self.std_rec_btn.setObjectName(u"std_rec_btn")
    self.std_rec_btn.setStyleSheet(u"QPushButton{\n"
"    background-color: rgb(255, 170, 127);\n"
"    height:100px;\n"
"    width: 100px;\n"

```

```

"    font: 700 15pt \"Segoe UI\";border-radius:5px;\n"
"    border: 1px solid black;\n"
"}")
    icon8 = QIcon()
    icon8.addFile(u":/assets/std_record.png", QSize(), QIcon.Normal, QIcon.Off)
    self.std_rec_btn.setIcon(icon8)
    self.std_rec_btn.setIconSize(QSize(60, 60))

    self.gridLayout.addWidget(self.std_rec_btn, 4, 1, 1, 1)

    self.staff_rec_btn = QPushButton(self.main_pag)
    self.staff_rec_btn.setObjectName(u"staff_rec_btn")
    self.staff_rec_btn.setStyleSheet(u"QPushButton{\n"
"    background-color: rgb(85, 85, 127);\n"
"    height:100px;\n"
"    width: 100px;\n"
"    font: 700 15pt \"Segoe UI\";border-radius:5px;\n"
"    border: 1px solid black;\n"
"}")
    icon9 = QIcon()
    icon9.addFile(u":/assets/staff_record.png", QSize(), QIcon.Normal,
QIcon.Off)
    self.staff_rec_btn.setIcon(icon9)
    self.staff_rec_btn.setIconSize(QSize(60, 60))

    self.gridLayout.addWidget(self.staff_rec_btn, 4, 2, 1, 1)

    self.fees_btn = QPushButton(self.main_pag)
    self.fees_btn.setObjectName(u"fees_btn")
    self.fees_btn.setStyleSheet(u"QPushButton{\n"
"    background-color: rgb(255, 0, 0);\n"
"    height:100px;\n"
"    width: 100px;\n"
"    font: 700 15pt \"Segoe UI\";border-radius:5px;\n"
"    border: 1px solid black;\n"
"}")
    icon10 = QIcon()
    icon10.addFile(u":/assets/fees.png", QSize(), QIcon.Normal, QIcon.Off)
    self.fees_btn.setIcon(icon10)
    self.fees_btn.setIconSize(QSize(60, 60))

    self.gridLayout.addWidget(self.fees_btn, 4, 3, 1, 1)

    self.verticalLayout_4.addLayout(self.gridLayout)

    self.verticalSpacer = QSpacerItem(20, 40, QSizePolicy.Minimum,
QSizePolicy.Expanding)

    self.verticalLayout_4.addItem(self.verticalSpacer)

```

```

self.horizontalLayout_3 = QHBoxLayout()
self.horizontalLayout_3.setObjectName(u"horizontalLayout_3")
self.horizontalLayout_3.setContentsMargins(-1, 20, -1, -1)
self.logout_btn = QPushButton(self.main_pag)
self.logout_btn.setObjectName(u"logout_btn")
self.logout_btn.setStyleSheet(u"QPushButton{\n"
"    background-color: rgb(255, 255, 255);\n"
"    border: 1px solid black;\n"
"    border-radius:5px;\n"
"    height:40px;\n"
"    font: 700 14pt \"Segoe UI\";\n"
"}")
icon11 = QIcon()
icon11.addFile(u":/assets/logout.png", QSize(), QIcon.Normal, QIcon.Off)
self.logout_btn.setIcon(icon11)
self.logout_btn.setIconSize(QSize(30, 30))

self.horizontalLayout_3.addWidget(self.logout_btn)

self.minimize_btn = QPushButton(self.main_pag)
self.minimize_btn.setObjectName(u"minimize_btn")
self.minimize_btn.setStyleSheet(u"QPushButton{\n"
"    background-color: rgb(255, 255, 255);\n"
"    border: 1px solid black;\n"
"    border-radius:5px;\n"
"    height:40px;\n"
"    font: 700 14pt \"Segoe UI\";\n"
"}")
icon12 = QIcon()
icon12.addFile(u":/assets/minimize.png", QSize(), QIcon.Normal, QIcon.Off)
self.minimize_btn.setIcon(icon12)
self.minimize_btn.setIconSize(QSize(30, 30))

self.horizontalLayout_3.addWidget(self.minimize_btn)

self.exit_btn = QPushButton(self.main_pag)
self.exit_btn.setObjectName(u"exit_btn")
self.exit_btn.setStyleSheet(u"QPushButton{\n"
"    background-color: rgb(255, 255, 255);\n"
"    border: 1px solid black;\n"
"    border-radius:5px;\n"
"    height:40px;\n"
"    font: 700 14pt \"Segoe UI\";\n"
"}")
icon13 = QIcon()
icon13.addFile(u":/assets/close.png", QSize(), QIcon.Normal, QIcon.Off)
self.exit_btn.setIcon(icon13)
self.exit_btn.setIconSize(QSize(30, 30))

```



```

self.horizontalLayout_3.addWidget(self.exit_btn)

self.verticalLayout_4.addLayout(self.horizontalLayout_3)

self.stackedWidget.addWidget(self.main_pag)

self.verticalLayout.addWidget(self.stackedWidget)

MainWindow.setCentralWidget(self.centralwidget)
QWidget.setTabOrder(self.admission_btn, self.edit_btn)
QWidget.setTabOrder(self.edit_btn, self.teache_btn)
QWidget.setTabOrder(self.teache_btn, self.student_attend_btn)
QWidget.setTabOrder(self.student_attend_btn, self.staff_attend_btn)
QWidget.setTabOrder(self.staff_attend_btn, self.marksheet_btn)
QWidget.setTabOrder(self.marksheet_btn, self.logout_btn)
QWidget.setTabOrder(self.logout_btn, self.minimize_btn)
QWidget.setTabOrder(self.minimize_btn, self.exit_btn)

self.retranslateUi(MainWindow)

self.stackedWidget.setCurrentIndex(0)

QMetaObject.connectSlotsByName(MainWindow)
# setupUi

def retranslateUi(self, MainWindow):
    MainWindow.setWindowTitle(QCoreApplication.translate("MainWindow",
u"School Management Software", None))
    self.label.setText(QCoreApplication.translate("MainWindow", u"School
Management", None))
    self.label_6.setText(QCoreApplication.translate("MainWindow",
u"Software", None))
    self.label_7.setText(QCoreApplication.translate("MainWindow", u"v1.0",
None))
    self.date_label.setText(QCoreApplication.translate("MainWindow", u"Date:
12/12/2024 11:00:02 PM", None))
    self.label_5.setText(QCoreApplication.translate("MainWindow", u"Admin
Dashboard", None))
    self.edit_staff_btn.setText(QCoreApplication.translate("MainWindow",
u"Edit Staff Details", None))
    self.staff_attend_btn.setText(QCoreApplication.translate("MainWindow",
u"Staff Attendance", None))
    self.edit_btn.setText(QCoreApplication.translate("MainWindow", u"Edit
Student Details", None))
    self.teache_btn.setText(QCoreApplication.translate("MainWindow", u"
Staff Details", None))

```



```
self.student_attend_btn.setText(QCoreApplication.translate("MainWindow",  
u"Student Attendance", None))  
    self.marksheet_btn.setText(QCoreApplication.translate("MainWindow",  
u"Marksheet", None))  
    self.admission_btn.setText(QCoreApplication.translate("MainWindow",  
u"Student Admission", None))  
    self.marks_btn.setText(QCoreApplication.translate("MainWindow",  
u"Marks", None))  
    self.std_rec_btn.setText(QCoreApplication.translate("MainWindow",  
u"Student Record", None))  
    self.staff_rec_btn.setText(QCoreApplication.translate("MainWindow",  
u"Staff Record", None))  
    self.fees_btn.setText(QCoreApplication.translate("MainWindow", u"Fees  
Module", None))  
    self.logout_btn.setText(QCoreApplication.translate("MainWindow",  
u"Logout", None))  
    self.minimize_btn.setText(QCoreApplication.translate("MainWindow",  
u"Minimize", None))  
    self.exit_btn.setText(QCoreApplication.translate("MainWindow", u"Exit",  
None))  
    # retranslateUi
```

13.14 ui_marks.py

```
# -*- coding: utf-8 -*-

#####
#####
## Form generated from reading UI file 'markstmdkKK.ui'
##
## Created by: Qt User Interface Compiler version 6.4.0
##
## WARNING! All changes made in this file will be lost when recompiling UI file!
#####
#####

from PySide6.QtCore import (QCoreApplication, QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt)
from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
    QFont, QFontDatabase, QGradient, QIcon,
    QImage, QKeySequence, QLinearGradient, QPainter,
    QPalette, QPixmap, QRadialGradient, QTransform)
from PySide6.QtWidgets import (QApplication, QFormLayout, QFrame,
    QHBoxLayout,
    QHeaderView, QLabel, QLineEdit, QMainWindow,
    QPushButton, QSizePolicy, QSpacerItem, QTabWidget,
    QTableWidgetItem, QTableWidgetItem, QVBoxLayout, QWidget)

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.setObjectName():
            MainWindow.setObjectName(u"MainWindow")
        MainWindow.resize(800, 600)
        self.centralwidget = QWidget(MainWindow)
        self.centralwidget.setObjectName(u"centralwidget")
        self.horizontalLayout_5 = QHBoxLayout(self.centralwidget)
        self.horizontalLayout_5.setObjectName(u"horizontalLayout_5")
        self.tabWidget = QTabWidget(self.centralwidget)
        self.tabWidget.setObjectName(u"tabWidget")
        self.tabWidget.setStyleSheet(u"/* Styling the Tab Widget */\n"
"QTabWidget {\n"
"    background-color: #f0f0f0;\n"
"    border: 2px solid black;\n"
"    border-radius: 10px;\n"
"    padding: 5px;\n"
"}\n"
"\n"
"\n"
"/* Styling the Tabs */\n"
"QTabBar::tab {\n"
"    background-color: #2196F3; /* Tab background color */\n"
"    color: white;\n"
"
```

```

" border: 1px solid #c0c0c0;\n"
" padding: 10px;\n"
" border-radius: 5px;\n"
" min-width: 80px;\n"
" min-height: 30px;\n"
"\n"
" font: 14pt \"Segoe UI\";\n"
"}\n"
"\n"
""))
    self.tab = QWidget()
    self.tab.setObjectName(u"tab")
    self.tab.setStyleSheet(u"")
    self.verticalLayout = QVBoxLayout(self.tab)
    self.verticalLayout.setObjectName(u"verticalLayout")
    self.horizontalLayout_2 = QHBoxLayout()
    self.horizontalLayout_2.setObjectName(u"horizontalLayout_2")
    self.horizontalLayout_2.setContentsMargins(-1, 0, -1, -1)
    self.formFrame = QFrame(self.tab)
    self.formFrame.setObjectName(u"formFrame")
    self.formFrame.setStyleSheet(u"*{\n"
"font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/\n"
"QLineEdit {\n"
" background-color: #f4f4f4; /\n"
" color: #333333; /\n"
" border: 2px solid #c0c0c0; /\n"
" border-radius: 5px; /\n"
" padding: 5px;\n"
"}\n"
"\n"
"/\n"
"QLineEdit:focus {\n"
" border: 2px solid #2196F3; /\n"
" background-color: #ffffff; /\n"
"}\n"
"\n"
"/\n"
"QLineEdit:disabled {\n"
" background-color: #e0e0e0; /\n"
" color: #888888; /\n"
" border: 2px solid #cccccc; /\n"
"}\n"
"\n"
"/\n"
"QLineEdit::placeholder {\n"
" color: #888888; /\n"

```

```

" font-style: italic; /* Italicize the placeholder text */\n"
""

        "}\n"
"\n"
"/* Styling the text cursor */\n"
"QLineEdit::cursor {\n"
"    color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
""
)

    self.formLayout = QFormLayout(self.formFrame)
    self.formLayout.setObjectName(u"formLayout")
    self.formLayout.setContentsMargins(-1, 0, 0, 0)
    self.label_14 = QLabel(self.formFrame)
    self.label_14.setObjectName(u"label_14")

    self.formLayout.addWidget(0, QFormLayout.LabelRole, self.label_14)

    self.std_id_box = QLineEdit(self.formFrame)
    self.std_id_box.setObjectName(u"std_id_box")

    self.formLayout.addWidget(0, QFormLayout.FieldRole, self.std_id_box)

    self.label = QLabel(self.formFrame)
    self.label.setObjectName(u"label")

    self.formLayout.addWidget(1, QFormLayout.LabelRole, self.label)

    self.horizontalLayout = QHBoxLayout()
    self.horizontalLayout.setObjectName(u"horizontalLayout")
    self.sub_name = QLineEdit(self.formFrame)
    self.sub_name.setObjectName(u"sub_name")

    self.horizontalLayout.addWidget(self.sub_name)

    self.sub_m1_box = QLineEdit(self.formFrame)
    self.sub_m1_box.setObjectName(u"sub_m1_box")

    self.horizontalLayout.addWidget(self.sub_m1_box)

    self.sub_m2_box = QLineEdit(self.formFrame)
    self.sub_m2_box.setObjectName(u"sub_m2_box")

    self.horizontalLayout.addWidget(self.sub_m2_box)

    self.sub_btn = QPushButton(self.formFrame)
    self.sub_btn.setObjectName(u"sub_btn")
    self.sub_btn.setStyleSheet(u"/* Styling the QPushButton */\n"
"QPushButton {\n"
"    background-color: #4CAF50; /* Green background */\n"
"    color: white; /* White text color */\n"

```

```

" border: 2px solid #4CAF50; /* Border color matches background */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 10px 20px; /* Vertical and horizontal padding */\n"
" \n"
" font: 14pt \"Segoe UI\"; /* Font size */\n"
" min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
/* Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
" background-color: #45a049; /* Darker green background */\n"
" border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
/* Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
" background-color: #388e3c; /* Even darker green */\n"
" border-color: #388e3c; /* Dark border */\n"
" padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
" padding-left: 18px; /* Shift the button to the left slightly */\n"
"}\n"
"\n"
/* Focused state (when the button is focused) */\n"
"QPushButton:focus {\n"
" border: 2px solid #2196F3; /* Blue border when focused */\n"
" outline: none; /* Removes the default outline */\n"
"}\n"
"\n"
/* Disabled state for the QPushButton */\n"
"QPushButton:disabled {\n"
" background-color: #e0e0e0; /* Light grey background */\n"
" color: #888888; /* Light grey text */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
/* Styling the text of the QPushButton */\n"
"QPushButton {\n"
" font-weight: bold; /* Make the text bold */\n"
"}\n"
"\n"
/* Styling the QPushButton icon (if it has one) */\n"
"QPushButton::icon {\n"
" padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"
"\n"
/* Border effect when the button is in a focus or pressed state */\n"
"QPushButton:focus, QPushButton:pressed {\n"
" border-style: inset;\n"

```

```
"}\n"
""")
```

```
self.horizontalLayout.addWidget(self.sub_btn)
```

```
self.formLayout.setLayout(1, QFormLayout.FieldRole,
self.horizontalLayout)
```

```
self.horizontalLayout_2.addWidget(self.formFrame, 0, Qt.AlignTop)
```

```
self.verticalLayout.addLayout(self.horizontalLayout_2)
```

```
self.horizontalLayout_4 = QHBoxLayout()
self.horizontalLayout_4.setObjectName(u"horizontalLayout_4")
self.horizontalLayout_4.setContentsMargins(-1, 0, -1, -1)
self.sub_table = QTableWidget(self.tab)
if (self.sub_table.columnCount() < 4):
    self.sub_table.setColumnCount(4)
    __qtablewidgetitem = QTableWidgetItem()
    self.sub_table.setHorizontalHeaderItem(0, __qtablewidgetitem)
    __qtablewidgetitem1 = QTableWidgetItem()
    self.sub_table.setHorizontalHeaderItem(1, __qtablewidgetitem1)
    __qtablewidgetitem2 = QTableWidgetItem()
    self.sub_table.setHorizontalHeaderItem(2, __qtablewidgetitem2)
    __qtablewidgetitem3 = QTableWidgetItem()
    self.sub_table.setHorizontalHeaderItem(3, __qtablewidgetitem3)
if (self.sub_table.rowCount() < 3):
    self.sub_table.setRowCount(3)
self.sub_table.setObjectName(u"sub_table")
font = QFont()
font.setPointSize(14)
font.setBold(False)
self.sub_table.setFont(font)
self.sub_table.setStyleSheet(u"QHeaderView::section {\n"
"background-color: rgb(0, 120, 212);\n"
" /* Green background for headers */\n"
" color: white; /* White text color */\n"
" font-weight: bold; /* Bold font for header text */\n"
" text-align: center; /* Center-align text */\n"
" \n"
" font: 700 14pt \"Segoe UI\";\n"
" border: 1px solid #ccc; /* Light gray border around header sections */\n"
" padding: 5px; /* Padding inside header cells */\n"
"}\n"
""")
```

```
self.horizontalLayout_4.addWidget(self.sub_table)
```

```

self.verticalLayout.addLayout(self.horizontalLayout_4)

self.horizontalLayout_3 = QHBoxLayout()
self.horizontalLayout_3.setObjectName(u"horizontalLayout_3")
self.horizontalLayout_3.setContentsMargins(-1, 0, -1, -1)
self.horizontalSpacer = QSpacerItem(40, 20, QSizePolicy.Expanding,
QSizePolicy.Minimum)

self.horizontalLayout_3.addItem(self.horizontalSpacer)

self.fetch_btn = QPushButton(self.tab)
self.fetch_btn.setObjectName(u"fetch_btn")
self.fetch_btn.setStyleSheet(u"/* Styling the QPushButton */\n"
"QPushButton {\n"
"  background-color: #4CAF50; /* Green background */\n"
"  color: white; /* White text color */\n"
"  border: 2px solid #4CAF50; /* Border color matches background */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 10px 20px; /* Vertical and horizontal padding */\n"
"  \n"
"  font: 14pt \"Segoe UI\"; /* Font size */\n"
"  min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
"/* Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
"  background-color: #45a049; /* Darker green background */\n"
"  border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
"/* Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
"  background-color: #388e3c; /* Even darker green */\n"
"  border-color: #388e3c; /* Dark border */\n"
"  padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
"  padding-left: 18px; /* Shift the button to the left slightly */\n"
"}\n"
"\n"
"/* Focused state (when the button is focused) */\n"
"QPushButton:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  outline: none; /* Removes the default outline */\n"
"}\n"
"\n"
"/* Disabled state for the QPushButton */\n"
"QPushButton:disabled {\n"

```

```

" background-color: #e0e0e0; /* Light grey background */\n"
" color: #888888; /* Light grey text */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
/* Styling the text of the QPushButton */\n"
"QPushButton {\n"
" font-weight: bold; /* Make the text bold */\n"
"}\n"
"\n"
/* Styling the QPushButton icon (if it has one) */\n"
"QPushButton::icon {\n"
" padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"
"\n"
/* Border effect when the button is in a focus or pressed state */\n"
"QPushButton:focus, QPushButton:pressed {\n"
" border-style: inset;\n"
"}\n"
""
)

self.horizontalLayout_3.addWidget(self.fetch_btn)

self.save_btn = QPushButton(self.tab)
self.save_btn.setObjectName(u"save_btn")
self.save_btn.setStyleSheet(u"/* Styling the QPushButton */\n"
"QPushButton {\n"
" background-color: #4CAF50; /* Green background */\n"
" color: white; /* White text color */\n"
" border: 2px solid #4CAF50; /* Border color matches background */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 10px 20px; /* Vertical and horizontal padding */\n"
" \n"
" font: 14pt \"Segoe UI\"; /* Font size */\n"
" min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
/* Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
" background-color: #45a049; /* Darker green background */\n"
" border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
/* Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
" background-color: #388e3c; /* Even darker green */\n"
" border-color: #388e3c; /* Dark border */\n"
" padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
" padding-left: 18px; /* Shift the button to t"

```



```

        "he left slightly */\n"
    }\n"
    \n"
    /* Focused state (when the button is focused) */\n"
    QPushButton:focus {\n"
    "    border: 2px solid #2196F3; /* Blue border when focused */\n"
    "    outline: none; /* Removes the default outline */\n"
    }\n"
    \n"
    /* Disabled state for the QPushButton */\n"
    QPushButton:disabled {\n"
    "    background-color: #e0e0e0; /* Light grey background */\n"
    "    color: #888888; /* Light grey text */\n"
    "    border: 2px solid #cccccc; /* Light grey border */\n"
    }\n"
    \n"
    /* Styling the text of the QPushButton */\n"
    QPushButton {\n"
    "    font-weight: bold; /* Make the text bold */\n"
    }\n"
    \n"
    /* Styling the QPushButton icon (if it has one) */\n"
    QPushButton::icon {\n"
    "    padding-right: 10px; /* Space between the icon and the text */\n"
    }\n"
    \n"
    /* Border effect when the button is in a focus or pressed state */\n"
    QPushButton:focus, QPushButton:pressed {\n"
    "    border-style: inset;\n"
    }\n"
    """)

    self.horizontalLayout_3.addWidget(self.save_btn)

    self.verticalLayout.addLayout(self.horizontalLayout_3)

    self.verticalSpacer = QSpacerItem(20, 40, QSizePolicy.Minimum,
    QSizePolicy.Expanding)

    self.verticalLayout.addItem(self.verticalSpacer)

    self.tabWidget.addTab(self.tab, "")

    self.horizontalLayout_5.addWidget(self.tabWidget)

    MainWindow.setCentralWidget(self.centralwidget)

    self.retranslateUi(MainWindow)

```

```

self.tabWidget.setCurrentIndex(0)

QMetaObject.connectSlotsByName(MainWindow)
# setupUi

def retranslateUi(self, MainWindow):
    MainWindow.setWindowTitle(QCoreApplication.translate("MainWindow",
u"MainWindow", None))
    self.label_14.setText(QCoreApplication.translate("MainWindow", u"Student
ID", None))
    self.label.setText(QCoreApplication.translate("MainWindow", u"Subject
Details", None))

self.sub_name.setPlaceholderText(QCoreApplication.translate("MainWindow",
u"Sub name", None))

self.sub_m1_box.setPlaceholderText(QCoreApplication.translate("MainWindow"
, u"Theory Marks", None))

self.sub_m2_box.setPlaceholderText(QCoreApplication.translate("MainWindow"
, u"Pract./Assign. Marks", None))
    self.sub_btn.setText(QCoreApplication.translate("MainWindow",
u"Publish", None))
    __qtablewidgetitem = self.sub_table.horizontalHeaderItem(0)
    __qtablewidgetitem.setText(QCoreApplication.translate("MainWindow",
u"Subject Name", None));
    __qtablewidgetitem1 = self.sub_table.horizontalHeaderItem(1)
    __qtablewidgetitem1.setText(QCoreApplication.translate("MainWindow",
u"Theory", None));
    __qtablewidgetitem2 = self.sub_table.horizontalHeaderItem(2)
    __qtablewidgetitem2.setText(QCoreApplication.translate("MainWindow",
u"Practical / Assignment", None));
    __qtablewidgetitem3 = self.sub_table.horizontalHeaderItem(3)
    __qtablewidgetitem3.setText(QCoreApplication.translate("MainWindow",
u"Total", None));
    self.fetch_btn.setText(QCoreApplication.translate("MainWindow", u"Fetch
Details", None))
    self.save_btn.setText(QCoreApplication.translate("MainWindow", u"Save
&& Exit", None))
    self.tabWidget.setTabText(self.tabWidget.indexOf(self.tab),
QCoreApplication.translate("MainWindow", u"Marks Pages", None))
# retranslateUi

```

13.15 ui_marksheet.py

```
# -*- coding: utf-8 -*-

#####
#####
## Form generated from reading UI file 'marksheetAaeLtZ.ui'
##
## Created by: Qt User Interface Compiler version 6.4.0
##
## WARNING! All changes made in this file will be lost when recompiling UI file!
#####
#####

from PySide6.QtCore import (QCoreApplication, QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt)
from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
    QFont, QFontDatabase, QGradient, QIcon,
    QImage, QKeySequence, QLinearGradient, QPainter,
    QPalette, QPixmap, QRadialGradient, QTransform)
from PySide6.QtWidgets import (QApplication, QFormLayout, QFrame,
    QHBoxLayout,
    QHeaderView, QLabel, QLineEdit, QListWidget,
    QListWidgetItem, QMainWindow, QPushButton, QSizePolicy,
    QSpacerItem, QTabWidget, QTableWidget, QTableWidgetItem,
    QVBoxLayout, QWidget)

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.setObjectName():
            MainWindow.setObjectName(u"MainWindow")
        MainWindow.resize(800, 600)
        self.centralwidget = QWidget(MainWindow)
        self.centralwidget.setObjectName(u"centralwidget")
        self.verticalLayout_2 = QVBoxLayout(self.centralwidget)
        self.verticalLayout_2.setObjectName(u"verticalLayout_2")
        self.tabWidget = QTabWidget(self.centralwidget)
        self.tabWidget.setObjectName(u"tabWidget")
        self.tabWidget.setStyleSheet(u"/* Styling the Tab Widget */\n"
"QTabWidget {\n"
"    background-color: #f0f0f0;\n"
"    border: 2px solid black;\n"
"    border-radius: 10px;\n"
"    padding: 5px;\n"
"}\n"
"\n"
"\n"
"/* Styling the Tabs */\n"
"QTabBar::tab {\n"
"    background-color: #2196F3; /* Tab background color */\n"
"
```

```

" color: white;\n"
" border: 1px solid #c0c0c0;\n"
" padding: 10px;\n"
" border-radius: 5px;\n"
" min-width: 80px;\n"
" min-height: 30px;\n"
"\n"
" font: 14pt \"Segoe UI\";\n"
"}\n"
"\n"
"")
    self.tab = QWidget()
    self.tab.setObjectName(u"tab")
    self.tab.setStyleSheet(u"")
    self.verticalLayout = QVBoxLayout(self.tab)
    self.verticalLayout.setObjectName(u"verticalLayout")
    self.horizontalLayout_2 = QHBoxLayout()
    self.horizontalLayout_2.setObjectName(u"horizontalLayout_2")
    self.horizontalLayout_2.setContentsMargins(-1, 0, -1, -1)
    self.formFrame = QFrame(self.tab)
    self.formFrame.setObjectName(u"formFrame")
    self.formFrame.setStyleSheet(u"*{\n"
"font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/* Styling the QLineEdit */\n"
"QLineEdit {\n"
" background-color: #f4f4f4; /* Light grey background */\n"
" color: #333333; /* Dark text color */\n"
" border: 2px solid #c0c0c0; /* Border color */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 5px;\n"
"}\n"
"\n"
"/* Focused state (when the QLineEdit is selected) */\n"
"QLineEdit:focus {\n"
" border: 2px solid #2196F3; /* Blue border when focused */\n"
" background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state */\n"
"QLineEdit:disabled {\n"
" background-color: #e0e0e0; /* Grey background when disabled */\n"
" color: #888888; /* Light grey text when disabled */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Placeholder text style */\n"
"QLineEdit::placeholder {\n"

```

```

" color: #888888; /* Light grey color for placeholder text */\n"
" font-style: italic; /* Italicize the placeholder text */\n"
""

        "}\n"
"\n"
"/\n" Styling the text cursor */\n"
"QLineEdit::cursor {\n"
" color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
""
    self.formLayout = QFormLayout(self.formFrame)
    self.formLayout.setObjectName(u"formLayout")
    self.formLayout.setContentsMargins(-1, 0, 0, 0)
    self.label_2 = QLabel(self.formFrame)
    self.label_2.setObjectName(u"label_2")

    self.formLayout.addWidget(0, QFormLayout.LabelRole, self.label_2)

    self.std_id_box = QLineEdit(self.formFrame)
    self.std_id_box.setObjectName(u"std_id_box")

    self.formLayout.addWidget(0, QFormLayout.FieldRole, self.std_id_box)

    self.horizontalLayout_2.addWidget(self.formFrame)

    self.verticalLayout.addLayout(self.horizontalLayout_2)

    self.horizontalLayout_3 = QHBoxLayout()
    self.horizontalLayout_3.setObjectName(u"horizontalLayout_3")
    self.horizontalLayout_3.setContentsMargins(-1, 0, -1, -1)
    self.horizontalSpacer = QSpacerItem(40, 20, QSizePolicy.Expanding,
QSizePolicy.Minimum)

    self.horizontalLayout_3.addItem(self.horizontalSpacer)

    self.fetch_btn = QPushButton(self.tab)
    self.fetch_btn.setObjectName(u"fetch_btn")
    self.fetch_btn.setStyleSheet(u"/\n" Styling the QPushButton */\n"
"QPushButton {\n"
" background-color: #4CAF50; /* Green background */\n"
" color: white; /* White text color */\n"
" border: 2px solid #4CAF50; /* Border color matches background */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 10px 20px; /* Vertical and horizontal padding */\n"
" \n"
" font: 14pt \"Segoe UI\"; /* Font size */\n"
" min-width: 100px; /* Minimum width for the button */\n"
"}\n"

```

```

"\n"
"/ * Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
"    background-color: #45a049; /* Darker green background */\n"
"    border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
"/ * Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
"    background-color: #388e3c; /* Even darker green */\n"
"    border-color: #388e3c; /* Dark border */\n"
"    padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
"    padding-left: 18px; /* Shift the button to the left slightly */\n"
"}\n"
"\n"
"/ * Focused state (when the button is focused) */\n"
"QPushButton:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    outline: none; /* Removes the default outline */\n"
"}\n"
"\n"
"/ * Disabled state for the QPushButton */\n"
"QPushButton:disabled {\n"
"    background-color: #e0e0e0; /* Light grey background */\n"
"    color: #888888; /* Light grey text */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/ * Styling the text of the QPushButton */\n"
"QPushButton {\n"
"    font-weight: bold; /* Make the text bold */\n"
"}\n"
"\n"
"/ * Styling the QPushButton icon (if it has one) */\n"
"QPushButton::icon {\n"
"    padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"
"\n"
"/ * Border effect when the button is in a focus or pressed state */\n"
"QPushButton:focus, QPushButton:pressed {\n"
"    border-style: inset;\n"
"}\n"
""

```

```

self.horizontalLayout_3.addWidget(self.fetch_btn)

```

```

self.print_btn = QPushButton(self.tab)
self.print_btn.setObjectName(u"print_btn")

```

```

        self.print_btn.setStyleSheet(u"/* Styling the QPushButton */\n"
"QPushButton {\n"
"    background-color: #4CAF50; /* Green background */\n"
"    color: white; /* White text color */\n"
"    border: 2px solid #4CAF50; /* Border color matches background */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 10px 20px; /* Vertical and horizontal padding */\n"
"    \n"
"    font: 14pt \"Segoe UI\"; /* Font size */\n"
"    min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
"/* Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
"    background-color: #45a049; /* Darker green background */\n"
"    border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
"/* Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
"    background-color: #388e3c; /* Even darker green */\n"
"    border-color: #388e3c; /* Dark border */\n"
"    padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
"    padding-left: 18px; /* Shift the button to the left slightly */\n"
"}\n"
"\n"
"/* Focused state (when the button is focused) */\n"
"QPushButton:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    outline: none; /* Removes the default outline */\n"
"}\n"
"\n"
"/* Disabled state for the QPushButton */\n"
"QPushButton:disabled {\n"
"    background-color: #e0e0e0; /* Light grey background */\n"
"    color: #888888; /* Light grey text */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Styling the text of the QPushButton */\n"
"QPushButton {\n"
"    font-weight: bold; /* Make the text bold */\n"
"}\n"
"\n"
"/* Styling the QPushButton icon (if it has one) */\n"
"QPushButton:icon {\n"
"    padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"

```

```

"\n"
"/** Border effect when the button is in a focus or pressed state */\n"
"QPushButton:focus, QPushButton:pressed {\n"
"    border-style: inset;\n"
"}\n"
""")

self.horizontalLayout_3.addWidget(self.print_btn)

self.verticalLayout.addLayout(self.horizontalLayout_3)

self.std_details_box = QListWidget(self.tab)
QListWidgetItem(self.std_details_box)
QListWidgetItem(self.std_details_box)
QListWidgetItem(self.std_details_box)
QListWidgetItem(self.std_details_box)
self.std_details_box.setObjectName(u"std_details_box")
self.std_details_box.setStyleSheet(u"QListView {\n"
"    background-color: #f0f0f0; /* Light grey background */\n"
"    border: 1px solid #ccc; /* Thin grey border */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 5px; /* Space between items and border */\n"
"    \n"
"    font: 14pt \"Segoe UI\";\n"
"}\n"
"\n"
"QListView::item {\n"
"    background-color: #ffffff; /* White background for items */\n"
"    border: 1px solid transparent; /* Transparent border by default */\n"
"    padding: 8px; /* Spacing around the text in each item */\n"
"    color: #333; /* Dark grey text color */\n"
"}\n"
"\n"
"QListView::item:hover {\n"
"    background-color: #e6f7ff; /* Light blue background when hovering */\n"
"    border: 1px solid #66afe9; /* Blue border on hover */\n"
"}\n"
"\n"
"QListView::item:selected {\n"
"    background-color: #66afe9; /* Blue background for selected items */\n"
"    color: white; /* White text for selected items */\n"
"    border: 1px solid #0078d7; /* Darker blue border for selected items */\n"
"}\n"
"\n"
"QListView::item:selected:focus {\n"
"    outline: none; /* Remove focus outline from selected item */\n"
"}\n"
""")

```



```

self.verticalLayout.addWidget(self.std_details_box)

self.std_marks_table = QTableWidgetItem(self.tab)
if (self.std_marks_table.columnCount() < 4):
    self.std_marks_table.setColumnCount(4)
    __qtablewidgetitem = QTableWidgetItem()
    self.std_marks_table.setHorizontalHeaderItem(0, __qtablewidgetitem)
    __qtablewidgetitem1 = QTableWidgetItem()
    self.std_marks_table.setHorizontalHeaderItem(1, __qtablewidgetitem1)
    __qtablewidgetitem2 = QTableWidgetItem()
    self.std_marks_table.setHorizontalHeaderItem(2, __qtablewidgetitem2)
    __qtablewidgetitem3 = QTableWidgetItem()
    self.std_marks_table.setHorizontalHeaderItem(3, __qtablewidgetitem3)
if (self.std_marks_table.rowCount() < 3):
    self.std_marks_table.setRowCount(3)
self.std_marks_table.setObjectName(u"std_marks_table")
font = QFont()
font.setPointSize(14)
font.setBold(False)
self.std_marks_table.setFont(font)
self.std_marks_table.setStyleSheet(u"QHeaderView::section {\n"
"background-color: rgb(0, 120, 212);\n"
" /* Green background for headers */\n"
" color: white; /* White text color */\n"
" font-weight: bold; /* Bold font for header text */\n"
" text-align: center; /* Center-align text */\n"
" \n"
" font: 700 14pt \"Segoe UI\";\n"
" border: 1px solid #ccc; /* Light gray border around header sections */\n"
" padding: 5px; /* Padding inside header cells */\n"
"}\n"
"")

self.verticalLayout.addWidget(self.std_marks_table)

self.verticalSpacer = QSpacerItem(20, 40, QSizePolicy.Minimum,
QSizePolicy.Expanding)

self.verticalLayout.addItem(self.verticalSpacer)

self.tabWidget.addTab(self.tab, "")

self.verticalLayout_2.addWidget(self.tabWidget)

MainWindow.setCentralWidget(self.centralwidget)

self.retranslateUi(MainWindow)

self.tabWidget.setCurrentIndex(0)

```

```

    QMetaObject.connectSlotsByName(MainWindow)
# setupUi

def retranslateUi(self, MainWindow):
    MainWindow.setWindowTitle(QCoreApplication.translate("MainWindow",
u"Marksheet Generator", None))
    self.label_2.setText(QCoreApplication.translate("MainWindow", u"Student
ID", None))
    self.fetch_btn.setText(QCoreApplication.translate("MainWindow", u"Fetch
Data", None))
    self.print_btn.setText(QCoreApplication.translate("MainWindow", u"Print
Marksheet", None))

    __sortingEnabled = self.std_details_box.isSortingEnabled()
    self.std_details_box.setSortingEnabled(False)
    __qlistwidgetitem = self.std_details_box.item(0)
    __qlistwidgetitem.setText(QCoreApplication.translate("MainWindow",
u"Student Name: ", None));
    __qlistwidgetitem1 = self.std_details_box.item(1)
    __qlistwidgetitem1.setText(QCoreApplication.translate("MainWindow",
u"Student ID:", None));
    __qlistwidgetitem2 = self.std_details_box.item(2)
    __qlistwidgetitem2.setText(QCoreApplication.translate("MainWindow",
u"Class:", None));
    __qlistwidgetitem3 = self.std_details_box.item(3)
    __qlistwidgetitem3.setText(QCoreApplication.translate("MainWindow",
u"Section:", None));
    self.std_details_box.setSortingEnabled(__sortingEnabled)

    __qtablewidgetitem = self.std_marks_table.horizontalHeaderItem(0)
    __qtablewidgetitem.setText(QCoreApplication.translate("MainWindow",
u"Subject Name", None));
    __qtablewidgetitem1 = self.std_marks_table.horizontalHeaderItem(1)
    __qtablewidgetitem1.setText(QCoreApplication.translate("MainWindow",
u"Theory", None));
    __qtablewidgetitem2 = self.std_marks_table.horizontalHeaderItem(2)
    __qtablewidgetitem2.setText(QCoreApplication.translate("MainWindow",
u"Practical / Assignment", None));
    __qtablewidgetitem3 = self.std_marks_table.horizontalHeaderItem(3)
    __qtablewidgetitem3.setText(QCoreApplication.translate("MainWindow",
u"Total", None));
    self.tabWidget.setTabText(self.tabWidget.indexOf(self.tab),
QCoreApplication.translate("MainWindow", u"Marksheet Generator", None))
# retranslateUi

```

13.16 ui_staff_attend.py

```
# -*- coding: utf-8 -*-

#####
#####
## Form generated from reading UI file 'staff_attendYtkHut.ui'
##
## Created by: Qt User Interface Compiler version 6.4.0
##
## WARNING! All changes made in this file will be lost when recompiling UI file!
#####
#####

from PySide6.QtCore import (QCoreApplication, QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt)
from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
    QFont, QFontDatabase, QGradient, QIcon,
    QImage, QKeySequence, QLinearGradient, QPainter,
    QPalette, QPixmap, QRadialGradient, QTransform)
from PySide6.QtWidgets import (QApplication, QCalendarWidget, QComboBox,
    QFormLayout,
    QFrame, QHBoxLayout, QHeaderView, QLabel,
    QMainWindow, QPushButton, QSizePolicy, QSpacerItem,
    QTabWidget, QTableWidget, QTableWidgetItem, QVBoxLayout,
    QWidget)

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.setObjectName():
            MainWindow.setObjectName(u"MainWindow")
        MainWindow.resize(800, 600)
        self.centralwidget = QWidget(MainWindow)
        self.centralwidget.setObjectName(u"centralwidget")
        self.verticalLayout_2 = QVBoxLayout(self.centralwidget)
        self.verticalLayout_2.setObjectName(u"verticalLayout_2")
        self.tabWidget = QTabWidget(self.centralwidget)
        self.tabWidget.setObjectName(u"tabWidget")
        self.tabWidget.setStyleSheet(u"/* Styling the Tab Widget */\n"
"QTabWidget {\n"
"    background-color: #f0f0f0;\n"
"    border: 2px solid black;\n"
"    border-radius: 10px;\n"
"    padding: 5px;\n"
"}\n"
"\n"
"\n"
"/* Styling the Tabs */\n"
"QTabBar::tab {\n"
"    background-color: #2196F3; /* Tab background color */\n"
"
```

```

" color: white;\n"
" border: 1px solid #c0c0c0;\n"
" padding: 10px;\n"
" border-radius: 5px;\n"
" min-width: 80px;\n"
" min-height: 30px;\n"
"\n"
" font: 14pt \"Segoe UI\";\n"
"}\n"
"\n"
"")
    self.tab = QWidget()
    self.tab.setObjectName(u"tab")
    self.tab.setStyleSheet(u"")
    self.verticalLayout = QVBoxLayout(self.tab)
    self.verticalLayout.setObjectName(u"verticalLayout")
    self.horizontalLayout_2 = QHBoxLayout()
    self.horizontalLayout_2.setObjectName(u"horizontalLayout_2")
    self.horizontalLayout_2.setContentsMargins(-1, 0, -1, -1)
    self.formFrame = QFrame(self.tab)
    self.formFrame.setObjectName(u"formFrame")
    self.formFrame.setStyleSheet(u"*{\n"
"font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/* Styling the QLineEdit */\n"
"QLineEdit {\n"
" background-color: #f4f4f4; /* Light grey background */\n"
" color: #333333; /* Dark text color */\n"
" border: 2px solid #c0c0c0; /* Border color */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 5px;\n"
"}\n"
"\n"
"/* Focused state (when the QLineEdit is selected) */\n"
"QLineEdit:focus {\n"
" border: 2px solid #2196F3; /* Blue border when focused */\n"
" background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state */\n"
"QLineEdit:disabled {\n"
" background-color: #e0e0e0; /* Grey background when disabled */\n"
" color: #888888; /* Light grey text when disabled */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Placeholder text style */\n"
"QLineEdit::placeholder {\n"

```

```

"    color: #888888; /* Light grey color for placeholder text */\n"
"    font-style: italic; /* Italicize the placeholder text */\n"
""

        "}\n"
"\n"
"/\n" Styling the text cursor */\n"
"QLineEdit::cursor {\n"
"    color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
""
)

    self.formLayout = QFormLayout(self.formFrame)
    self.formLayout.setObjectName(u"formLayout")
    self.formLayout.setContentsMargins(-1, 0, 0, 0)
    self.label_13 = QLabel(self.formFrame)
    self.label_13.setObjectName(u"label_13")

    self.formLayout.addWidget(1, QFormLayout.LabelRole, self.label_13)

    self.staff_dept_box = QComboBox(self.formFrame)
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.setObjectName(u"staff_dept_box")
    self.staff_dept_box.setStyleSheet(u"/\n" Styling the QComboBox */\n"
"QComboBox {\n"
"    background-color: #f4f4f4; /* Light grey background */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 2px solid #c0c0c0; /* Border color */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 5px;\n"
"    font: 14pt \"Segoe UI\";\n"
"\n"
"    min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
"/\n" Styling the combo box when focused */\n"
"QComboBox:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/\n" Disabled state for the combo box */\n"
"QComboBox:disabled {\n"

```

```

" background-color: #e0e0e0; /* Grey background when disabled */\n"
" color: #888888; /* Light grey text when disabled */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Arrow button of the combo box */\n"
"\n"
"\n"
"/* Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
" background-color"
"         ": #ffffff; /* White background for the dropdown list */\n"
" color: #333333; /* Dark text color */\n"
" border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
" border-radius: 5px;\n"
" padding: 5px;\n"
" min-width: 150px;\n"
"}\n"
"\n"
"/* Hover effect on the dropdown list items */\n"
"QComboBox QAbstractItemView::item:hover {\n"
" background-color: #2196F3; /* Blue background on hover */\n"
" color: #ffffff; /* White text on hover */\n"
"}\n"
"\n"
"/* Selected item in the dropdown list */\n"
"QComboBox QAbstractItemView::item:selected {\n"
" background-color: #64B5F6; /* Light blue background when selected */\n"
" color: #ffffff; /* White text when selected */\n"
"}\n"
""")

```

```

self.formLayout.setWidget(1, QFormLayout.FieldRole, self.staff_dept_box)

```

```

self.horizontalLayout_2.addWidget(self.formFrame, 0, Qt.AlignTop)

```

```

self.verticalLayout.addLayout(self.horizontalLayout_2)

```

```

self.horizontalLayout_4 = QHBoxLayout()
self.horizontalLayout_4.setObjectName(u"horizontalLayout_4")
self.horizontalLayout_4.setContentsMargins(-1, 0, -1, -1)
self.staff_attend_table = QTableWidgetItem(self.tab)
if (self.staff_attend_table.columnCount() < 5):
    self.staff_attend_table.setColumnCount(5)
__qtablewidgetitem = QTableWidgetItem()
self.staff_attend_table.setHorizontalHeaderItem(0, __qtablewidgetitem)
__qtablewidgetitem1 = QTableWidgetItem()
self.staff_attend_table.setHorizontalHeaderItem(1, __qtablewidgetitem1)

```

```

__qtablewidgetitem2 = QTableWidgetItem()
self.staff_attend_table.setHorizontalHeaderItem(2, __qtablewidgetitem2)
__qtablewidgetitem3 = QTableWidgetItem()
self.staff_attend_table.setHorizontalHeaderItem(3, __qtablewidgetitem3)
__qtablewidgetitem4 = QTableWidgetItem()
self.staff_attend_table.setHorizontalHeaderItem(4, __qtablewidgetitem4)
if (self.staff_attend_table.rowCount() < 3):
    self.staff_attend_table.setRowCount(3)
self.staff_attend_table.setObjectName(u"staff_attend_table")
font = QFont()
font.setPointSize(14)
font.setBold(False)
self.staff_attend_table.setFont(font)
self.staff_attend_table.setStyleSheet(u"QHeaderView::section {\n"
"background-color: rgb(0, 120, 212);\n"
" /* Green background for headers */\n"
" color: white; /* White text color */\n"
" font-weight: bold; /* Bold font for header text */\n"
" text-align: center; /* Center-align text */\n"
" \n"
" font: 700 14pt \"Segoe UI\";\n"
" border: 1px solid #ccc; /* Light gray border around header sections */\n"
" padding: 5px; /* Padding inside header cells */\n"
"}\n"
"")

self.horizontalLayout_4.addWidget(self.staff_attend_table)

self.staff_calender_box = QCalendarWidget(self.tab)
self.staff_calender_box.setObjectName(u"staff_calender_box")

self.horizontalLayout_4.addWidget(self.staff_calender_box, 0,
Qt.AlignRight)

self.verticalLayout.addLayout(self.horizontalLayout_4)

self.horizontalLayout_3 = QHBoxLayout()
self.horizontalLayout_3.setObjectName(u"horizontalLayout_3")
self.horizontalLayout_3.setContentsMargins(-1, 0, -1, -1)
self.horizontalSpacer = QSpacerItem(40, 20, QSizePolicy.Expanding,
QSizePolicy.Minimum)

self.horizontalLayout_3.addItem(self.horizontalSpacer)

self.fetch_btn = QPushButton(self.tab)
self.fetch_btn.setObjectName(u"fetch_btn")
self.fetch_btn.setStyleSheet(u"/* Styling the QPushButton */\n"
"QPushButton {\n"
" /* Green background */\n"

```



```

"    background-color: rgb(0, 120, 212);\n"
"    color: white; /* White text color */\n"
"    border: 2px solid #4CAF50; /* Border color matches background */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 10px 20px; /* Vertical and horizontal padding */\n"
"    \n"
"    font: 14pt \"Segoe UI\"; /* Font size */\n"
"    min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
"/* Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
"    background-color: #45a049; /* Darker green background */\n"
"    border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
"/* Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
"    background-color: #388e3c; /* Even darker green */\n"
"    border-color: #388e3c; /* Dark border */\n"
"    padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
"    padding-left: 18px; /* Shift the"
"        button to the left slightly */\n"
"}\n"
"\n"
"/* Focused state (when the button is focused) */\n"
"QPushButton:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    outline: none; /* Removes the default outline */\n"
"}\n"
"\n"
"/* Disabled state for the QPushButton */\n"
"QPushButton:disabled {\n"
"    background-color: #e0e0e0; /* Light grey background */\n"
"    color: #888888; /* Light grey text */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Styling the text of the QPushButton */\n"
"QPushButton {\n"
"    font-weight: bold; /* Make the text bold */\n"
"}\n"
"\n"
"/* Styling the QPushButton icon (if it has one) */\n"
"QPushButton::icon {\n"
"    padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"
"\n"
"/* Border effect when the button is in a focus or pressed state */\n"

```



```

"QPushButton:focus, QPushButton:pressed {\n"
"  border-style: inset;\n"
"}\n"
""

    self.horizontalLayout_3.addWidget(self.fetch_btn)

    self.save_btn = QPushButton(self.tab)
    self.save_btn.setObjectName(u"save_btn")
    self.save_btn.setStyleSheet(u"/* Styling the QPushButton */\n"
"QPushButton {\n"
"background-color: rgb(0, 120, 212);\n"
"  /* Green background */\n"
"  color: white; /* White text color */\n"
"  border: 2px solid #4CAF50; /* Border color matches background */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 10px 20px; /* Vertical and horizontal padding */\n"
"  \n"
"  font: 14pt \"Segoe UI\"; /* Font size */\n"
"  min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
"/* Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
"  background-color: #45a049; /* Darker green background */\n"
"  border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
"/* Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
"  background-color: #388e3c; /* Even darker green */\n"
"  border-color: #388e3c; /* Dark border */\n"
"  padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
"  padding-left: 18px; /* Shift the button to the left slightly */\n"
"}\n"
"\n"
"/* Focused state (when the button is focused) */\n"
"QPushButton:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  outline: none; /* Removes the default outline */\n"
"}\n"
"\n"
"/* Disabled state for the QPushButton */\n"
"QPushButton:disabled {\n"
"  background-color: #e0e0e0; /* Light grey background */\n"
"  color: #888888; /* Light grey text */\n"
"  border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"

```

```

"\n"
"/ * Styling the text of the QPushButton */\n"
"QPushButton {\n"
"    font-weight: bold; /* Make the text bold */\n"
"}\n"
"\n"
"/ * Styling the QPushButton icon (if it has one) */\n"
"QPushButton::icon {\n"
"    padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"
"\n"
"/ * Border effect when the button is in a focus or pressed state */\n"
"QPushButton:focus, QPushButton:pressed {\n"
"    border-style: inset;\n"
"}\n"
""
)

```

```

        self.horizontalLayout_3.addWidget(self.save_btn)

        self.verticalLayout.addLayout(self.horizontalLayout_3)

        self.verticalSpacer = QSpacerItem(20, 40, QSizePolicy.Minimum,
        QSizePolicy.Expanding)

        self.verticalLayout.addItem(self.verticalSpacer)

        self.tabWidget.addTab(self.tab, "")

        self.verticalLayout_2.addWidget(self.tabWidget)

        MainWindow.setCentralWidget(self.centralwidget)

        self.retranslateUi(MainWindow)

        self.tabWidget.setCurrentIndex(0)


        QMetaObject.connectSlotsByName(MainWindow)
    # setupUi

    def retranslateUi(self, MainWindow):
        MainWindow.setWindowTitle(QCoreApplication.translate("MainWindow",
        u"Staff Attendance", None))
        self.label_13.setText(QCoreApplication.translate("MainWindow",
        u"Department", None))
        self.staff_dept_box.setItemText(0,
        QCoreApplication.translate("MainWindow", u"Primary - 1 to 5", None))
        self.staff_dept_box.setItemText(1,
        QCoreApplication.translate("MainWindow", u"Middle High - 6 to 8", None))

```

```

        self.staff_dept_box.setItemText(2,
QtCoreApplication.translate("MainWindow", u"Secondary - 9 to 10", None))
        self.staff_dept_box.setItemText(3,
QtCoreApplication.translate("MainWindow", u"Science", None))
        self.staff_dept_box.setItemText(4,
QtCoreApplication.translate("MainWindow", u"Commerce", None))
        self.staff_dept_box.setItemText(5,
QtCoreApplication.translate("MainWindow", u"Humanities", None))
        self.staff_dept_box.setItemText(6,
QtCoreApplication.translate("MainWindow", u"Utilities", None))
        self.staff_dept_box.setItemText(7,
QtCoreApplication.translate("MainWindow", u"Lab Assistant", None))
        self.staff_dept_box.setItemText(8,
QtCoreApplication.translate("MainWindow", u"Finance", None))
        self.staff_dept_box.setItemText(9,
QtCoreApplication.translate("MainWindow", u"Office", None))

        __qtablewidgetitem = self.staff_attend_table.horizontalHeaderItem(0)
        __qtablewidgetitem.setText(QtCoreApplication.translate("MainWindow",
u"Staff ID", None));
        __qtablewidgetitem1 = self.staff_attend_table.horizontalHeaderItem(1)
        __qtablewidgetitem1.setText(QtCoreApplication.translate("MainWindow",
u"Staff Role", None));
        __qtablewidgetitem2 = self.staff_attend_table.horizontalHeaderItem(2)
        __qtablewidgetitem2.setText(QtCoreApplication.translate("MainWindow",
u"Staff Name", None));
        __qtablewidgetitem3 = self.staff_attend_table.horizontalHeaderItem(3)
        __qtablewidgetitem3.setText(QtCoreApplication.translate("MainWindow",
u"Date", None));
        __qtablewidgetitem4 = self.staff_attend_table.horizontalHeaderItem(4)
        __qtablewidgetitem4.setText(QtCoreApplication.translate("MainWindow",
u"Attendance", None));
        self.fetch_btn.setText(QtCoreApplication.translate("MainWindow", u"Fetch
Attendance", None))
        self.save_btn.setText(QtCoreApplication.translate("MainWindow", u"Save
Attendance", None))
        self.tabWidget.setTabText(self.tabWidget.indexOf(self.tab),
QtCoreApplication.translate("MainWindow", u"Staff Attendance", None))
# retranslateUi

```

```

# -*- coding: utf-8 -*-

#####
#####
## Form generated from reading UI file 'staff_recwocrxw.ui'
##
## Created by: Qt User Interface Compiler version 6.4.0
##
## WARNING! All changes made in this file will be lost when recompiling UI file!
#####
#####

from PySide6.QtCore import (QCoreApplication, QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt)
from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
    QFont, QFontDatabase, QGradient, QIcon,
    QImage, QKeySequence, QLinearGradient, QPainter,
    QPalette, QPixmap, QRadialGradient, QTransform)
from PySide6.QtWidgets import (QApplication, QComboBox, QFormLayout,
    QFrame,
    QHBoxLayout, QHeaderView, QLabel, QLineEdit,
    QMainWindow, QPushButton, QSizePolicy, QSpacerItem,
    QTabWidget, QTableWidget, QTableWidgetItem, QVBoxLayout,
    QWidget)

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.setObjectName():
            MainWindow.setObjectName(u"MainWindow")
        MainWindow.resize(800, 600)
        self.centralwidget = QWidget(MainWindow)
        self.centralwidget.setObjectName(u"centralwidget")
        self.horizontalLayout = QHBoxLayout(self.centralwidget)
        self.horizontalLayout.setObjectName(u"horizontalLayout")
        self.tabWidget = QTabWidget(self.centralwidget)
        self.tabWidget.setObjectName(u"tabWidget")
        self.tabWidget.setStyleSheet(u"/* Styling the Tab Widget */\n"
"QTabWidget {\n"
"    background-color: #f0f0f0;\n"
"    border: 2px solid black;\n"
"    border-radius: 10px;\n"
"    padding: 5px;\n"
"}\n"
"\n"
"\n"
"/* Styling the Tabs */\n"
"QTabBar::tab {\n"
"    background-color: #2196F3; /* Tab background color */\n"

```

```

" color: white;\n"
" border: 1px solid #c0c0c0;\n"
" padding: 10px;\n"
" border-radius: 5px;\n"
" min-width: 80px;\n"
" min-height: 30px;\n"
"\n"
" font: 14pt \"Segoe UI\";\n"
"}\n"
"\n"
""
    self.tab = QWidget()
    self.tab.setObjectName(u"tab")
    self.tab.setStyleSheet(u"")
    self.verticalLayout = QVBoxLayout(self.tab)
    self.verticalLayout.setObjectName(u"verticalLayout")
    self.horizontalLayout_2 = QHBoxLayout()
    self.horizontalLayout_2.setObjectName(u"horizontalLayout_2")
    self.horizontalLayout_2.setContentsMargins(-1, 0, -1, -1)
    self.formFrame = QFrame(self.tab)
    self.formFrame.setObjectName(u"formFrame")
    self.formFrame.setStyleSheet(u"*{\n"
"font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/* Styling the QLineEdit */\n"
"QLineEdit {\n"
" background-color: #f4f4f4; /* Light grey background */\n"
" color: #333333; /* Dark text color */\n"
" border: 2px solid #c0c0c0; /* Border color */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 5px;\n"
"}\n"
"\n"
"/* Focused state (when the QLineEdit is selected) */\n"
"QLineEdit:focus {\n"
" border: 2px solid #2196F3; /* Blue border when focused */\n"
" background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state */\n"
"QLineEdit:disabled {\n"
" background-color: #e0e0e0; /* Grey background when disabled */\n"
" color: #888888; /* Light grey text when disabled */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Placeholder text style */\n"
"QLineEdit::placeholder {\n"

```

```

" color: #888888; /* Light grey color for placeholder text */\n"
" font-style: italic; /* Italicize the placeholder text */\n"
""

    "}\n"
\n"
/* Styling the text cursor */\n"
"QLineEdit::cursor {\n"
" color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
""
    self.formLayout = QFormLayout(self.formFrame)
    self.formLayout.setObjectName(u"formLayout")
    self.formLayout.setContentsMargins(-1, 0, 0, 0)
    self.staff_dept_box = QComboBox(self.formFrame)
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.addItem("")
    self.staff_dept_box.setObjectName(u"staff_dept_box")
    self.staff_dept_box.setStyleSheet(u"/* Styling the QComboBox */\n"
"QComboBox {\n"
" background-color: #f4f4f4; /* Light grey background */\n"
" color: #333333; /* Dark text color */\n"
" border: 2px solid #c0c0c0; /* Border color */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 5px;\n"
" font: 14pt \"Segoe UI\";\n"
"\n"
" min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
/* Styling the combo box when focused */\n"
"QComboBox:focus {\n"
" border: 2px solid #2196F3; /* Blue border when focused */\n"
" background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
/* Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
" background-color: #e0e0e0; /* Grey background when disabled */\n"
" color: #888888; /* Light grey text when disabled */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"

```

```

"/ * Arrow button of the combo box */\n"
"\n"
"\n"
"/ * Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
"    background-color"
        ": #ffffff; /* White background for the dropdown list */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
"    border-radius: 5px;\n"
"    padding: 5px;\n"
"    min-width: 150px;\n"
"}\n"
"\n"
"/ * Hover effect on the dropdown list items */\n"
"QComboBox QAbstractItemView::item:hover {\n"
"    background-color: #2196F3; /* Blue background on hover */\n"
"    color: #ffffff; /* White text on hover */\n"
"}\n"
"\n"
"/ * Selected item in the dropdown list */\n"
"QComboBox QAbstractItemView::item:selected {\n"
"    background-color: #64B5F6; /* Light blue background when selected */\n"
"    color: #ffffff; /* White text when selected */\n"
"}\n"
""
)

self.formLayout.setWidget(3, QFormLayout.FieldRole, self.staff_dept_box)

self.label_16 = QLabel(self.formFrame)
self.label_16.setObjectName(u"label_16")

self.formLayout.setWidget(3, QFormLayout.LabelRole, self.label_16)

self.label_14 = QLabel(self.formFrame)
self.label_14.setObjectName(u"label_14")

self.formLayout.setWidget(2, QFormLayout.LabelRole, self.label_14)

self.staff_id_box = QLineEdit(self.formFrame)
self.staff_id_box.setObjectName(u"staff_id_box")

self.formLayout.setWidget(2, QFormLayout.FieldRole, self.staff_id_box)

self.horizontalLayout_2.addWidget(self.formFrame, 0, Qt.AlignTop)

self.verticalLayout.addLayout(self.horizontalLayout_2)

```



```

self.horizontalLayout_4 = QHBoxLayout()
self.horizontalLayout_4.setObjectName(u"horizontalLayout_4")
self.horizontalLayout_4.setContentsMargins(-1, 0, -1, -1)
self.staff_table = QTableWidgetItem(self.tab)
if (self.staff_table.columnCount() < 7):
    self.staff_table.setColumnCount(7)
__qtablewidgetitem = QTableWidgetItem()
self.staff_table.setHorizontalHeaderItem(0, __qtablewidgetitem)
__qtablewidgetitem1 = QTableWidgetItem()
self.staff_table.setHorizontalHeaderItem(1, __qtablewidgetitem1)
__qtablewidgetitem2 = QTableWidgetItem()
self.staff_table.setHorizontalHeaderItem(2, __qtablewidgetitem2)
__qtablewidgetitem3 = QTableWidgetItem()
self.staff_table.setHorizontalHeaderItem(3, __qtablewidgetitem3)
__qtablewidgetitem4 = QTableWidgetItem()
self.staff_table.setHorizontalHeaderItem(4, __qtablewidgetitem4)
__qtablewidgetitem5 = QTableWidgetItem()
self.staff_table.setHorizontalHeaderItem(5, __qtablewidgetitem5)
__qtablewidgetitem6 = QTableWidgetItem()
self.staff_table.setHorizontalHeaderItem(6, __qtablewidgetitem6)
if (self.staff_table.rowCount() < 3):
    self.staff_table.setRowCount(3)
self.staff_table.setObjectName(u"staff_table")
font = QFont()
font.setPointSize(14)
font.setBold(False)
self.staff_table.setFont(font)
self.staff_table.setStyleSheet(u"QHeaderView::section {
\n"
"background-color: rgb(0, 120, 212);\n"
" /* Green background for headers */\n"
" color: white; /* White text color */\n"
" font-weight: bold; /* Bold font for header text */\n"
" text-align: center; /* Center-align text */\n"
" \n"
" font: 700 14pt \"Segoe UI\";\n"
" border: 1px solid #ccc; /* Light gray border around header sections */\n"
" padding: 5px; /* Padding inside header cells */\n"
"}\n"
""")

self.horizontalLayout_4.addWidget(self.staff_table)

self.verticalLayout.addLayout(self.horizontalLayout_4)

self.horizontalLayout_3 = QHBoxLayout()
self.horizontalLayout_3.setObjectName(u"horizontalLayout_3")
self.horizontalLayout_3.setContentsMargins(-1, 0, -1, -1)
self.horizontalSpacer = QSpacerItem(40, 20, QSizePolicy.Expanding,
QSizePolicy.Minimum)

```



```

self.horizontalLayout_3.addItem(self.horizontalSpacer)

self.fetch_btn = QPushButton(self.tab)
self.fetch_btn.setObjectName(u"fetch_btn")
self.fetch_btn.setStyleSheet(u"/* Styling the QPushButton */\n"
"QPushButton {\n"
"  background-color: #4CAF50; /* Green background */\n"
"  color: white; /* White text color */\n"
"  border: 2px solid #4CAF50; /* Border color matches background */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 10px 20px; /* Vertical and horizontal padding */\n"
"  \n"
"  font: 14pt \"Segoe UI\"; /* Font size */\n"
"  min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
"/* Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
"  background-color: #45a049; /* Darker green background */\n"
"  border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
"/* Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
"  background-color: #388e3c; /* Even darker green */\n"
"  border-color: #388e3c; /* Dark border */\n"
"  padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
"  padding-left: 18px; /* Shift the button to the left slightly */\n"
"}\n"
"\n"
"/* Focused state (when the button is focused) */\n"
"QPushButton:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  outline: none; /* Removes the default outline */\n"
"}\n"
"\n"
"/* Disabled state for the QPushButton */\n"
"QPushButton:disabled {\n"
"  background-color: #e0e0e0; /* Light grey background */\n"
"  color: #888888; /* Light grey text */\n"
"  border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Styling the text of the QPushButton */\n"
"QPushButton {\n"
"  font-weight: bold; /* Make the text bold */\n"
"}\n"

```

```

"\n"
"/** Styling the QPushButton icon (if it has one) */\n"
"QPushButton::icon {\n"
"    padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"
"\n"
"/** Border effect when the button is in a focus or pressed state */\n"
"QPushButton:focus, QPushButton:pressed {\n"
"    border-style: inset;\n"
"}\n"
""")

self.horizontalLayout_3.addWidget(self.fetch_btn)

self.verticalLayout.addLayout(self.horizontalLayout_3)

self.verticalSpacer = QSpacerItem(20, 40, QSizePolicy.Minimum,
QSizePolicy.Expanding)

self.verticalLayout.addItem(self.verticalSpacer)

self.tabWidget.addTab(self.tab, "")

self.horizontalLayout.addWidget(self.tabWidget)

MainWindow.setCentralWidget(self.centralwidget)

self.retranslateUi(MainWindow)

self.tabWidget.setCurrentIndex(0)


QMetaObject.connectSlotsByName(MainWindow)
# setupUi

def retranslateUi(self, MainWindow):
    MainWindow.setWindowTitle(QCoreApplication.translate("MainWindow",
u"MainWindow", None))
    self.staff_dept_box.setItemText(0,
QCoreApplication.translate("MainWindow", u"Primary - 1 to 5", None))
    self.staff_dept_box.setItemText(1,
QCoreApplication.translate("MainWindow", u"Middle High - 6 to 8", None))
    self.staff_dept_box.setItemText(2,
QCoreApplication.translate("MainWindow", u"Secondary - 9 to 10", None))
    self.staff_dept_box.setItemText(3,
QCoreApplication.translate("MainWindow", u"Science", None))
    self.staff_dept_box.setItemText(4,
QCoreApplication.translate("MainWindow", u"Commerce", None))

```

```

        self.staff_dept_box.setItemText(5,
QtCoreApplication.translate("MainWindow", u"Humanities", None))
        self.staff_dept_box.setItemText(6,
QtCoreApplication.translate("MainWindow", u"Utilities", None))
        self.staff_dept_box.setItemText(7,
QtCoreApplication.translate("MainWindow", u"Lab Assistant", None))
        self.staff_dept_box.setItemText(8,
QtCoreApplication.translate("MainWindow", u"Finance", None))
        self.staff_dept_box.setItemText(9,
QtCoreApplication.translate("MainWindow", u"Office", None))

        self.label_16.setText(QtCoreApplication.translate("MainWindow",
u"Department", None))
        self.label_14.setText(QtCoreApplication.translate("MainWindow", u"Staff
ID", None))
        __qtablewidgetitem = self.staff_table.horizontalHeaderItem(0)
        __qtablewidgetitem.setText(QtCoreApplication.translate("MainWindow",
u"Staff ID", None));
        __qtablewidgetitem1 = self.staff_table.horizontalHeaderItem(1)
        __qtablewidgetitem1.setText(QtCoreApplication.translate("MainWindow",
u"Category", None));
        __qtablewidgetitem2 = self.staff_table.horizontalHeaderItem(2)
        __qtablewidgetitem2.setText(QtCoreApplication.translate("MainWindow",
u"Department", None));
        __qtablewidgetitem3 = self.staff_table.horizontalHeaderItem(3)
        __qtablewidgetitem3.setText(QtCoreApplication.translate("MainWindow",
u"Staff Name", None));
        __qtablewidgetitem4 = self.staff_table.horizontalHeaderItem(4)
        __qtablewidgetitem4.setText(QtCoreApplication.translate("MainWindow",
u"DOB", None));
        __qtablewidgetitem5 = self.staff_table.horizontalHeaderItem(5)
        __qtablewidgetitem5.setText(QtCoreApplication.translate("MainWindow",
u"Gender", None));
        __qtablewidgetitem6 = self.staff_table.horizontalHeaderItem(6)
        __qtablewidgetitem6.setText(QtCoreApplication.translate("MainWindow",
u"Contact No.", None));
        self.fetch_btn.setText(QtCoreApplication.translate("MainWindow", u"Fetch
Details", None))
        self.tabWidget.setTabText(self.tabWidget.indexOf(self.tab),
QtCoreApplication.translate("MainWindow", u"Staff Records", None))
# retranslateUi

```

13.18 ui_staff.py

```
# -*- coding: utf-8 -*-

#####
#####
## Form generated from reading UI file 'staffHHqxs.ui'
##
## Created by: Qt User Interface Compiler version 6.4.0
##
## WARNING! All changes made in this file will be lost when recompiling UI file!
#####
#####

from PySide6.QtCore import (QCoreApplication, QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt)
from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
    QFont, QFontDatabase, QGradient, QIcon,
    QImage, QKeySequence, QLinearGradient, QPainter,
    QPalette, QPixmap, QRadialGradient, QTransform)
from PySide6.QtWidgets import (QApplication, QComboBox, QFormLayout,
    QFrame,
    QHBoxLayout, QHeaderView, QLabel, QLineEdit,
    QMainWindow, QPlainTextEdit, QPushButton, QSizePolicy,
    QSpacerItem, QTabWidget, QTableWidget, QTableWidgetItem,
    QVBoxLayout, QWidget)

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.setObjectName():
            MainWindow.setObjectName(u"MainWindow")
        MainWindow.resize(800, 600)
        self.centralwidget = QWidget(MainWindow)
        self.centralwidget.setObjectName(u"centralwidget")
        self.verticalLayout_2 = QVBoxLayout(self.centralwidget)
        self.verticalLayout_2.setObjectName(u"verticalLayout_2")
        self.tabWidget = QTabWidget(self.centralwidget)
        self.tabWidget.setObjectName(u"tabWidget")
        self.tabWidget.setStyleSheet(u"/* Styling the Tab Widget */\n"
"QTabWidget {\n"
"    background-color: #f0f0f0;\n"
"    border: 2px solid black;\n"
"    border-radius: 10px;\n"
"    padding: 5px;\n"
"}\n"
"\n"
"\n"
"/* Styling the Tabs */\n"
"QTabBar::tab {\n"
"    background-color: #2196F3; /* Tab background color */\n"
```

```

" color: white;\n"
" border: 1px solid #c0c0c0;\n"
" padding: 10px;\n"
" border-radius: 5px;\n"
" min-width: 80px;\n"
" min-height: 30px;\n"
"\n"
" font: 14pt \"Segoe UI\";\n"
"}\n"
"\n"
""
    self.tab = QWidget()
    self.tab.setObjectName(u"tab")
    self.tab.setStyleSheet(u"")
    self.verticalLayout = QVBoxLayout(self.tab)
    self.verticalLayout.setObjectName(u"verticalLayout")
    self.horizontalLayout_2 = QHBoxLayout()
    self.horizontalLayout_2.setObjectName(u"horizontalLayout_2")
    self.horizontalLayout_2.setContentsMargins(-1, 0, -1, -1)
    self.formFrame = QFrame(self.tab)
    self.formFrame.setObjectName(u"formFrame")
    self.formFrame.setStyleSheet(u"*{\n"
"font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/* Styling the QLineEdit */\n"
"QLineEdit {\n"
" background-color: #f4f4f4; /* Light grey background */\n"
" color: #333333; /* Dark text color */\n"
" border: 2px solid #c0c0c0; /* Border color */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 5px;\n"
"}\n"
"\n"
"/* Focused state (when the QLineEdit is selected) */\n"
"QLineEdit:focus {\n"
" border: 2px solid #2196F3; /* Blue border when focused */\n"
" background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state */\n"
"QLineEdit:disabled {\n"
" background-color: #e0e0e0; /* Grey background when disabled */\n"
" color: #888888; /* Light grey text when disabled */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Placeholder text style */\n"
"QLineEdit::placeholder {\n"

```

```

"    color: #888888; /* Light grey color for placeholder text */\n"
"    font-style: italic; /* Italicize the placeholder text */\n"
""
        "}\n"
"\n"
"/\n" Styling the text cursor */\n"
"QLineEdit::cursor {\n"
"    color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
""
    self.formLayout = QFormLayout(self.formFrame)
    self.formLayout.setObjectName(u"formLayout")
    self.formLayout.setContentsMargins(-1, 0, 0, 0)
    self.label_2 = QLabel(self.formFrame)
    self.label_2.setObjectName(u"label_2")

    self.formLayout.addWidget(0, QFormLayout.LabelRole, self.label_2)

    self.staff_id_label = QLabel(self.formFrame)
    self.staff_id_label.setObjectName(u"staff_id_label")
    self.staff_id_label.setStyleSheet(u"color: rgb(255, 0, 0);\n"
"font: 700 14pt \"Segoe UI\";")

    self.formLayout.addWidget(0, QFormLayout.FieldRole, self.staff_id_label)

    self.label_4 = QLabel(self.formFrame)
    self.label_4.setObjectName(u"label_4")

    self.formLayout.addWidget(1, QFormLayout.LabelRole, self.label_4)

    self.staff_category_box = QComboBox(self.formFrame)
    self.staff_category_box.addItem("")
    self.staff_category_box.addItem("")
    self.staff_category_box.setObjectName(u"staff_category_box")
    self.staff_category_box.setStyleSheet(u"/\n" Styling the QComboBox */\n"
"QComboBox {\n"
"    background-color: #f4f4f4; /* Light grey background */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 2px solid #c0c0c0; /* Border color */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 5px;\n"
"    font: 14pt \"Segoe UI\";\n"
"}\n"
"\n"
"    min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
"/\n" Styling the combo box when focused */\n"
"QComboBox:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    background-color: #ffffff; /* White background on focus */\n"

```

```

"}\n"
"\n"
"/ * Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
"    background-color: #e0e0e0; /* Grey background when disabled */\n"
"    color: #888888; /* Light grey text when disabled */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/ * Arrow button of the combo box */\n"
"\n"
"\n"
"/ * Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
"    background-color
        ": #ffffff; /* White background for the dropdown list */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
"    border-radius: 5px;\n"
"    padding: 5px;\n"
"    min-width: 150px;\n"
"}\n"
"\n"
"/ * Hover effect on the dropdown list items */\n"
"QComboBox QAbstractItemView::item:hover {\n"
"    background-color: #2196F3; /* Blue background on hover */\n"
"    color: #ffffff; /* White text on hover */\n"
"}\n"
"\n"
"/ * Selected item in the dropdown list */\n"
"QComboBox QAbstractItemView::item:selected {\n"
"    background-color: #64B5F6; /* Light blue background when selected */\n"
"    color: #ffffff; /* White text when selected */\n"
"}\n"
""
)

    self.formLayout.addWidget(1, QFormLayout.FieldRole,
self.staff_category_box)

    self.label = QLabel(self.formFrame)
    self.label.setObjectName(u"label")

    self.formLayout.addWidget(2, QFormLayout.LabelRole, self.label)

    self.staff_fname_box = QLineEdit(self.formFrame)
    self.staff_fname_box.setObjectName(u"staff_fname_box")

    self.formLayout.addWidget(2, QFormLayout.FieldRole, self.staff_fname_box)

    self.label_3 = QLabel(self.formFrame)

```



```

self.label_3.setObjectName(u"label_3")

self.formLayout.addWidget(3, QFormLayout.LabelRole, self.label_3)

self.staff_lname_box = QLineEdit(self.formFrame)
self.staff_lname_box.setObjectName(u"staff_lname_box")

self.formLayout.addWidget(3, QFormLayout.FieldRole, self.staff_lname_box)

self.label_6 = QLabel(self.formFrame)
self.label_6.setObjectName(u"label_6")

self.formLayout.addWidget(4, QFormLayout.LabelRole, self.label_6)

self.staff_gen_box = QComboBox(self.formFrame)
self.staff_gen_box.addItem("")
self.staff_gen_box.addItem("")
self.staff_gen_box.setObjectName(u"staff_gen_box")
self.staff_gen_box.setStyleSheet(u"/* Styling the QComboBox */\n"
"QComboBox {\n"
"  background-color: #f4f4f4; /* Light grey background */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 2px solid #c0c0c0; /* Border color */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 5px;\n"
"  font: 14pt \"Segoe UI\";\n"
"\n"
"  min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
"/* Styling the combo box when focused */\n"
"QComboBox:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
"  background-color: #e0e0e0; /* Grey background when disabled */\n"
"  color: #888888; /* Light grey text when disabled */\n"
"  border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Arrow button of the combo box */\n"
"\n"
"\n"
"/* Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
"  background-color\n"
"    : #ffffff; /* White background for the dropdown list */\n"

```



```

" color: #333333; /* Dark text color */\n"
" border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
" border-radius: 5px;\n"
" padding: 5px;\n"
" min-width: 150px;\n"
"}\n"
"\n"
/* Hover effect on the dropdown list items */\n"
"QComboBox QAbstractItemView::item:hover {\n"
" background-color: #2196F3; /* Blue background on hover */\n"
" color: #ffffff; /* White text on hover */\n"
"}\n"
"\n"
/* Selected item in the dropdown list */\n"
"QComboBox QAbstractItemView::item:selected {\n"
" background-color: #64B5F6; /* Light blue background when selected */\n"
" color: #ffffff; /* White text when selected */\n"
"}\n"
""")

self.formLayout.addWidget(4, QFormLayout.FieldRole, self.staff_gen_box)

self.label_7 = QLabel(self.formFrame)
self.label_7.setObjectName(u"label_7")

self.formLayout.addWidget(6, QFormLayout.LabelRole, self.label_7)

self.staff_dob_box = QLineEdit(self.formFrame)
self.staff_dob_box.setObjectName(u"staff_dob_box")

self.formLayout.addWidget(6, QFormLayout.FieldRole, self.staff_dob_box)

self.horizontalLayout_2.addWidget(self.formFrame, 0, Qt.AlignTop)

self.formFrame_2 = QFrame(self.tab)
self.formFrame_2.setObjectName(u"formFrame_2")
self.formFrame_2.setStyleSheet(u"*{\n"
"font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
/* Styling the QLineEdit */\n"
"QLineEdit {\n"
" background-color: #f4f4f4; /* Light grey background */\n"
" color: #333333; /* Dark text color */\n"
" border: 2px solid #c0c0c0; /* Border color */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 5px;\n"
"}\n"

```

```

"\n"
"/ * Focused state (when the QLineEdit is selected) */\n"
"QLineEdit:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/ * Disabled state */\n"
"QLineEdit:disabled {\n"
"    background-color: #e0e0e0; /* Grey background when disabled */\n"
"    color: #888888; /* Light grey text when disabled */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/ * Placeholder text style */\n"
"QLineEdit::placeholder {\n"
"    color: #888888; /* Light grey color for placeholder text */\n"
"    font-style: italic; /* Italicize the placeholder text */\n"
""
    "}\n"
"\n"
"/ * Styling the text cursor */\n"
"QLineEdit::cursor {\n"
"    color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
""
    self.formLayout_2 = QFormLayout(self.formFrame_2)
    self.formLayout_2.setObjectName(u"formLayout_2")
    self.formLayout_2.setContentsMargins(-1, 0, 0, 0)
    self.label_11 = QLabel(self.formFrame_2)
    self.label_11.setObjectName(u"label_11")

    self.formLayout_2.setWidget(0, QFormLayout.LabelRole, self.label_11)

    self.label_12 = QLabel(self.formFrame_2)
    self.label_12.setObjectName(u"label_12")

    self.formLayout_2.setWidget(2, QFormLayout.LabelRole, self.label_12)

    self.staff_contact_box = QLineEdit(self.formFrame_2)
    self.staff_contact_box.setObjectName(u"staff_contact_box")

    self.formLayout_2.setWidget(2, QFormLayout.FieldRole,
self.staff_contact_box)

    self.label_17 = QLabel(self.formFrame_2)
    self.label_17.setObjectName(u"label_17")

    self.formLayout_2.setWidget(3, QFormLayout.LabelRole, self.label_17)

```

```

self.staff_email_box = QLineEdit(self.formFrame_2)
self.staff_email_box.setObjectName(u"staff_email_box")

self.formLayout_2.setWidget(3, QFormLayout.FieldRole,
self.staff_email_box)

self.label_13 = QLabel(self.formFrame_2)
self.label_13.setObjectName(u"label_13")

self.formLayout_2.setWidget(4, QFormLayout.LabelRole, self.label_13)

self.label_14 = QLabel(self.formFrame_2)
self.label_14.setObjectName(u"label_14")

self.formLayout_2.setWidget(8, QFormLayout.LabelRole, self.label_14)

self.staff_address_box = QPlainTextEdit(self.formFrame_2)
self.staff_address_box.setObjectName(u"staff_address_box")
self.staff_address_box.setMaximumSize(QSize(16777215, 80))
self.staff_address_box.setStyleSheet(u"/** Styling the QPlainTextEdit */\n"
"QPlainTextEdit {\n"
"  background-color: #f4f4f4; /* Light grey background */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 2px solid #c0c0c0; /* Border color */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 8px;\n"
"  font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/** Focused state (when the QPlainTextEdit is selected) */\n"
"QPlainTextEdit:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/** Disabled state */\n"
"QPlainTextEdit:disabled {\n"
"  background-color: #e0e0e0; /* Grey background when disabled */\n"
"  color: #888888; /* Light grey text when disabled */\n"
"  border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/** Styling the text cursor */\n"
"QPlainTextEdit::cursor {\n"
"  color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
"\n"
"/** Styling the selected text */\n"
"QPlainTextEdit::"

```

```

        "selection {\n"
        "    background-color: #64B5F6; /* Light blue background for selected text */\n"
        "    color: #ffffff; /* White text color for selected text */\n"
        "}\n"
        "\n"
        "/* Styling the scroll bar */\n"
        "QPlainTextEdit QScrollBar:vertical {\n"
        "    border: none;\n"
        "    background: #f4f4f4;\n"
        "    width: 12px;\n"
        "    margin: 0px, 0px, 0px, 0px;\n"
        "    border-radius: 6px;\n"
        "}\n"
        "\n"
        "QPlainTextEdit QScrollBar::handle:vertical {\n"
        "    background: #2196F3;\n"
        "    border-radius: 6px;\n"
        "}\n"
        "\n"
        "QPlainTextEdit QScrollBar::add-line:vertical, \n"
        "QPlainTextEdit QScrollBar::sub-line:vertical {\n"
        "    border: none;\n"
        "    background: none;\n"
        "}\n"
        """)

```

```

        self.formLayout_2.setWidget(0, QFormLayout.FieldRole,
        self.staff_address_box)

```

```

        self.staff_dept_box = QComboBox(self.formFrame_2)
        self.staff_dept_box.addItem("")
        self.staff_dept_box.addItem("")
        self.staff_dept_box.addItem("")
        self.staff_dept_box.addItem("")
        self.staff_dept_box.addItem("")
        self.staff_dept_box.addItem("")
        self.staff_dept_box.addItem("")
        self.staff_dept_box.addItem("")
        self.staff_dept_box.addItem("")
        self.staff_dept_box.addItem("")
        self.staff_dept_box.setObjectName(u"staff_dept_box")
        self.staff_dept_box.setStyleSheet(u"/* Styling the QComboBox */\n"
"QComboBox {\n"
"    background-color: #f4f4f4; /* Light grey background */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 2px solid #c0c0c0; /* Border color */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 5px;\n"
"    font: 14pt \"Segoe UI\";\n"
"\n"

```

```

"    min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
"/* Styling the combo box when focused */\n"
"QComboBox:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
"    background-color: #e0e0e0; /* Grey background when disabled */\n"
"    color: #888888; /* Light grey text when disabled */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Arrow button of the combo box */\n"
"\n"
"\n"
"/* Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
"    background-color\n"
"        : #ffffff; /* White background for the dropdown list */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
"    border-radius: 5px;\n"
"    padding: 5px;\n"
"    min-width: 150px;\n"
"}\n"
"\n"
"/* Hover effect on the dropdown list items */\n"
"QComboBox QAbstractItemView::item:hover {\n"
"    background-color: #2196F3; /* Blue background on hover */\n"
"    color: #ffffff; /* White text on hover */\n"
"}\n"
"\n"
"/* Selected item in the dropdown list */\n"
"QComboBox QAbstractItemView::item:selected {\n"
"    background-color: #64B5F6; /* Light blue background when selected */\n"
"    color: #ffffff; /* White text when selected */\n"
"}\n"
""
)

```

```

        self.formLayout_2.setWidget(4, QFormLayout.FieldRole,
self.staff_dept_box)

```

```

        self.staff_sal_box = QLineEdit(self.formFrame_2)
        self.staff_sal_box.setObjectName(u"staff_sal_box")

```

```

        self.formLayout_2.setWidget(8, QFormLayout.FieldRole, self.staff_sal_box)

```

```

self.horizontalLayout_2.addWidget(self.formFrame_2, 0, Qt.AlignTop)

self.verticalLayout.addLayout(self.horizontalLayout_2)

self.horizontalLayout_3 = QHBoxLayout()
self.horizontalLayout_3.setObjectName(u"horizontalLayout_3")
self.horizontalLayout_3.setContentsMargins(-1, 0, -1, -1)
self.horizontalSpacer = QSpacerItem(40, 20, QSizePolicy.Expanding,
QSizePolicy.Minimum)

self.horizontalLayout_3.addItem(self.horizontalSpacer)

self.save_btn = QPushButton(self.tab)
self.save_btn.setObjectName(u"save_btn")
self.save_btn.setStyleSheet(u"/* Styling the QPushButton */\n"
"QPushButton {\n"
"  background-color: #4CAF50; /* Green background */\n"
"  color: white; /* White text color */\n"
"  border: 2px solid #4CAF50; /* Border color matches background */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 10px 20px; /* Vertical and horizontal padding */\n"
"  \n"
"  font: 14pt \"Segoe UI\"; /* Font size */\n"
"  min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
"/* Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
"  background-color: #45a049; /* Darker green background */\n"
"  border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
"/* Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
"  background-color: #388e3c; /* Even darker green */\n"
"  border-color: #388e3c; /* Dark border */\n"
"  padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
"  padding-left: 18px; /* Shift the button to the left slightly */\n"
"}\n"
"\n"
"/* Focused state (when the button is focused) */\n"
"QPushButton:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  outline: none; /* Removes the default outline */\n"
"}\n"

```

```

"\n"
"/ * Disabled state for the QPushButton */\n"
"QPushButton:disabled {\n"
"    background-color: #e0e0e0; /* Light grey background */\n"
"    color: #888888; /* Light grey text */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/ * Styling the text of the QPushButton */\n"
"QPushButton {\n"
"    font-weight: bold; /* Make the text bold */\n"
"}\n"
"\n"
"/ * Styling the QPushButton icon (if it has one) */\n"
"QPushButton::icon {\n"
"    padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"
"\n"
"/ * Border effect when the button is in a focus or pressed state */\n"
"QPushButton:focus, QPushButton:pressed {\n"
"    border-style: inset;\n"
"}\n"
""")

```

```

self.horizontalLayout_3.addWidget(self.save_btn)

```

```

self.verticalLayout.addLayout(self.horizontalLayout_3)

```

```

self.staff_table = QTableWidgetItem()
if (self.staff_table.columnCount() < 10):
    self.staff_table.setColumnCount(10)
    __qtablewidgetitem = QTableWidgetItem()
    self.staff_table.setHorizontalHeaderItem(0, __qtablewidgetitem)
    __qtablewidgetitem1 = QTableWidgetItem()
    self.staff_table.setHorizontalHeaderItem(1, __qtablewidgetitem1)
    __qtablewidgetitem2 = QTableWidgetItem()
    self.staff_table.setHorizontalHeaderItem(2, __qtablewidgetitem2)
    __qtablewidgetitem3 = QTableWidgetItem()
    self.staff_table.setHorizontalHeaderItem(3, __qtablewidgetitem3)
    __qtablewidgetitem4 = QTableWidgetItem()
    self.staff_table.setHorizontalHeaderItem(4, __qtablewidgetitem4)
    __qtablewidgetitem5 = QTableWidgetItem()
    self.staff_table.setHorizontalHeaderItem(5, __qtablewidgetitem5)
    __qtablewidgetitem6 = QTableWidgetItem()
    self.staff_table.setHorizontalHeaderItem(6, __qtablewidgetitem6)
    __qtablewidgetitem7 = QTableWidgetItem()
    self.staff_table.setHorizontalHeaderItem(7, __qtablewidgetitem7)
    __qtablewidgetitem8 = QTableWidgetItem()
    self.staff_table.setHorizontalHeaderItem(8, __qtablewidgetitem8)

```

```

__qtablewidgetitem9 = QTableWidgetItem()
self.staff_table.setHorizontalHeaderItem(9, __qtablewidgetitem9)
if (self.staff_table.rowCount() < 3):
    self.staff_table.setRowCount(3)
self.staff_table.setObjectName(u"staff_table")
font = QFont()
font.setPointSize(14)
font.setBold(False)
self.staff_table.setFont(font)
self.staff_table.setStyleSheet(u"QHeaderView::section {\n"
"background-color: rgb(0, 120, 212);\n"
" /* Green background for headers */\n"
" color: white; /* White text color */\n"
" font-weight: bold; /* Bold font for header text */\n"
" text-align: center; /* Center-align text */\n"
" \n"
" font: 700 14pt \"Segoe UI\";\n"
" border: 1px solid #ccc; /* Light gray border around header sections */\n"
" padding: 5px; /* Padding inside header cells */\n"
"}\n"
"")

```

```

self.verticalLayout.addWidget(self.staff_table)

```

```

self.verticalSpacer = QSpacerItem(20, 40, QSizePolicy.Minimum,
QSizePolicy.Expanding)

```

```

self.verticalLayout.addItem(self.verticalSpacer)

```

```

self.tabWidget.addTab(self.tab, "")

```

```

self.verticalLayout_2.addWidget(self.tabWidget)

```

```

MainWindow.setCentralWidget(self.centralwidget)

```

```

self.retranslateUi(MainWindow)

```

```

self.tabWidget.setCurrentIndex(0)

```

```

QMetaObject.connectSlotsByName(MainWindow)
# setupUi

```

```

def retranslateUi(self, MainWindow):
    MainWindow.setWindowTitle(QCoreApplication.translate("MainWindow",
u"MainWindow", None))
    self.label_2.setText(QCoreApplication.translate("MainWindow", u"Staff
ID", None))
    self.staff_id_label.setText(QCoreApplication.translate("MainWindow",
u"RG-1209", None))

```



```

        self.label_4.setText(QCoreApplication.translate("MainWindow",
u"Category", None))
        self.staff_category_box.setItemText(0,
QCoreApplication.translate("MainWindow", u"Teaching Staff", None))
        self.staff_category_box.setItemText(1,
QCoreApplication.translate("MainWindow", u"Non-Teaching Staff", None))

        self.label.setText(QCoreApplication.translate("MainWindow", u"First
Name", None))
        self.label_3.setText(QCoreApplication.translate("MainWindow", u"Last
Name", None))
        self.label_6.setText(QCoreApplication.translate("MainWindow",
u"Gender", None))
        self.staff_gen_box.setItemText(0,
QCoreApplication.translate("MainWindow", u"Male", None))
        self.staff_gen_box.setItemText(1,
QCoreApplication.translate("MainWindow", u"Female", None))

        self.label_7.setText(QCoreApplication.translate("MainWindow", u"DOB",
None))
        self.label_11.setText(QCoreApplication.translate("MainWindow",
u"Address", None))
        self.label_12.setText(QCoreApplication.translate("MainWindow",
u"Contact No.", None))
        self.label_17.setText(QCoreApplication.translate("MainWindow",
u"Email", None))
        self.label_13.setText(QCoreApplication.translate("MainWindow",
u"Department", None))
        self.label_14.setText(QCoreApplication.translate("MainWindow",
u"Salary", None))
        self.staff_dept_box.setItemText(0,
QCoreApplication.translate("MainWindow", u"Primary - 1 to 5", None))
        self.staff_dept_box.setItemText(1,
QCoreApplication.translate("MainWindow", u"Middle High - 6 to 8", None))
        self.staff_dept_box.setItemText(2,
QCoreApplication.translate("MainWindow", u"Secondary - 9 to 10", None))
        self.staff_dept_box.setItemText(3,
QCoreApplication.translate("MainWindow", u"Science", None))
        self.staff_dept_box.setItemText(4,
QCoreApplication.translate("MainWindow", u"Commerce", None))
        self.staff_dept_box.setItemText(5,
QCoreApplication.translate("MainWindow", u"Humanities", None))
        self.staff_dept_box.setItemText(6,
QCoreApplication.translate("MainWindow", u"Utilities", None))
        self.staff_dept_box.setItemText(7,
QCoreApplication.translate("MainWindow", u"Lab Assistant", None))
        self.staff_dept_box.setItemText(8,
QCoreApplication.translate("MainWindow", u"Finance", None))
        self.staff_dept_box.setItemText(9,
QCoreApplication.translate("MainWindow", u"Office", None))

```

```

        self.save_btn.setText(QCoreApplication.translate("MainWindow", u"Save
Details", None))
        __qtablewidgetitem = self.staff_table.horizontalHeaderItem(0)
        __qtablewidgetitem.setText(QCoreApplication.translate("MainWindow",
u"Staff ID", None));
        __qtablewidgetitem1 = self.staff_table.horizontalHeaderItem(1)
        __qtablewidgetitem1.setText(QCoreApplication.translate("MainWindow",
u"Category", None));
        __qtablewidgetitem2 = self.staff_table.horizontalHeaderItem(2)
        __qtablewidgetitem2.setText(QCoreApplication.translate("MainWindow",
u"Staff Name", None));
        __qtablewidgetitem3 = self.staff_table.horizontalHeaderItem(3)
        __qtablewidgetitem3.setText(QCoreApplication.translate("MainWindow",
u"Gender", None));
        __qtablewidgetitem4 = self.staff_table.horizontalHeaderItem(4)
        __qtablewidgetitem4.setText(QCoreApplication.translate("MainWindow",
u"DOB", None));
        __qtablewidgetitem5 = self.staff_table.horizontalHeaderItem(5)
        __qtablewidgetitem5.setText(QCoreApplication.translate("MainWindow",
u"Address", None));
        __qtablewidgetitem6 = self.staff_table.horizontalHeaderItem(6)
        __qtablewidgetitem6.setText(QCoreApplication.translate("MainWindow",
u"Contact No.", None));
        __qtablewidgetitem7 = self.staff_table.horizontalHeaderItem(7)
        __qtablewidgetitem7.setText(QCoreApplication.translate("MainWindow",
u"Email", None));
        __qtablewidgetitem8 = self.staff_table.horizontalHeaderItem(8)
        __qtablewidgetitem8.setText(QCoreApplication.translate("MainWindow",
u"Dept", None));
        __qtablewidgetitem9 = self.staff_table.horizontalHeaderItem(9)
        __qtablewidgetitem9.setText(QCoreApplication.translate("MainWindow",
u"Salary", None));
        self.tabWidget.setTabText(self.tabWidget.indexOf(self.tab),
QCoreApplication.translate("MainWindow", u"Enter Staff Details", None))
        # retranslateUi

```

13.19 ui_std_records.py

```
# -*- coding: utf-8 -*-

#####
#####
## Form generated from reading UI file 'std_recordsIYJwIV.ui'
##
## Created by: Qt User Interface Compiler version 6.4.0
##
## WARNING! All changes made in this file will be lost when recompiling UI file!
#####
#####

from PySide6.QtCore import (QCoreApplication, QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt)
from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
    QFont, QFontDatabase, QGradient, QIcon,
    QImage, QKeySequence, QLinearGradient, QPainter,
    QPalette, QPixmap, QRadialGradient, QTransform)
from PySide6.QtWidgets import (QApplication, QComboBox, QFormLayout,
    QFrame,
    QHBoxLayout, QHeaderView, QLabel, QLineEdit,
    QMainWindow, QPushButton, QSizePolicy, QSpacerItem,
    QTabWidget, QTableWidget, QTableWidgetItem, QVBoxLayout,
    QWidget)

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.setObjectName():
            MainWindow.setObjectName(u"MainWindow")
        MainWindow.resize(800, 600)
        self.centralwidget = QWidget(MainWindow)
        self.centralwidget.setObjectName(u"centralwidget")
        self.horizontalLayout = QHBoxLayout(self.centralwidget)
        self.horizontalLayout.setObjectName(u"horizontalLayout")
        self.tabWidget = QTabWidget(self.centralwidget)
        self.tabWidget.setObjectName(u"tabWidget")
        self.tabWidget.setStyleSheet(u"/* Styling the Tab Widget */\n"
"QTabWidget {\n"
"    background-color: #f0f0f0;\n"
"    border: 2px solid black;\n"
"    border-radius: 10px;\n"
"    padding: 5px;\n"
"}\n"
"\n"
"\n"
"/* Styling the Tabs */\n"
"QTabBar::tab {\n"
"    background-color: #2196F3; /* Tab background color */\n"
```

```

" color: white;\n"
" border: 1px solid #c0c0c0;\n"
" padding: 10px;\n"
" border-radius: 5px;\n"
" min-width: 80px;\n"
" min-height: 30px;\n"
"\n"
" font: 14pt \"Segoe UI\";\n"
"}\n"
"\n"
"")
    self.tab = QWidget()
    self.tab.setObjectName(u"tab")
    self.tab.setStyleSheet(u"")
    self.verticalLayout = QVBoxLayout(self.tab)
    self.verticalLayout.setObjectName(u"verticalLayout")
    self.horizontalLayout_2 = QHBoxLayout()
    self.horizontalLayout_2.setObjectName(u"horizontalLayout_2")
    self.horizontalLayout_2.setContentsMargins(-1, 0, -1, -1)
    self.formFrame = QFrame(self.tab)
    self.formFrame.setObjectName(u"formFrame")
    self.formFrame.setStyleSheet(u"*{\n"
"font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/* Styling the QLineEdit */\n"
"QLineEdit {\n"
" background-color: #f4f4f4; /* Light grey background */\n"
" color: #333333; /* Dark text color */\n"
" border: 2px solid #c0c0c0; /* Border color */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 5px;\n"
"}\n"
"\n"
"/* Focused state (when the QLineEdit is selected) */\n"
"QLineEdit:focus {\n"
" border: 2px solid #2196F3; /* Blue border when focused */\n"
" background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state */\n"
"QLineEdit:disabled {\n"
" background-color: #e0e0e0; /* Grey background when disabled */\n"
" color: #888888; /* Light grey text when disabled */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Placeholder text style */\n"
"QLineEdit::placeholder {\n"

```

```

"    color: #888888; /* Light grey color for placeholder text */\n"
"    font-style: italic; /* Italicize the placeholder text */\n"
""

        "}\n"
"\n"
"/** Styling the text cursor */\n"
"QLineEdit::cursor {\n"
"    color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
""
    self.formLayout = QFormLayout(self.formFrame)
    self.formLayout.setObjectName(u"formLayout")
    self.formLayout.setContentsMargins(-1, 0, 0, 0)
    self.label_14 = QLabel(self.formFrame)
    self.label_14.setObjectName(u"label_14")

    self.formLayout.addWidget(0, QFormLayout.LabelRole, self.label_14)

    self.std_id_box = QLineEdit(self.formFrame)
    self.std_id_box.setObjectName(u"std_id_box")

    self.formLayout.addWidget(0, QFormLayout.FieldRole, self.std_id_box)

    self.label_13 = QLabel(self.formFrame)
    self.label_13.setObjectName(u"label_13")

    self.formLayout.addWidget(1, QFormLayout.LabelRole, self.label_13)

    self.label_15 = QLabel(self.formFrame)
    self.label_15.setObjectName(u"label_15")

    self.formLayout.addWidget(2, QFormLayout.LabelRole, self.label_15)

    self.std_class_box = QComboBox(self.formFrame)
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.setObjectName(u"std_class_box")
    self.std_class_box.setStyleSheet(u"/** Styling the QComboBox */\n"
"QComboBox {\n"
"    background-color: #f4f4f4; /* Light grey background */\n"

```

```

" color: #333333; /* Dark text color */\n"
" border: 2px solid #c0c0c0; /* Border color */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 5px;\n"
" font: 14pt \"Segoe UI\";\n"
"\n"
" min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
/* Styling the combo box when focused */\n"
"QComboBox:focus {\n"
" border: 2px solid #2196F3; /* Blue border when focused */\n"
" background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
/* Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
" background-color: #e0e0e0; /* Grey background when disabled */\n"
" color: #888888; /* Light grey text when disabled */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
/* Arrow button of the combo box */\n"
"\n"
"\n"
/* Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
" background-color\n"
"     ": #ffffff; /* White background for the dropdown list */\n"
" color: #333333; /* Dark text color */\n"
" border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
" border-radius: 5px;\n"
" padding: 5px;\n"
" min-width: 150px;\n"
"}\n"
"\n"
/* Hover effect on the dropdown list items */\n"
"QComboBox QAbstractItemView::item:hover {\n"
" background-color: #2196F3; /* Blue background on hover */\n"
" color: #ffffff; /* White text on hover */\n"
"}\n"
"\n"
/* Selected item in the dropdown list */\n"
"QComboBox QAbstractItemView::item:selected {\n"
" background-color: #64B5F6; /* Light blue background when selected */\n"
" color: #ffffff; /* White text when selected */\n"
"}\n"
""

```

```

self.formLayout.addWidget(1, QFormLayout.FieldRole, self.std_class_box)

```

```

self.std_section_box = QComboBox(self.formFrame)
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.setObjectName(u"std_section_box")
self.std_section_box.setStyleSheet(u"/** Styling the QComboBox */\n"
"QComboBox {\n"
"  background-color: #f4f4f4; /* Light grey background */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 2px solid #c0c0c0; /* Border color */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 5px;\n"
"  font: 14pt \"Segoe UI\";\n"
"\n"
"  min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
"/** Styling the combo box when focused */\n"
"QComboBox:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/** Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
"  background-color: #e0e0e0; /* Grey background when disabled */\n"
"  color: #888888; /* Light grey text when disabled */\n"
"  border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/** Arrow button of the combo box */\n"
"\n"
"\n"
"/** Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
"  background-color\n"
"      : #ffffff; /* White background for the dropdown list */\n"
"  color: #333333; /* Dark text color */\n"
"  border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
"  border-radius: 5px;\n"
"  padding: 5px;\n"
"  min-width: 150px;\n"
"}\n"
"\n"
"/** Hover effect on the dropdown list items */\n"

```



```

"QComboBox QAbstractItemView::item:hover {\n"
"  background-color: #2196F3; /* Blue background on hover */\n"
"  color: #ffffff; /* White text on hover */\n"
"}\n"
"\n"
"/* Selected item in the dropdown list */\n"
"QComboBox QAbstractItemView::item:selected {\n"
"  background-color: #64B5F6; /* Light blue background when selected */\n"
"  color: #ffffff; /* White text when selected */\n"
"}\n"
""")

self.formLayout.setWidget(2, QFormLayout.FieldRole, self.std_section_box)

self.horizontalLayout_2.addWidget(self.formFrame, 0, Qt.AlignTop)

self.verticalLayout.addLayout(self.horizontalLayout_2)

self.horizontalLayout_4 = QHBoxLayout()
self.horizontalLayout_4.setObjectName(u"horizontalLayout_4")
self.horizontalLayout_4.setContentsMargins(-1, 0, -1, -1)
self.std_table = QTableWidgetItem(self.tab)
if (self.std_table.columnCount() < 7):
    self.std_table.setColumnCount(7)
    __qtablewidgetitem = QTableWidgetItem()
    self.std_table.setHorizontalHeaderItem(0, __qtablewidgetitem)
    __qtablewidgetitem1 = QTableWidgetItem()
    self.std_table.setHorizontalHeaderItem(1, __qtablewidgetitem1)
    __qtablewidgetitem2 = QTableWidgetItem()
    self.std_table.setHorizontalHeaderItem(2, __qtablewidgetitem2)
    __qtablewidgetitem3 = QTableWidgetItem()
    self.std_table.setHorizontalHeaderItem(3, __qtablewidgetitem3)
    __qtablewidgetitem4 = QTableWidgetItem()
    self.std_table.setHorizontalHeaderItem(4, __qtablewidgetitem4)
    __qtablewidgetitem5 = QTableWidgetItem()
    self.std_table.setHorizontalHeaderItem(5, __qtablewidgetitem5)
    __qtablewidgetitem6 = QTableWidgetItem()
    self.std_table.setHorizontalHeaderItem(6, __qtablewidgetitem6)
if (self.std_table.rowCount() < 3):
    self.std_table.setRowCount(3)
self.std_table.setObjectName(u"std_table")
font = QFont()
font.setPointSize(14)
font.setBold(False)
self.std_table.setFont(font)
self.std_table.setStyleSheet(u"QHeaderView::section {\n"
"background-color: rgb(0, 120, 212);\n"
" /* Green background for headers */\n"

```



```

" color: white;          /* White text color */\n"
" font-weight: bold;     /* Bold font for header text */\n"
" text-align: center;    /* Center-align text */\n"
" \n"
" font: 700 14pt \"Segoe UI\";\n"
" border: 1px solid #ccc; /* Light gray border around header sections */\n"
" padding: 5px;          /* Padding inside header cells */\n"
"}\n"
""
)

```

```

self.horizontalLayout_4.addWidget(self.std_table)

```

```

self.verticalLayout.addLayout(self.horizontalLayout_4)

```

```

self.horizontalLayout_3 = QHBoxLayout()
self.horizontalLayout_3.setObjectName(u"horizontalLayout_3")
self.horizontalLayout_3.setContentsMargins(-1, 0, -1, -1)
self.horizontalSpacer = QSpacerItem(40, 20, QSizePolicy.Expanding,
QSizePolicy.Minimum)

```

```

self.horizontalLayout_3.addItem(self.horizontalSpacer)

```

```

self.fetch_btn = QPushButton(self.tab)
self.fetch_btn.setObjectName(u"fetch_btn")
self.fetch_btn.setStyleSheet(u"/* Styling the QPushButton */\n"
"QPushButton {\n"
" background-color: #4CAF50; /* Green background */\n"
" color: white; /* White text color */\n"
" border: 2px solid #4CAF50; /* Border color matches background */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 10px 20px; /* Vertical and horizontal padding */\n"
" \n"
" font: 14pt \"Segoe UI\"; /* Font size */\n"
" min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
"/* Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
" background-color: #45a049; /* Darker green background */\n"
" border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
"/* Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
" background-color: #388e3c; /* Even darker green */\n"
" border-color: #388e3c; /* Dark border */\n"
" padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
" padding-left: 18px; /* Shift the button to t"

```

```

        "he left slightly */\n"
    }\n"
\n"
    /* Focused state (when the button is focused) */\n"
    QPushButton:focus {\n"
    "    border: 2px solid #2196F3; /* Blue border when focused */\n"
    "    outline: none; /* Removes the default outline */\n"
    }\n"
\n"
    /* Disabled state for the QPushButton */\n"
    QPushButton:disabled {\n"
    "    background-color: #e0e0e0; /* Light grey background */\n"
    "    color: #888888; /* Light grey text */\n"
    "    border: 2px solid #cccccc; /* Light grey border */\n"
    }\n"
\n"
    /* Styling the text of the QPushButton */\n"
    QPushButton {\n"
    "    font-weight: bold; /* Make the text bold */\n"
    }\n"
\n"
    /* Styling the QPushButton icon (if it has one) */\n"
    QPushButton::icon {\n"
    "    padding-right: 10px; /* Space between the icon and the text */\n"
    }\n"
\n"
    /* Border effect when the button is in a focus or pressed state */\n"
    QPushButton:focus, QPushButton:pressed {\n"
    "    border-style: inset;\n"
    }\n"
    """)

    self.horizontalLayout_3.addWidget(self.fetch_btn)

    self.verticalLayout.addLayout(self.horizontalLayout_3)

    self.verticalSpacer = QSpacerItem(20, 40, QSizePolicy.Minimum,
    QSizePolicy.Expanding)

    self.verticalLayout.addItem(self.verticalSpacer)

    self.tabWidget.addTab(self.tab, "")

    self.horizontalLayout.addWidget(self.tabWidget)

    MainWindow.setCentralWidget(self.centralwidget)

    self.retranslateUi(MainWindow)

```

```

self.tabWidget.setCurrentIndex(0)

QMetaObject.connectSlotsByName(MainWindow)
# setupUi

def retranslateUi(self, MainWindow):
    MainWindow.setWindowTitle(QCoreApplication.translate("MainWindow",
u"MainWindow", None))
    self.label_14.setText(QCoreApplication.translate("MainWindow", u"Student
ID", None))
    self.label_13.setText(QCoreApplication.translate("MainWindow",
u"Standard", None))
    self.label_15.setText(QCoreApplication.translate("MainWindow",
u"Division", None))
    self.std_class_box.setItemText(0,
QCoreApplication.translate("MainWindow", u"Class 1", None))
    self.std_class_box.setItemText(1,
QCoreApplication.translate("MainWindow", u"Class 2", None))
    self.std_class_box.setItemText(2,
QCoreApplication.translate("MainWindow", u"Class 3", None))
    self.std_class_box.setItemText(3,
QCoreApplication.translate("MainWindow", u"Class 4", None))
    self.std_class_box.setItemText(4,
QCoreApplication.translate("MainWindow", u"Class 5", None))
    self.std_class_box.setItemText(5,
QCoreApplication.translate("MainWindow", u"Class 6", None))
    self.std_class_box.setItemText(6,
QCoreApplication.translate("MainWindow", u"Class 7", None))
    self.std_class_box.setItemText(7,
QCoreApplication.translate("MainWindow", u"Class 8", None))
    self.std_class_box.setItemText(8,
QCoreApplication.translate("MainWindow", u"Class 9", None))
    self.std_class_box.setItemText(9,
QCoreApplication.translate("MainWindow", u"Class 10", None))
    self.std_class_box.setItemText(10,
QCoreApplication.translate("MainWindow", u"Class 11", None))
    self.std_class_box.setItemText(11,
QCoreApplication.translate("MainWindow", u"Class 12", None))

    self.std_section_box.setItemText(0,
QCoreApplication.translate("MainWindow", u"A", None))
    self.std_section_box.setItemText(1,
QCoreApplication.translate("MainWindow", u"B", None))
    self.std_section_box.setItemText(2,
QCoreApplication.translate("MainWindow", u"C", None))
    self.std_section_box.setItemText(3,
QCoreApplication.translate("MainWindow", u"D", None))
    self.std_section_box.setItemText(4,
QCoreApplication.translate("MainWindow", u"E", None))

```

```

        self.std_section_box.setItemText(5,
        QApplication.translate("MainWindow", u"F", None))
        self.std_section_box.setItemText(6,
        QApplication.translate("MainWindow", u"G", None))

        __qtablewidgetitem = self.std_table.horizontalHeaderItem(0)
        __qtablewidgetitem.setText(QCoreApplication.translate("MainWindow",
        u"Student ID", None));
        __qtablewidgetitem1 = self.std_table.horizontalHeaderItem(1)
        __qtablewidgetitem1.setText(QCoreApplication.translate("MainWindow",
        u"Standard", None));
        __qtablewidgetitem2 = self.std_table.horizontalHeaderItem(2)
        __qtablewidgetitem2.setText(QCoreApplication.translate("MainWindow",
        u"Division", None));
        __qtablewidgetitem3 = self.std_table.horizontalHeaderItem(3)
        __qtablewidgetitem3.setText(QCoreApplication.translate("MainWindow",
        u"Student Name", None));
        __qtablewidgetitem4 = self.std_table.horizontalHeaderItem(4)
        __qtablewidgetitem4.setText(QCoreApplication.translate("MainWindow",
        u"DOB", None));
        __qtablewidgetitem5 = self.std_table.horizontalHeaderItem(5)
        __qtablewidgetitem5.setText(QCoreApplication.translate("MainWindow",
        u"Gender", None));
        __qtablewidgetitem6 = self.std_table.horizontalHeaderItem(6)
        __qtablewidgetitem6.setText(QCoreApplication.translate("MainWindow",
        u"Contact No.", None));
        self.fetch_btn.setText(QCoreApplication.translate("MainWindow", u"Fetch
        Details", None))
        self.tabWidget.setTabText(self.tabWidget.indexOf(self.tab),
        QApplication.translate("MainWindow", u"Student Records", None))
        # retranslateUi

```

13.20 ui_stud_attend.py

```
# -*- coding: utf-8 -*-

#####
#####
## Form generated from reading UI file 'stud_attendVLOCdg.ui'
##
## Created by: Qt User Interface Compiler version 6.4.0
##
## WARNING! All changes made in this file will be lost when recompiling UI file!
#####
#####

from PySide6.QtCore import (QCoreApplication, QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt)
from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
    QFont, QFontDatabase, QGradient, QIcon,
    QImage, QKeySequence, QLinearGradient, QPainter,
    QPalette, QPixmap, QRadialGradient, QTransform)
from PySide6.QtWidgets import (QApplication, QCalendarWidget, QComboBox,
    QFormLayout,
    QFrame, QHBoxLayout, QHeaderView, QLabel,
    QMainWindow, QPushButton, QSizePolicy, QSpacerItem,
    QTabWidget, QTableWidget, QTableWidgetItem, QVBoxLayout,
    QWidget)

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.setObjectName():
            MainWindow.setObjectName(u"MainWindow")
            MainWindow.resize(800, 600)
            self.centralwidget = QWidget(MainWindow)
            self.centralwidget.setObjectName(u"centralwidget")
            self.horizontalLayout = QHBoxLayout(self.centralwidget)
            self.horizontalLayout.setObjectName(u"horizontalLayout")
            self.tabWidget = QTabWidget(self.centralwidget)
            self.tabWidget.setObjectName(u"tabWidget")
            self.tabWidget.setStyleSheet(u"/* Styling the Tab Widget */\n"
"QTabWidget {\n"
"    background-color: #f0f0f0;\n"
"    border: 2px solid black;\n"
"    border-radius: 10px;\n"
"    padding: 5px;\n"
"}\n"
"\n"
"\n"
"/* Styling the Tabs */\n"
"QTabBar::tab {\n"
"    background-color: #2196F3; /* Tab background color */\n"
```

```

" color: white;\n"
" border: 1px solid #c0c0c0;\n"
" padding: 10px;\n"
" border-radius: 5px;\n"
" min-width: 80px;\n"
" min-height: 30px;\n"
"\n"
" font: 14pt \"Segoe UI\";\n"
"}\n"
"\n"
"")
    self.tab = QWidget()
    self.tab.setObjectName(u"tab")
    self.tab.setStyleSheet(u"")
    self.verticalLayout = QVBoxLayout(self.tab)
    self.verticalLayout.setObjectName(u"verticalLayout")
    self.horizontalLayout_2 = QHBoxLayout()
    self.horizontalLayout_2.setObjectName(u"horizontalLayout_2")
    self.horizontalLayout_2.setContentsMargins(-1, 0, -1, -1)
    self.formFrame = QFrame(self.tab)
    self.formFrame.setObjectName(u"formFrame")
    self.formFrame.setStyleSheet(u"*{\n"
"font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/* Styling the QLineEdit */\n"
"QLineEdit {\n"
" background-color: #f4f4f4; /* Light grey background */\n"
" color: #333333; /* Dark text color */\n"
" border: 2px solid #c0c0c0; /* Border color */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 5px;\n"
"}\n"
"\n"
"/* Focused state (when the QLineEdit is selected) */\n"
"QLineEdit:focus {\n"
" border: 2px solid #2196F3; /* Blue border when focused */\n"
" background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state */\n"
"QLineEdit:disabled {\n"
" background-color: #e0e0e0; /* Grey background when disabled */\n"
" color: #888888; /* Light grey text when disabled */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Placeholder text style */\n"
"QLineEdit::placeholder {\n"

```

```

"    color: #888888; /* Light grey color for placeholder text */\n"
"    font-style: italic; /* Italicize the placeholder text */\n"
""

        "}\n"
"\n"
"/** Styling the text cursor */\n"
"QLineEdit::cursor {\n"
"    color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
""
)

    self.formLayout = QFormLayout(self.formFrame)
    self.formLayout.setObjectName(u"formLayout")
    self.formLayout.setContentsMargins(-1, 0, 0, 0)
    self.label_13 = QLabel(self.formFrame)
    self.label_13.setObjectName(u"label_13")

    self.formLayout.addWidget(0, QFormLayout.LabelRole, self.label_13)

    self.std_class_box = QComboBox(self.formFrame)
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.addItem("")
    self.std_class_box.setObjectName(u"std_class_box")
    self.std_class_box.setStyleSheet(u"/** Styling the QComboBox */\n"
"QComboBox {\n"
"    background-color: #f4f4f4; /* Light grey background */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 2px solid #c0c0c0; /* Border color */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 5px;\n"
"    font: 14pt \"Segoe UI\";\n"
"}\n"
"    min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
"/** Styling the combo box when focused */\n"
"QComboBox:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"

```



```

"/* Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
"    background-color: #e0e0e0; /* Grey background when disabled */\n"
"    color: #888888; /* Light grey text when disabled */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Arrow button of the combo box */\n"
"\n"
"\n"
"/* Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
"    background-color\n"
"        ": #ffffff; /* White background for the dropdown list */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
"    border-radius: 5px;\n"
"    padding: 5px;\n"
"    min-width: 150px;\n"
"}\n"
"\n"
"/* Hover effect on the dropdown list items */\n"
"QComboBox QAbstractItemView::item:hover {\n"
"    background-color: #2196F3; /* Blue background on hover */\n"
"    color: #ffffff; /* White text on hover */\n"
"}\n"
"\n"
"/* Selected item in the dropdown list */\n"
"QComboBox QAbstractItemView::item:selected {\n"
"    background-color: #64B5F6; /* Light blue background when selected */\n"
"    color: #ffffff; /* White text when selected */\n"
"}\n"
""
)

```

```

self.formLayout.addWidget(0, QFormLayout.FieldRole, self.std_class_box)

```

```

self.label_14 = QLabel(self.formFrame)
self.label_14.setObjectName(u"label_14")

```

```

self.formLayout.addWidget(1, QFormLayout.LabelRole, self.label_14)

```

```

self.std_section_box = QComboBox(self.formFrame)
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.addItem("")
self.std_section_box.setObjectName(u"std_section_box")

```



```

        self.std_section_box.setStyleSheet(u"/* Styling the QComboBox */\n"
"QComboBox {\n"
"    background-color: #f4f4f4; /* Light grey background */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 2px solid #c0c0c0; /* Border color */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 5px;\n"
"    font: 14pt \"Segoe UI\";\n"
"\n"
"    min-width: 150px; /* Minimum width of the combo box */\n"
"}\n"
"\n"
"/* Styling the combo box when focused */\n"
"QComboBox:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state for the combo box */\n"
"QComboBox:disabled {\n"
"    background-color: #e0e0e0; /* Grey background when disabled */\n"
"    color: #888888; /* Light grey text when disabled */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Arrow button of the combo box */\n"
"\n"
"\n"
"/* Styling the dropdown list */\n"
"QComboBox QAbstractItemView {\n"
"    background-color: #ffffff; /* White background for the dropdown list */\n"
"    color: #333333; /* Dark text color */\n"
"    border: 1px solid #c0c0c0; /* Border around the dropdown list */\n"
"    border-radius: 5px;\n"
"    padding: 5px;\n"
"    min-width: 150px;\n"
"}\n"
"\n"
"/* Hover effect on the dropdown list items */\n"
"QComboBox QAbstractItemView::item:hover {\n"
"    background-color: #2196F3; /* Blue background on hover */\n"
"    color: #ffffff; /* White text on hover */\n"
"}\n"
"\n"
"/* Selected item in the dropdown list */\n"
"QComboBox QAbstractItemView::item:selected {\n"
"    background-color: #64B5F6; /* Light blue background when selected */\n"
"    color: #ffffff; /* White text when selected */\n"
"}\n"

```

""")

```
self.formLayout.addWidget(1, QFormLayout.FieldRole, self.std_section_box)
```

```
self.horizontalLayout_2.addWidget(self.formFrame)
```

```
self.verticalLayout.addLayout(self.horizontalLayout_2)
```

```
self.horizontalLayout_4 = QHBoxLayout()
```

```
self.horizontalLayout_4.setObjectName(u"horizontalLayout_4")
```

```
self.horizontalLayout_4.setContentsMargins(-1, 0, -1, -1)
```

```
self.std_attend_table = QTableWidget(self.tab)
```

```
if (self.std_attend_table.columnCount() < 6):
```

```
    self.std_attend_table.setColumnCount(6)
```

```
    __qtablewidgetitem = QTableWidgetItem()
```

```
    self.std_attend_table.setHorizontalHeaderItem(0, __qtablewidgetitem)
```

```
    __qtablewidgetitem1 = QTableWidgetItem()
```

```
    self.std_attend_table.setHorizontalHeaderItem(1, __qtablewidgetitem1)
```

```
    __qtablewidgetitem2 = QTableWidgetItem()
```

```
    self.std_attend_table.setHorizontalHeaderItem(2, __qtablewidgetitem2)
```

```
    __qtablewidgetitem3 = QTableWidgetItem()
```

```
    self.std_attend_table.setHorizontalHeaderItem(3, __qtablewidgetitem3)
```

```
    __qtablewidgetitem4 = QTableWidgetItem()
```

```
    self.std_attend_table.setHorizontalHeaderItem(4, __qtablewidgetitem4)
```

```
    __qtablewidgetitem5 = QTableWidgetItem()
```

```
    self.std_attend_table.setHorizontalHeaderItem(5, __qtablewidgetitem5)
```

```
if (self.std_attend_table.rowCount() < 3):
```

```
    self.std_attend_table.setRowCount(3)
```

```
self.std_attend_table.setObjectName(u"std_attend_table")
```

```
font = QFont()
```

```
font.setPointSize(14)
```

```
font.setBold(False)
```

```
self.std_attend_table.setFont(font)
```

```
self.std_attend_table.setStyleSheet(u"QHeaderView::section {\n"
```

```
"background-color: rgb(0, 120, 212);\n"
```

```
" /* Green background for headers */\n"
```

```
" color: white; /* White text color */\n"
```

```
" font-weight: bold; /* Bold font for header text */\n"
```

```
" text-align: center; /* Center-align text */\n"
```

```
" \n"
```

```
" font: 700 14pt \"Segoe UI\";\n"
```

```
" border: 1px solid #ccc; /* Light gray border around header sections */\n"
```

```
" padding: 5px; /* Padding inside header cells */\n"
```

```
"}\n"
```

```
""")
```

```
self.horizontalLayout_4.addWidget(self.std_attend_table)
```

```

self.std_calender_box = QCalendarWidget(self.tab)
self.std_calender_box.setObjectName(u"std_calender_box")

self.horizontalLayout_4.addWidget(self.std_calender_box, 0, Qt.AlignRight)

self.verticalLayout.addLayout(self.horizontalLayout_4)

self.horizontalLayout_3 = QHBoxLayout()
self.horizontalLayout_3.setObjectName(u"horizontalLayout_3")
self.horizontalLayout_3.setContentsMargins(-1, 0, -1, -1)
self.horizontalSpacer = QSpacerItem(40, 20, QSizePolicy.Expanding,
QSizePolicy.Minimum)

self.horizontalLayout_3.addItem(self.horizontalSpacer)

self.fetch_btn = QPushButton(self.tab)
self.fetch_btn.setObjectName(u"fetch_btn")
self.fetch_btn.setStyleSheet(u"/* Styling the QPushButton */\n"
"QPushButton {\n"
"    /* Green background */\n"
"    background-color: rgb(0, 120, 212);\n"
"    color: white; /* White text color */\n"
"    border: 2px solid #4CAF50; /* Border color matches background */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 10px 20px; /* Vertical and horizontal padding */\n"
"    \n"
"    font: 14pt \"Segoe UI\"; /* Font size */\n"
"    min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
"/* Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
"    background-color: #45a049; /* Darker green background */\n"
"    border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
"/* Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
"    background-color: #388e3c; /* Even darker green */\n"
"    border-color: #388e3c; /* Dark border */\n"
"    padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
"    padding-left: 18px; /* Shift the"
"        button to the left slightly */\n"
"}\n"
"\n"
"/* Focused state (when the button is focused) */\n"
"QPushButton:focus {\n"
"    border: 2px solid #2196F3; /* Blue border when focused */\n"

```

```

"    outline: none; /* Removes the default outline */\n"
"}\n"
"\n"
"/* Disabled state for the QPushButton */\n"
"QPushButton:disabled {\n"
"    background-color: #e0e0e0; /* Light grey background */\n"
"    color: #888888; /* Light grey text */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Styling the text of the QPushButton */\n"
"QPushButton {\n"
"    font-weight: bold; /* Make the text bold */\n"
"}\n"
"\n"
"/* Styling the QPushButton icon (if it has one) */\n"
"QPushButton::icon {\n"
"    padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"
"\n"
"/* Border effect when the button is in a focus or pressed state */\n"
"QPushButton:focus, QPushButton:pressed {\n"
"    border-style: inset;\n"
"}\n"
""
)

self.horizontalLayout_3.addWidget(self.fetch_btn)

self.save_btn = QPushButton(self.tab)
self.save_btn.setObjectName(u"save_btn")
self.save_btn.setStyleSheet(u"/* Styling the QPushButton */\n"
"QPushButton {\n"
"background-color: rgb(0, 120, 212);\n"
"    /* Green background */\n"
"    color: white; /* White text color */\n"
"    border: 2px solid #4CAF50; /* Border color matches background */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 10px 20px; /* Vertical and horizontal padding */\n"
"    \n"
"    font: 14pt \"Segoe UI\"; /* Font size */\n"
"    min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
"/* Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
"    background-color: #45a049; /* Darker green background */\n"
"    border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
"/* Pressed effect when the button is clicked */\n"

```

```

QPushButton:pressed {\n"
"   background-color: #388e3c; /* Even darker green */\n"
"   border-color: #388e3c; /* Dark border */\n"
"   padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\"
effect */\n"
"   padding-left: 18px; /* Shift t"
           "he button to the left slightly */\n"
"}\n"
"\n"
/* Focused state (when the button is focused) */\n"
QPushButton:focus {\n"
"   border: 2px solid #2196F3; /* Blue border when focused */\n"
"   outline: none; /* Removes the default outline */\n"
"}\n"
"\n"
/* Disabled state for the QPushButton */\n"
QPushButton:disabled {\n"
"   background-color: #e0e0e0; /* Light grey background */\n"
"   color: #888888; /* Light grey text */\n"
"   border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
/* Styling the text of the QPushButton */\n"
QPushButton {\n"
"   font-weight: bold; /* Make the text bold */\n"
"}\n"
"\n"
/* Styling the QPushButton icon (if it has one) */\n"
QPushButton::icon {\n"
"   padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"
"\n"
/* Border effect when the button is in a focus or pressed state */\n"
QPushButton:focus, QPushButton:pressed {\n"
"   border-style: inset;\n"
"}\n"
""
)

```

```

self.horizontalLayout_3.addWidget(self.save_btn)

self.verticalLayout.addLayout(self.horizontalLayout_3)

self.verticalSpacer = QSpacerItem(20, 40, QSizePolicy.Minimum,
QSizePolicy.Expanding)

self.verticalLayout.addItem(self.verticalSpacer)

self.tabWidget.addTab(self.tab, "")

```

```

self.horizontalLayout.addWidget(self.tabWidget)

MainWindow.setCentralWidget(self.centralwidget)

self.retranslateUi(MainWindow)

self.tabWidget.setCurrentIndex(0)


QMetaObject.connectSlotsByName(MainWindow)
# setupUi

def retranslateUi(self, MainWindow):
    MainWindow.setWindowTitle(QCoreApplication.translate("MainWindow",
u"Student Attendance", None))
    self.label_13.setText(QCoreApplication.translate("MainWindow",
u"Standard", None))
    self.std_class_box.setItemText(0,
QCoreApplication.translate("MainWindow", u"Class 1", None))
    self.std_class_box.setItemText(1,
QCoreApplication.translate("MainWindow", u"Class 2", None))
    self.std_class_box.setItemText(2,
QCoreApplication.translate("MainWindow", u"Class 3", None))
    self.std_class_box.setItemText(3,
QCoreApplication.translate("MainWindow", u"Class 4", None))
    self.std_class_box.setItemText(4,
QCoreApplication.translate("MainWindow", u"Class 5", None))
    self.std_class_box.setItemText(5,
QCoreApplication.translate("MainWindow", u"Class 6", None))
    self.std_class_box.setItemText(6,
QCoreApplication.translate("MainWindow", u"Class 7", None))
    self.std_class_box.setItemText(7,
QCoreApplication.translate("MainWindow", u"Class 8", None))
    self.std_class_box.setItemText(8,
QCoreApplication.translate("MainWindow", u"Class 9", None))
    self.std_class_box.setItemText(9,
QCoreApplication.translate("MainWindow", u"Class 10", None))
    self.std_class_box.setItemText(10,
QCoreApplication.translate("MainWindow", u"Class 11", None))
    self.std_class_box.setItemText(11,
QCoreApplication.translate("MainWindow", u"Class 12", None))

    self.label_14.setText(QCoreApplication.translate("MainWindow",
u"Division", None))
    self.std_section_box.setItemText(0,
QCoreApplication.translate("MainWindow", u"A", None))
    self.std_section_box.setItemText(1,
QCoreApplication.translate("MainWindow", u"B", None))
    self.std_section_box.setItemText(2,
QCoreApplication.translate("MainWindow", u"C", None))

```

```

        self.std_section_box.setItemText(3,
        QCoreApplication.translate("MainWindow", u"D", None))
        self.std_section_box.setItemText(4,
        QCoreApplication.translate("MainWindow", u"E", None))
        self.std_section_box.setItemText(5,
        QCoreApplication.translate("MainWindow", u"F", None))
        self.std_section_box.setItemText(6,
        QCoreApplication.translate("MainWindow", u"G", None))

        __qtablewidgetitem = self.std_attend_table.horizontalHeaderItem(0)
        __qtablewidgetitem.setText(QCoreApplication.translate("MainWindow",
        u"Standard", None));
        __qtablewidgetitem1 = self.std_attend_table.horizontalHeaderItem(1)
        __qtablewidgetitem1.setText(QCoreApplication.translate("MainWindow",
        u"Division", None));
        __qtablewidgetitem2 = self.std_attend_table.horizontalHeaderItem(2)
        __qtablewidgetitem2.setText(QCoreApplication.translate("MainWindow",
        u"Date", None));
        __qtablewidgetitem3 = self.std_attend_table.horizontalHeaderItem(3)
        __qtablewidgetitem3.setText(QCoreApplication.translate("MainWindow",
        u"Student ID", None));
        __qtablewidgetitem4 = self.std_attend_table.horizontalHeaderItem(4)
        __qtablewidgetitem4.setText(QCoreApplication.translate("MainWindow",
        u"Student Name", None));
        __qtablewidgetitem5 = self.std_attend_table.horizontalHeaderItem(5)
        __qtablewidgetitem5.setText(QCoreApplication.translate("MainWindow",
        u"Attendance", None));
        self.fetch_btn.setText(QCoreApplication.translate("MainWindow", u"Fetch
        Attendance", None))
        self.save_btn.setText(QCoreApplication.translate("MainWindow", u"Save
        Attendance", None))
        self.tabWidget.setTabText(self.tabWidget.indexOf(self.tab),
        QCoreApplication.translate("MainWindow", u"Student Attendance", None))
        # retranslateUi

```


13.21 ui_stud_fees.py

```
# -*- coding: utf-8 -*-

#####
#####
## Form generated from reading UI file 'stud_feesytQPdv.ui'
##
## Created by: Qt User Interface Compiler version 6.4.0
##
## WARNING! All changes made in this file will be lost when recompiling UI file!
#####
#####

from PySide6.QtCore import (QCoreApplication, QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt)
from PySide6.QtGui import (QBrush, QColor, QConicalGradient, QCursor,
    QFont, QFontDatabase, QGradient, QIcon,
    QImage, QKeySequence, QLinearGradient, QPainter,
    QPalette, QPixmap, QRadialGradient, QTransform)
from PySide6.QtWidgets import (QApplication, QFormLayout, QFrame,
    QHBoxLayout,
    QHeaderView, QLabel, QLineEdit, QMainWindow,
    QPushButton, QSizePolicy, QSpacerItem, QTabWidget,
    QTableWidgetItem, QTableWidgetItem, QVBoxLayout, QWidget)

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.setObjectName():
            MainWindow.setObjectName(u"MainWindow")
        MainWindow.resize(800, 600)
        self.centralwidget = QWidget(MainWindow)
        self.centralwidget.setObjectName(u"centralwidget")
        self.horizontalLayout = QHBoxLayout(self.centralwidget)
        self.horizontalLayout.setObjectName(u"horizontalLayout")
        self.tabWidget = QTabWidget(self.centralwidget)
        self.tabWidget.setObjectName(u"tabWidget")
        self.tabWidget.setStyleSheet(u"/* Styling the Tab Widget */\n"
"QTabWidget {\n"
"    background-color: #f0f0f0;\n"
"    border: 2px solid black;\n"
"    border-radius: 10px;\n"
"    padding: 5px;\n"
"}\n"
"\n"
"\n"
"/* Styling the Tabs */\n"
"QTabBar::tab {\n"
"    background-color: #2196F3; /* Tab background color */\n"
"    color: white;\n"
"
```



```

" border: 1px solid #c0c0c0;\n"
" padding: 10px;\n"
" border-radius: 5px;\n"
" min-width: 80px;\n"
" min-height: 30px;\n"
"\n"
" font: 14pt \"Segoe UI\";\n"
"}\n"
"\n"
""))
    self.tab = QWidget()
    self.tab.setObjectName(u"tab")
    self.tab.setStyleSheet(u"")
    self.verticalLayout = QVBoxLayout(self.tab)
    self.verticalLayout.setObjectName(u"verticalLayout")
    self.horizontalLayout_2 = QHBoxLayout()
    self.horizontalLayout_2.setObjectName(u"horizontalLayout_2")
    self.horizontalLayout_2.setContentsMargins(-1, 0, -1, -1)
    self.formFrame = QFrame(self.tab)
    self.formFrame.setObjectName(u"formFrame")
    self.formFrame.setStyleSheet(u"*{\n"
"font: 14pt \"Segoe UI\";\n"
"\n"
"}\n"
"\n"
"/* Styling the QLineEdit */\n"
"QLineEdit {\n"
" background-color: #f4f4f4; /* Light grey background */\n"
" color: #333333; /* Dark text color */\n"
" border: 2px solid #c0c0c0; /* Border color */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 5px;\n"
"}\n"
"\n"
"/* Focused state (when the QLineEdit is selected) */\n"
"QLineEdit:focus {\n"
" border: 2px solid #2196F3; /* Blue border when focused */\n"
" background-color: #ffffff; /* White background on focus */\n"
"}\n"
"\n"
"/* Disabled state */\n"
"QLineEdit:disabled {\n"
" background-color: #e0e0e0; /* Grey background when disabled */\n"
" color: #888888; /* Light grey text when disabled */\n"
" border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
"/* Placeholder text style */\n"
"QLineEdit::placeholder {\n"
" color: #888888; /* Light grey color for placeholder text */\n"

```

```

" font-style: italic; /* Italicize the placeholder text */\n"
""

        "}\n"
"\n"
/* Styling the text cursor */\n"
"QLineEdit::cursor {\n"
" color: #2196F3; /* Change cursor color to blue */\n"
"}\n"
""
)

    self.formLayout = QFormLayout(self.formFrame)
    self.formLayout.setObjectName(u"formLayout")
    self.formLayout.setContentsMargins(-1, 0, 0, 0)
    self.label_14 = QLabel(self.formFrame)
    self.label_14.setObjectName(u"label_14")

    self.formLayout.addWidget(0, QFormLayout.LabelRole, self.label_14)

    self.fees_id_box = QLineEdit(self.formFrame)
    self.fees_id_box.setObjectName(u"fees_id_box")

    self.formLayout.addWidget(0, QFormLayout.FieldRole, self.fees_id_box)

    self.horizontalLayout_5 = QHBoxLayout()
    self.horizontalLayout_5.setObjectName(u"horizontalLayout_5")
    self.horizontalLayout_5.setContentsMargins(-1, 0, -1, -1)
    self.lineEdit = QLineEdit(self.formFrame)
    self.lineEdit.setObjectName(u"lineEdit")

    self.horizontalLayout_5.addWidget(self.lineEdit)

    self.entry_btn = QPushButton(self.formFrame)
    self.entry_btn.setObjectName(u"entry_btn")
    self.entry_btn.setStyleSheet(u"/* Styling the QPushButton */\n"
"QPushButton {\n"
" background-color: #4CAF50; /* Green background */\n"
" color: white; /* White text color */\n"
" border: 2px solid #4CAF50; /* Border color matches background */\n"
" border-radius: 5px; /* Rounded corners */\n"
" padding: 10px 20px; /* Vertical and horizontal padding */\n"
" \n"
" font: 14pt \"Segoe UI\"; /* Font size */\n"
" min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
/* Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
" background-color: #45a049; /* Darker green background */\n"
" border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"

```

```

"/ * Pressed effect when the button is clicked */
QPushButton:pressed {
    background-color: #388e3c; /* Even darker green */
    border-color: #388e3c; /* Dark border */
    padding-top: 12px; /* Slightly reduces padding to simulate a "pressed"
effect */
    padding-left: 18px; /* Shift the button to the
left slightly */
}
\n
\n
"/ * Focused state (when the button is focused) */
QPushButton:focus {
    border: 2px solid #2196F3; /* Blue border when focused */
    outline: none; /* Removes the default outline */
}
\n
\n
"/ * Disabled state for the QPushButton */
QPushButton:disabled {
    background-color: #e0e0e0; /* Light grey background */
    color: #888888; /* Light grey text */
    border: 2px solid #cccccc; /* Light grey border */
}
\n
\n
"/ * Styling the text of the QPushButton */
QPushButton {
    font-weight: bold; /* Make the text bold */
}
\n
\n
"/ * Styling the QPushButton icon (if it has one) */
QPushButton::icon {
    padding-right: 10px; /* Space between the icon and the text */
}
\n
\n
"/ * Border effect when the button is in a focus or pressed state */
QPushButton:focus, QPushButton:pressed {
    border-style: inset;
}
\n
""

```

```

self.horizontalLayout_5.addWidget(self.entry_btn)

```

```

self.formLayout.setLayout(1, QFormLayout.FieldRole,
self.horizontalLayout_5)

```

```

self.label = QLabel(self.formFrame)
self.label.setObjectName(u"label")

```

```

self.formLayout.addWidget(1, QFormLayout.LabelRole, self.label)

```

```

self.horizontalLayout_2.addWidget(self.formFrame, 0, Qt.AlignTop)

self.verticalLayout.addLayout(self.horizontalLayout_2)

self.horizontalLayout_4 = QHBoxLayout()
self.horizontalLayout_4.setObjectName(u"horizontalLayout_4")
self.horizontalLayout_4.setContentsMargins(-1, 0, -1, -1)
self.fees_table = QTableWidgetItem()
if (self.fees_table.columnCount() < 6):
    self.fees_table.setColumnCount(6)
    __qtablewidgetitem = QTableWidgetItem()
    self.fees_table.setHorizontalHeaderItem(0, __qtablewidgetitem)
    __qtablewidgetitem1 = QTableWidgetItem()
    self.fees_table.setHorizontalHeaderItem(1, __qtablewidgetitem1)
    __qtablewidgetitem2 = QTableWidgetItem()
    self.fees_table.setHorizontalHeaderItem(2, __qtablewidgetitem2)
    __qtablewidgetitem3 = QTableWidgetItem()
    self.fees_table.setHorizontalHeaderItem(3, __qtablewidgetitem3)
    __qtablewidgetitem4 = QTableWidgetItem()
    self.fees_table.setHorizontalHeaderItem(4, __qtablewidgetitem4)
    __qtablewidgetitem5 = QTableWidgetItem()
    self.fees_table.setHorizontalHeaderItem(5, __qtablewidgetitem5)
if (self.fees_table.rowCount() < 3):
    self.fees_table.setRowCount(3)
self.fees_table.setObjectName(u"fees_table")
font = QFont()
font.setPointSize(14)
font.setBold(False)
self.fees_table.setFont(font)
self.fees_table.setStyleSheet(u"QHeaderView::section {\n"
"background-color: rgb(0, 120, 212);\n"
" /* Green background for headers */\n"
" color: white; /* White text color */\n"
" font-weight: bold; /* Bold font for header text */\n"
" text-align: center; /* Center-align text */\n"
" \n"
" font: 700 14pt \"Segoe UI\";\n"
" border: 1px solid #ccc; /* Light gray border around header sections */\n"
" padding: 5px; /* Padding inside header cells */\n"
"}\n"
"")

self.horizontalLayout_4.addWidget(self.fees_table)

self.verticalLayout.addLayout(self.horizontalLayout_4)

self.horizontalLayout_3 = QHBoxLayout()

```

```

self.horizontalLayout_3.setObjectName(u"horizontalLayout_3")
self.horizontalLayout_3.setContentsMargins(-1, 0, -1, -1)
self.horizontalSpacer = QSpacerItem(40, 20, QSizePolicy.Expanding,
QSizePolicy.Minimum)

self.horizontalLayout_3.addItem(self.horizontalSpacer)

self.fetch_btn = QPushButton(self.tab)
self.fetch_btn.setObjectName(u"fetch_btn")
self.fetch_btn.setStyleSheet(u"/* Styling the QPushButton */\n"
"QPushButton {\n"
"  background-color: #4CAF50; /* Green background */\n"
"  color: white; /* White text color */\n"
"  border: 2px solid #4CAF50; /* Border color matches background */\n"
"  border-radius: 5px; /* Rounded corners */\n"
"  padding: 10px 20px; /* Vertical and horizontal padding */\n"
"  \n"
"  font: 14pt \"Segoe UI\"; /* Font size */\n"
"  min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
"/* Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
"  background-color: #45a049; /* Darker green background */\n"
"  border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
"/* Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
"  background-color: #388e3c; /* Even darker green */\n"
"  border-color: #388e3c; /* Dark border */\n"
"  padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
"  padding-left: 18px; /* Shift the button to the left slightly */\n"
"}\n"
"\n"
"/* Focused state (when the button is focused) */\n"
"QPushButton:focus {\n"
"  border: 2px solid #2196F3; /* Blue border when focused */\n"
"  outline: none; /* Removes the default outline */\n"
"}\n"
"\n"
"/* Disabled state for the QPushButton */\n"
"QPushButton:disabled {\n"
"  background-color: #e0e0e0; /* Light grey background */\n"
"  color: #888888; /* Light grey text */\n"
"  border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"

```

```

"/ * Styling the text of the QPushButton */\n"
"QPushButton {\n"
"    font-weight: bold; /* Make the text bold */\n"
"}\n"
"\n"
"/ * Styling the QPushButton icon (if it has one) */\n"
"QPushButton::icon {\n"
"    padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"
"\n"
"/ * Border effect when the button is in a focus or pressed state */\n"
"QPushButton:focus, QPushButton:pressed {\n"
"    border-style: inset;\n"
"}\n"
""")

self.horizontalLayout_3.addWidget(self.fetch_btn)

self.save_btn = QPushButton(self.tab)
self.save_btn.setObjectName(u"save_btn")
self.save_btn.setStyleSheet(u"/ * Styling the QPushButton */\n"
"QPushButton {\n"
"    background-color: #4CAF50; /* Green background */\n"
"    color: white; /* White text color */\n"
"    border: 2px solid #4CAF50; /* Border color matches background */\n"
"    border-radius: 5px; /* Rounded corners */\n"
"    padding: 10px 20px; /* Vertical and horizontal padding */\n"
"    \n"
"    font: 14pt \"Segoe UI\"; /* Font size */\n"
"    min-width: 100px; /* Minimum width for the button */\n"
"}\n"
"\n"
"/ * Hover effect when the mouse is over the button */\n"
"QPushButton:hover {\n"
"    background-color: #45a049; /* Darker green background */\n"
"    border-color: #45a049; /* Darker green border */\n"
"}\n"
"\n"
"/ * Pressed effect when the button is clicked */\n"
"QPushButton:pressed {\n"
"    background-color: #388e3c; /* Even darker green */\n"
"    border-color: #388e3c; /* Dark border */\n"
"    padding-top: 12px; /* Slightly reduces padding to simulate a \"pressed\" effect */\n"
"    padding-left: 18px; /* Shift the button to the left slightly */\n"
"}\n"
"\n"
"/ * Focused state (when the button is focused) */\n"
"QPushButton:focus {\n"

```

```

"    border: 2px solid #2196F3; /* Blue border when focused */\n"
"    outline: none; /* Removes the default outline */\n"
"}\n"
"\n"
/* Disabled state for the QPushButton */\n"
"QPushButton:disabled {\n"
"    background-color: #e0e0e0; /* Light grey background */\n"
"    color: #888888; /* Light grey text */\n"
"    border: 2px solid #cccccc; /* Light grey border */\n"
"}\n"
"\n"
/* Styling the text of the QPushButton */\n"
"QPushButton {\n"
"    font-weight: bold; /* Make the text bold */\n"
"}\n"
"\n"
/* Styling the QPushButton icon (if it has one) */\n"
"QPushButton:icon {\n"
"    padding-right: 10px; /* Space between the icon and the text */\n"
"}\n"
"\n"
/* Border effect when the button is in a focus or pressed state */\n"
"QPushButton:focus, QPushButton:pressed {\n"
"    border-style: inset;\n"
"}\n"
""
)

```

```

    self.horizontalLayout_3.addWidget(self.save_btn)

    self.verticalLayout.addLayout(self.horizontalLayout_3)

    self.verticalSpacer = QSpacerItem(20, 40, QSizePolicy.Minimum,
    QSizePolicy.Expanding)

    self.verticalLayout.addItem(self.verticalSpacer)

    self.tabWidget.addTab(self.tab, "")

    self.horizontalLayout.addWidget(self.tabWidget)

    MainWindow.setCentralWidget(self.centralwidget)

    self.retranslateUi(MainWindow)

    self.tabWidget.setCurrentIndex(0)


    QMetaObject.connectSlotsByName(MainWindow)
# setupUi

```

```

def retranslateUi(self, MainWindow):
    MainWindow.setWindowTitle(QCoreApplication.translate("MainWindow",
u"Student Fees", None))
    self.label_14.setText(QCoreApplication.translate("MainWindow", u"Student
ID", None))
    self.entry_btn.setText(QCoreApplication.translate("MainWindow",
u"Entry", None))
    self.label.setText(QCoreApplication.translate("MainWindow", u"Amount",
None))
    __qtablewidgetitem = self.fees_table.horizontalHeaderItem(0)
    __qtablewidgetitem.setText(QCoreApplication.translate("MainWindow",
u"Date", None));
    __qtablewidgetitem1 = self.fees_table.horizontalHeaderItem(1)
    __qtablewidgetitem1.setText(QCoreApplication.translate("MainWindow",
u"Student ID", None));
    __qtablewidgetitem2 = self.fees_table.horizontalHeaderItem(2)
    __qtablewidgetitem2.setText(QCoreApplication.translate("MainWindow",
u"Class", None));
    __qtablewidgetitem3 = self.fees_table.horizontalHeaderItem(3)
    __qtablewidgetitem3.setText(QCoreApplication.translate("MainWindow",
u"Section", None));
    __qtablewidgetitem4 = self.fees_table.horizontalHeaderItem(4)
    __qtablewidgetitem4.setText(QCoreApplication.translate("MainWindow",
u"Student Name", None));
    __qtablewidgetitem5 = self.fees_table.horizontalHeaderItem(5)
    __qtablewidgetitem5.setText(QCoreApplication.translate("MainWindow",
u"Fees Paid", None));
    self.fetch_btn.setText(QCoreApplication.translate("MainWindow", u"Fetch
Fees History", None))
    self.save_btn.setText(QCoreApplication.translate("MainWindow", u"Save
&& Exit", None))
    self.tabWidget.setTabText(self.tabWidget.indexOf(self.tab),
QCoreApplication.translate("MainWindow", u"Student Fees", None))
    # retranslateUi

```


14. CONCLUSION

The School Management System successfully achieves its objective of automating and simplifying various school operations. The system integrates multiple functionalities such as student management, staff handling, attendance tracking, marks management, and fee collection into a single, user-friendly platform.

By replacing manual record-keeping with a digital system, the application reduces errors, saves time, and improves overall efficiency. The use of a structured database ensures data consistency, while the graphical user interface makes the system easy to use for both technical and non-technical users.

The modular design of the system allows easy maintenance and future enhancements. It also provides a strong foundation for further development, such as adding authentication, cloud integration, and web-based access.

In conclusion, the project demonstrates how technology can be effectively used to improve administrative processes in educational institutions. It serves as a practical and scalable solution for managing school data in an organized and efficient manner.